

Beej의 네트워크 프로그래밍 안내서

인터넷 소켓 사용법

Brian "Beej Jorgensen" Hall

v3.3.2, Copyright © April 18, 2026

Contents

1	도입부	1
1.1	읽는이에게	1
1.2	플랫폼과 컴파일러	1
1.3	공식 홈페이지와 책 구매	1
1.4	Solaris/SunOS/illumos 프로그래머들을 위한 노트	2
1.5	Windows 프로그래머들을 위한 노트	2
1.6	이메일 정책	4
1.7	미러링	4
1.8	번역자를 위한 노트	5
1.9	Copyright, Distribution, and Legal	5
1.10	헌사	5
1.11	출판 정보	6
1.12	옮긴이의 말: 2023년 번역, 원문 3.1.5 기반 한국어판	6
1.13	옮긴이의 말: 2026년 개정, 원문 3.3.2 기반 한국어판	6
2	소켓이란 무엇인가?	9
2.1	두 종류의 인터넷 소켓	9
2.2	저수준 넌센스와 네트워크 이론	11
3	IP 주소, struct, 그리고 데이터 다루기	13
3.1	IP 주소: 버전 4와 6	13
3.1.1	서브넷	14
3.1.2	포트 번호	15
3.2	바이트 순서	15
3.3	struct	17
3.4	IP 주소, 파트 2	19
3.4.1	사설(또는 분리된) 망	21
4	IPv4에서 IPv6으로 넘어가기	23
5	시스템 콜이 아니면 죽음을	25
5.1	getaddrinfo() —발사 준비!	25
5.2	socket() —파일 설명자를 받자!	29
5.3	bind() —나는 어떤 포트에 있나요?	30
5.4	connect() —거기 안녕!	32
5.5	listen() —누가 연락 좀 해줄래요?	33
5.6	accept() —“3490 포트에 접속해주셔서 감사합니다.”	33
5.7	send() 와 recv() —말 좀 해봐요!	35
5.8	sendto() 와 recvfrom() —DGRAM 방식으로 말해 봐요	36
5.9	close() 와 shutdown() —이제 그만 가!	37
5.10	getpeername() —누구세요?	38

5.11	<code>gethostname()</code> —나는 누구인가?	38
6	클라이언트-서버 배경지식	41
6.1	단순한 스트림 서버	41
6.2	단순한 스트림 클라이언트	45
6.3	데이터그램 소켓	47
7	약간 더 고급스러운 기술	51
7.1	블로킹	51
7.2	<code>poll()</code> —동기 입출력 다중화	52
7.3	<code>select()</code> —동기화된 I/O 멀티플렉싱, 예전 방식	59
7.4	부분적인 <code>send()</code> 처리하기	67
7.5	직렬화—데이터를 포장하는 방법	68
7.6	망할 데이터 캡슐화	82
7.7	브로드캐스트(Broadcast) 패킷 —Hello, World!	84
8	일반적인 질문들	89
9	맨페이지	95
9.1	<code>accept()</code>	95
9.2	<code>bind()</code>	97
9.3	<code>connect()</code>	99
9.4	<code>close()</code>	100
9.5	<code>getaddrinfo()</code> , <code>freeaddrinfo()</code> , <code>gai_strerror()</code>	101
9.6	<code>gethostname()</code>	104
9.7	<code>gethostbyname()</code> , <code>gethostbyaddr()</code>	105
9.8	<code>getnameinfo()</code>	108
9.9	<code>getpeername()</code>	109
9.10	<code>errno</code>	110
9.11	<code>fcntl()</code>	111
9.12	<code>htons()</code> , <code>htonl()</code> , <code>ntohs()</code> , <code>ntohl()</code>	112
9.13	<code>inet_ntoa()</code> , <code>inet_aton()</code> , <code>inet_addr</code>	114
9.14	<code>inet_ntop()</code> , <code>inet_pton()</code>	115
9.15	<code>listen()</code>	117
9.16	<code>perror()</code> , <code>strerror()</code>	118
9.17	<code>poll()</code>	120
9.18	<code>recv()</code> , <code>recvfrom()</code>	122
9.19	<code>select()</code>	124
9.20	<code>setsockopt()</code> , <code>getsockopt()</code>	126
9.21	<code>send()</code> , <code>sendto()</code>	128
9.22	<code>shutdown()</code>	129
9.23	<code>socket()</code>	130
9.24	<code>struct sockaddr</code> 와 친구들	132
10	더 많은 참고문헌	135
10.1	책들	135

10.2	웹 참고문헌	135
10.3	RFCs	136

Chapter 1

도입부

안녕하세요! 소켓 프로그래밍 때문에 힘든가요? `man` 페이지로 공부하기가 너무 어려운가요? 멋진 인터넷 프로그래밍을 하고 싶지만 `connect()` 를 호출하기 전에 `bind()` 를 호출해야 하는지 알아내려고 한 무더기의 `struct` 를 헤집고 다닐 시간이 없나요?

그런데 제가 그 귀찮은 일을 이미 다 해냈습니다. 그리고 그 정보를 여러분에게 공유하고 싶어서 안달이 나 있습니다. 제대로 찾아오셨습니다. 이 문서는 보통 수준의 C 프로그래머가 귀찮은 네트워킹을 처리할 수 있게 도와줄 것입니다.

그리고 이것도 보세요. 제가 드디어 미래의 기술을 따라잡았고(정말 겨우 시간을 맞췄죠) IPv6에 대한 안내를 갱신했습니다. 재밌게 보세요!

1.1 읽는이에게

이 문서는 완전한 참고 문서가 아닌 튜토리얼로서 작성되었습니다. 아마도 이제 막 소켓 프로그래밍을 시작해서 발판이 필요한 사람이 읽기에 적합할 것입니다. 이것은 분명히 어떤 의미로든 완벽하고 완전한 소켓 프로그래밍 가이드는 아닙니다.

그럼에도 불구하고 이 문서를 읽고 나면 맨페이지가 이해되기 시작할 것입니다.

1.2 플랫폼과 컴파일러

이 문서에 포함된 코드는 GNU의 `gcc` 컴파일러를 사용하는 리눅스 PC에서 컴파일되었습니다. 그러나 그 코드들은 `gcc` 를 사용하는 거의 모든 플랫폼에서 빌드될 것입니다. 다만 Windows용 프로그램을 만들고 있다면 해당 사항이 없습니다. 그런 경우라면 윈도우 프로그래밍을 위한 절을 참고하세요.

1.3 공식 홈페이지와 책 구매

이 문서의 공식 위치는 아래와 같습니다.

- <https://beej.us/guide/bgnet/>

이곳에서 예제 코드와 이 안내서의 여러 언어로 된 번역본도 찾을 수 있습니다.

(사람들이 책이라고 부르는) 잘 제본된 인쇄본을 사고 싶다면 아래 링크에 방문하세요.

- <https://beej.us/guide/url/bgbuy>

책을 구매해주신다면 글을 써서 먹고 사는 라이프 스타일을 유지하는 일에 도움이 되므로 감사하겠습니다.

1.4 Solaris/SunOS/illumos 프로그래머들을 위한 노트

Solaris 변종이나 SunOS를 위해서 컴파일할 때, 정확한 라이브러리에 링크하기 위해서 약간의 추가적인 명령줄 스위치를 지정해야 합니다. 그러기 위해서 컴파일 명령의 끝에 “-lnsl -lsocket -lresolv”를 아래와 같이 덧붙이세요.

```
$ cc -o server server.c -lnsl -lsocket -lresolv
```

여전히 오류가 있다면, 그 명령 뒤에 -lnet를 덧붙여 보세요. 그것이 정확히 무엇을 하는지 저는 모르지만, 몇몇 사람들은 그렇게 해야 했던 것으로 보입니다.

setsockopt()을 호출하는 곳에서도 문제가 생길 수 있습니다. 제가 쓰는 리눅스 시스템과는 함수 원형이 다릅니다. 그러므로 아래의 코드 대신

```
int yes=1;
```

이렇게 입력하세요.

```
char yes='1';
```

저에게 Sun 시스템이 없으므로 위에 적은 내용들을 시험해 보지는 않았습니다. 저 내용들은 단지 사람들이 저에게 이메일로 알려준 것입니다.

1.5 Windows 프로그래머들을 위한 노트

역사적으로 안내서의 이 부분에서 저는 제가 그다지 좋아하지 않는다는 이유로 Windows를 조금 놀려 왔습니다. 하지만 그 뒤 Windows와 Microsoft라는 회사는 많이 나아졌습니다. Windows 10에 아래에서 말할 WSL을 결들이면 꽤 괜찮은 운영체제가 됩니다. 딱히 크게 불평할 것도 없습니다.

음, 조금은 있습니다. 예를 들어 저는 이 글을 2025년에 2015년형 노트북에서 쓰고 있습니다. 예전에는 Windows 10을 돌리던 컴퓨터였습니다. 결국 너무 느려져서 Linux를 설치했고, 그 뒤로 계속 그렇게 쓰고 있습니다.

그런데 이제 Windows 11은 Windows 10보다 더 강력한 하드웨어를 요구하는 것 같습니다. 저는 그 점이 마음에 들지 않습니다. 운영체제는 가능한 한 뒤로 물러나 있어야 하고, 여러분에게 돈을 더 쓰라고 요구해서는 안 됩니다. 추가 CPU 성능은 운영체제가 아니라 응용프로그램을 위한 것이어야 합니다! 게다가 Microsoft는 여러분이 더 많은 광고를 원한다고 생각하는 모양입니다. 그것도 운영체제 안에요! 광고가 필요한가요? Windows 11이라면 가능합니다.

그래서... 저는 여전히 윈도우 대신 Linux¹, BSD², illumos³ 또는 다른 종류의 유닉스를 써 보라고 권하고 싶습니다.

갑자기 흥분해서 연설을 해버렸네요.

하지만 사람들은 하던 대로 하고 싶어 하기 마련이고, 윈도우를 쓰는 분들은 이 문서의 정보가 Windows에도 약간의 차이만 빼면 보통 적용된다는 것을 알면 기뻐할 것입니다.

여러분이 진지하게 고려해봐야 할 것은 Windows Subsystem for Linux⁴입니다. 이것은 간단히 말하자면 Windows 10에 리눅스 VM 비슷한 것을 깔게 해 줍니다. 그렇게 하면 개발 환경을 갖출 수 있고, 예제 프로그램을 있는 그대로 빌드하고 실행할 수 있습니다. (역자 주: WSL은 Windows 11에서도 사용할 수 있습니다.)

¹<https://www.linux.com/>

²<https://bsd.org/>

³<https://www.illumos.org/>

⁴<https://learn.microsoft.com/en-us/windows/wsl/>

여러분이 할 수 있는 다른 일은 Cygwin⁵을 설치하는 것입니다. 이것은 Windows를 위한 유닉스 도구 모음입니다. 그렇게 하면 예제 프로그램을 수정 없이 컴파일할 수 있다고 들었습니다만 직접 해 보지는 않았습니다.

여러분 중 몇몇은 순수한 Windows 방식으로 이 일을 하고 싶을지도 모르겠습니다. 그렇다면 아주 배짱이 두둑한 일이 되겠군요. 그렇게 하고 싶다면 당장 집 밖으로 가서 유닉스를 돌릴 컴퓨터를 사세요! 장난입니다. 요새는 윈도우에 (좀 더) 친화적으로 행동하려고 노력하고 있습니다.

좋습니다, 좋습니다. 이제 본론으로 돌아가겠습니다.

여러분이 할 일은 이렇습니다. 첫 번째로 제가 이 문서에서 언급하는 거의 모든 시스템 헤더 파일을 무시하세요. 그 대신 아래의 헤더 파일을 포함하세요.

```
#include <winsock2.h>
#include <ws2tcpip.h>
```

`winsock2` 는 "새로운"(대략 1994년 기준으로) 윈도우 소켓 라이브러리입니다.

불행하게도 여러분이 `windows.h` 를 인클루드하면 그것이 자동으로 버전1인 오래된 `winsock.h` 를 끌어오고 `winsock2.h` 와 충돌을 일으킬 것입니다. 정말 재밌지요.

그러므로 만약 `windows.h` 를 인클루드해야 한다면 그것이 오래된 헤더를 포함하지 않도록 아래의 매크로를 정의해야 합니다.

```
#define WIN32_LEAN_AND_MEAN // 이렇게 적으세요.

#include <windows.h> // 이제 이것을 인클루드해도 됩니다.
#include <winsock2.h> // 이것도 인클루드해도 됩니다.
```

잠깐! 소켓 라이브러리를 쓰기 전에 `WSAStartup()` 을 호출해야 합니다. 이 함수에 사용하려는 Winsock 버전(예를 들어 2.2)을 인수로 넘기고 반환값을 확인해서 쓰고자 하는 버전이 사용 가능한지 확인해야 합니다.

그 작업을 하는 코드는 아래와 비슷할 것입니다.

```
#include <winsock2.h>

{
    WSADATA wsaData;

    if (WSAStartup(MAKEWORD(2, 2), &wsaData) != 0) {
        fprintf(stderr, "WSAStartup failed.\n");
        exit(1);
    }

    if (LOBYTE(wsaData.wVersion) != 2 ||
        HIBYTE(wsaData.wVersion) != 2)
    {
        fprintf(stderr, "Version 2.2 of Winsock not available.\n");
        WSACleanup();
        exit(2);
    }
}
```

저기 보이는 `WSACleanup()` 호출부를 주목하세요. Winsock 라이브러리를 다 쓴 후에 이것을 호출해야 합니다.

⁵<https://cygwin.com/>

또한 컴파일러에게 `ws2_32.lib` 라는 Winsock2 라이브러리를 링크하라고 말해줘야 합니다. VC++에서는 프로젝트 메뉴에서 설정으로 가서 링크 탭을 클릭하고 “오브젝트/라이브러리 모듈”이라는 제목이 붙은 상자를 찾으세요. 그리고 거기에 “ws2_32.lib”나 여러분이 원하는 다른 라이브러리를 추가하세요. (역자 주: 최신 Visual Studio에서는 이 링크를 참고하세요.)

직접 해 본 것은 아닙니다.

그렇게 하고 나면, 이 튜토리얼의 나머지 예제들은 거의 그대로 쓸 수 있을 것입니다. 몇 가지 예외가 있는데 소켓을 닫기 위해서 `close()` 를 쓸 수 없습니다. 대신 `closesocket()` 을 써야 합니다. 또한 `select()` 는 파일 설명자가 아닌 소켓 설명자에만 쓸 수 있습니다. (`stdin` 의 `0` 같은 파일 설명자)

여러분이 쓸 수 있는 소켓 클래스도 있습니다. `CSocket` 입니다. 자세한 정보는 컴파일러의 도움말 페이지를 참고하세요.

Winsock에 대한 정보를 더 얻고 싶다면 마이크로소프트의 공식 홈페이지를 참고하세요.

마지막으로 윈도우에는 `fork()` 가 없다고 들었습니다. 불행히도 제 예제 코드 중 일부는 `fork()` 를 사용합니다. 아마 그것을 동작하게 하려면 POSIX 라이브러리에 링크하거나 다른 작업이 필요할 것입니다. 아니면 `CreateProcess()` 를 대신 쓸 수도 있습니다. `fork()` 는 인수를 받지 않지만 `CreateProcess()` 는 인수를 480억 개 정도 받습니다. 그게 부담스럽다면 `CreateThread()` 이 조금 더 쓰기 쉬울 겁니다. 불행히도 멀티스레딩에 대한 논의는 이 문서의 범위를 벗어납니다. 저는 이 정도까지밖에 말씀드릴 수가 없습니다.

정말 마지막으로, Steven Mitchell이 Winsock으로 변환한 많은 예제⁶도 확인해 볼 만합니다.

1.6 이메일 정책

저는 대체로 이메일로 오는 문의사항에 답을 드리고자 하니 이메일 보내기를 주저하지 마세요. 그러나 응답을 보장하지는 못합니다. 저는 바쁜 삶을 살고 있고 제가 여러분이 가진 궁금증에 대답할 수 없는 때가 많이 있습니다. 그런 경우라면 저는 그 메시지를 그냥 삭제합니다. 개인적인 감정이 아닙니다. 그저 여러분이 필요로 하는 자세한 답을 할 시간이 없을 것이라 생각하기 때문입니다.

원칙적으로 질문이 복잡할수록 제가 응답할 가능성은 낮아질 것입니다. 메일을 보내기 전에 질문의 범위를 좁히고 적절한 정보(플랫폼, 컴파일러, 받은 오류 메시지, 문제 해결에 도움이 될 만한 다른 정보)를 첨부한다면 제 응답을 받을 확률이 올라갈 것입니다. 더 자세한 지침은 ESR의 문서인 *How To Ask Questions The Smart Way*⁷를 참고하세요.

여러분이 제 회신을 받지 못한다면, 문제를 더 파고들어보고, 답을 찾기 위해 노력해보세요. 그래도 확실한 답을 얻지 못했다면 그동안 알아낸 정보를 가지고 저에게 다시 메일을 보내세요. 어쩌면 제가 답을 드릴 수 있을지도 모릅니다.

저에게 메일을 보낼 때 이렇게 해라 저렇게 해라 말이 많았습니다만 이 안내서에 지난 몇 년 동안 보내주신 모든 칭찬에 정말로 감사한다는 사실을 말씀드리고 싶습니다. 그것은 정말로 정신적인 힘이 됩니다. 이 안내서가 좋은 일에 쓰였다는 말을 듣는 일은 저를 기쁘게 합니다. :-) 감사합니다!

1.7 미러링

이 웹사이트를 공개로든 사적으로든 미러링하는 것은 얼마든지 환영합니다. 이 웹사이트를 공개적으로 미러링하고 제가 메인 페이지에 링크를 걸게 하고 싶다면 `beej@beej.us` 로 메일을 보내주세요.

⁶<https://www.tallyhawk.net/WinsockExamples/>

⁷<http://www.catb.org/~esr/faqs/smart-questions.html>

1.8 번역자를 위한 노트

이 안내서를 다른 언어로 번역하고 싶다면 `beej@beej.us` 에 메일을 보내주세요. 여러분의 번역본의 링크를 제 메인 페이지에 걸어두겠습니다. 여러분의 이름과 연락처 정보를 번역본에 추가하시는 것도 환영합니다.

이 원본 마크다운 문서는 UTF-8로 인코딩되었습니다.

아래의 Copyright, Distribution, and Legal 절에 있는 라이선스 제한 사항에 유의하세요.

제가 번역본을 호스트하길 바란다면 말씀해 주세요. 여러분이 호스트하길 원한다면 그것도 링크하겠습니다. 어느 쪽이든 좋습니다.

1.9 Copyright, Distribution, and Legal

(Translator's Note: This section has not been translated to keep its legal information.) (역자 주: 이 절은 법적 정보를 보존하기 위해 번역하지 않았습니다.)

Beej's Guide to Network Programming is Copyright © 2019 Brian "Beej Jorgensen" Hall.

With specific exceptions for source code and translations, below, this work is licensed under the Creative Commons Attribution- Noncommercial- No Derivative Works 3.0 License. To view a copy of this license, visit

<https://creativecommons.org/licenses/by-nc-nd/3.0/>

or send a letter to Creative Commons, 171 Second Street, Suite 300, San Francisco, California, 94105, USA.

One specific exception to the "No Derivative Works" portion of the license is as follows: this guide may be freely translated into any language, provided the translation is accurate, and the guide is reprinted in its entirety. The same license restrictions apply to the translation as to the original guide. The translation may also include the name and contact information for the translator.

The C source code presented in this document is hereby granted to the public domain, and is completely free of any license restriction.

Educators are freely encouraged to recommend or supply copies of this guide to their students.

Unless otherwise mutually agreed by the parties in writing, the author offers the work as-is and makes no representations or warranties of any kind concerning the work, express, implied, statutory or otherwise, including, without limitation, warranties of title, merchantability, fitness for a particular purpose, noninfringement, or the absence of latent or other defects, accuracy, or the presence of absence of errors, whether or not discoverable.

Except to the extent required by applicable law, in no event will the author be liable to you on any legal theory for any special, incidental, consequential, punitive or exemplary damages arising out of the use of the work, even if the author has been advised of the possibility of such damages.

Contact `beej@beej.us` for more information.

1.10 헌사

이 안내서를 쓸 수 있도록 저를 과거에 도와주신, 그리고 미래에 도와주실 모든 분들에게 감사드립니다. 이 안내서를 만들기 위해 사용한 자유 소프트웨어와 패키지(GNU, Linux, Slackware, vim, Python, Inkscape, pandoc, 기타 등등...)를 만든 모든 분들에게 감사드립니다. 또한 이 안내서의 발전을 위한 제안을 해주시고 응원의 말씀을 보내주신 수천 명의 사람들에게 감사드립니다.

이 안내서를 컴퓨터 세계에서 저의 가장 위대한 영웅이자 영감을 주는 이들인 Donald Knuth, Bruce Schneier, W. Richard Stevens, Steve The Woz Wozniak, 독자 여러분, 그리고 모든 자유 및 공개 소프트웨어 커뮤니티에 바칩니다.

1.11 출판 정보

이 책은 GNU 도구를 갖춘 Arch Linux 시스템에서 Vim 편집기를 사용해서 Markdown 으로 작성되었습니다. 표지 “미술”과 다이어그램은 Inkscape로 작성되었습니다. Markdown은 Python과 Pandoc, XeLaTeX를 통해 HTML과 LaTeX/PDF로 변환되었습니다. 문서에는 Liberation 폰트를 사용했습니다. 틀체인은 전적으로 자유/공개 소프트웨어를 사용해서 구성했습니다.

1.12 율긴이의 말: 2023년 번역, 원문 3.1.5 기반 한국어판

원문의 첫 한국어 번역 버전은 박성호(tempter@fourthline.com)님이 1998/08/20(yyyy/MM/d)에 인터넷의 어딘가에 게시하신 것으로 보입니다. 현재는 KLDP에 있습니다.

원문의 두 번째 한국어 번역 버전은 김수봉(연락처 없음)님이 2003/12/15(yyyy/MM/d)에 KLDP에 게시하셨습니다. 첫 번역 문서를 html에서 wiki형태로 변환했다고 적혀 있습니다.

지금 읽고 계시는 세 번째 버전은 정민석(javalia.javalia@gmail.com)이 작업했으며, 2023년 5월 28일부터 작업했습니다.

원문의 번역본은 1998년에 최초로 한국어 버전이 작성된 이래로 한국인이 Socket 프로그래밍에 대해서 참고할 수 있는 문서 중 리눅스 맨페이지를 제외하면 가장 1차적인 문서였습니다. 실제로 많은 소켓 프로그래밍 관련 글과 출판된 책의 예제 코드도 이 문서의 것을 차용하고 있습니다. 원문은 세월이 흐르면서 조금씩 개정되어 내용이 상세해지고 IPv6에 대해서도 다루고 있으나 번역본은 20년 전의 모습으로 멈추어 있었습니다. 소켓 프로그래밍을 공부하던 시절 가장 큰 도움을 받은 문서의 번역본이 개정되지 않음을 안타깝게 생각해서 이렇게 새로운 번역본을 만들게 되었습니다. 이 글이 새로운 네트워크 라이브러리를 개발하는 프로그래머에게 도움이 되기를 바랍니다.

이 문서는 원문의 3.1.5 버전을 기반으로 번역되었습니다. 원문에 등장하는 고유명사는 해당 명사에 통용되는 한국어 번역이 없는 한 원어 그대로 실었습니다. 영어 일반명사는 음역을 기본으로 하였으나, 일부는 의역하기도 하였고 혼동을 줄이기 위해서 병기한 부분도 있습니다. 작업 과정에서 이전 번역자들의 원문을 유지하지는 못했습니다. 원문/번역본/새 번역본 사이의 대조/교정 작업은 개인적인 시간을 짜내서 진행하는 이 일에는 너무 큰 작업이었습니다. 이러한 사정으로 인해 이전 번역자들의 작업이 직접적으로 유지되지는 못하지만, 다른 프로그래머들을 위해 수십 년 전 글을 남기신 번역자 분들의 노력을 이어받고 그 작업을 존중하는 마음으로 다음 몇 년간 사람들이 읽을 수 있는 번역을 제공하기 위해 노력했습니다.

읽어주셔서 감사합니다.

1.13 율긴이의 말: 2026년 개정, 원문 3.3.2 기반 한국어판

원문 3.1.5 버전을 번역한 뒤 몇 해가 지나, 개정판을 다시 작업하게 되었습니다.

기존 번역판에 있던 수많은 오타자와 잘못된 번역을 교정했으며, 기존에는 분량상의 이유로 번역하지 못했던 맨페이지 부분까지 번역했습니다. 기존과 마찬가지로 문서 전체에 걸쳐 Pretendard 폰트를 사용했으며 개정판에서는 코드 스니펫에 D2Coding 폰트를 사용해서 한글 주석이 조금 더 코드와 잘 어울리게 했습니다.

문서 전체를 개정하는 대규모 작업은 AI 덕분에 가능했습니다. 초판은 인터넷 검색과 수작업으로 번역했으나, 개정판은 거의 모든 부분을 AI의 도움으로 번역/교정했으며, 저는 최종적인 검수와 몇몇 부적절한 번역에 대한 조율 위주로 작업했습니다.

AI는 번역 작업을 쉽게 만들어 주었으며, 나아가 이런 안내 문서의 필요성마저 줄이고 있습니다. 소켓 프로그래밍에 대한 궁금증은 LLM도 답할 수 있고, 원문도 여러 LLM이 이미 학습한 것으로 보입니다.

이 책이 다루는 소켓 프로그래밍에 관심 있는 분들도 이제는 이런 문서보다는 LLM을 통해 학습하는 것이 더 자연스러우리라 생각합니다. 그런 만큼, 이런 시대에 번역 작업물의 개정안을 내는 것이 어떤 의미를 가지는지 생각하지 않을 수 없었습니다.

그러나 이 작업물은 인터넷에서 찾을 수 있는 가장 잘 짜인 소켓 프로그래밍 안내서 중 하나이며, 이 번역본은 그 안내서의 꽤 훌륭한 한국어 번역입니다. 누군가 이 안내서를 필요로 한다면, 개정 작업은 충분히 가치 있을 것입니다.

이 안내서를 직접 천천히 읽으셔도 좋고, LLM의 도움을 받아 훑어보셔도 좋습니다. 어떤 방식으로 보시든, 이 번역본이 여러분에게 도움이 되기를 바랍니다.

훌륭한 안내서를 수십 년간 다듬어온 원저자, 그리고 이 안내서의 원문과 번역문을 만들기 위해서 필요했던 지식과 도구를 만들어준 모든 사람들에게 감사의 마음을 전합니다.

읽어주셔서 감사합니다. - 2026년 7월 30일, 정민석.

Chapter 2

소켓이란 무엇인가?

여러분은 “소켓”이란 단어를 자주 들을 것입니다. 그리고 어쩌면 그것이 정확히 무슨 뜻인지 궁금하시겠지요. 그것은 표준 유닉스 파일 설명자를 통해서 다른 프로그램과 이야기하는 방법을 의미합니다.

무슨 말이냐고요?

좋습니다. 아마 어떤 유닉스 해커들이 “이런, 유닉스에선 모든 것이 파일이야!” 라고 말하는 것을 들어보셨을 것입니다. 그들이 말하는 바는 유닉스 프로그램이 어떤 종류의 입출력을 하든, 파일 설명자에서 읽거나 파일 설명자에 쓰는 방식으로 동작한다는 의미입니다. 파일 설명자는 단순히 열려 있는 파일과 연관된 정수입니다. 그러나 (이 부분이 중요합니다.) 그 파일은 네트워크 연결이나 선입선출, 파이프, 터미널, 디스크에 있는 진짜 파일이나 다른 어떤 것이든 될 수 있습니다. 유닉스의 모든 것은 파일입니다! 그러니 인터넷 너머의 다른 프로그램과 통신하고 싶다면 파일 설명자를 통해서 하는 것이 당연합니다.

“잘난 척은 알겠고, 그래서 이 네트워크 통신을 위한 파일 설명자를 어디에서 구하죠?” 라고 아마 지금쯤 생각하고 계실 것입니다. 그래도 답을 드리자면 `socket()` 시스템 콜을 호출하면 된다는 것입니다. 그것은 소켓 설명자를 반환하고, 여러분은 특화된 `send()` 와 `recv()` (`man send`, `man recv`) 소켓 함수를 써서 그 소켓을 통해 통신합니다.

“그런데 잠시만요!” 아마 다른 의문이 생길 것입니다. “그게 그냥 파일 설명자라면, 어쩌서 그냥 평범한 `read()` 와 `write()` 함수를 사용해서 소켓 통신을 하면 안 되나요?” 짧은 답은 “그래도 됩니다!”입니다. 긴 답은 “그래도 되지만, `send()` 와 `recv()` 가 데이터 전송을 더 많이 조정할 수 있게 해 줍니다”가 되겠습니다.

다음에 궁금하신가요? 세상에는 온갖 종류의 소켓이 있습니다. DARPA 인터넷 주소(인터넷 소켓), 로컬 노드의 경로(유닉스 소켓), CCITT X.25 주소(무시해도 상관없는 X.25 소켓), 그리고 여러분이 실행 중인 유닉스의 종류에 따라 다른 많은 종류의 소켓이 있습니다. 이 문서는 첫 번째 것에 대해서만 다룹니다. 바로 인터넷 소켓입니다.

2.1 두 종류의 인터넷 소켓

인터넷 소켓이 두 종류라니 무슨 소리냐고요? 사실 거짓말입니다. 더 많은 종류가 있습니다. 그러나 여러분을 겁주기가 싫었습니다. 여기서는 두 종류에 대해서만 이야기하겠습니다. `raw` 소켓도 있으며 아주 강력하니 여러분도 한 번쯤 살펴보시길 권한다는 지금 이 문장을 제외하고 말입니다.

이쯤 하고, 그 두 종류가 무엇인지 이야기하자면 하나는 스트림 소켓이고 다른 하나는 데이터그램 소켓입니다. 앞으로 이 둘을 각각 `SOCK_STREAM` 과 `SOCK_DGRAM` 으로 칭하겠습니다. 데이터그램 소켓은 때때로 비연결형 소켓이라고 불립니다. (그러나 그것도 정말로 필요하다면 `connect()` 함수를 사용할 수 있습니다. 아래의 `connect()` 를 참고하세요.)

스트림 소켓은 신뢰성 있는 양방향 연결 통신 스트림입니다. 이 소켓으로 데이터 1과 2를 차례로 보내면, 상대방에서도 같은 순서로 받습니다. 또한 전달되는 데이터에는 오류가 없습니다. 이것은 정말로 확실한 내용이어서, 저는 다른 사람들이 이것에 대해서 반박한다면 귀를 막고 노래를 불러서라도 무시하겠습니다.

스트림 소켓의 사용처가 궁금한가요? `telnet`이나 `ssh` 응용프로그램에 대해서 들어보셨나요? 그것들이 스트림 소켓을 사용합니다. 입력한 모든 문자가 입력한 순서 그대로 도착해야 합니다. 또한, 웹브라우저가 쓰는 하이퍼텍스트 전송 프로토콜 (HTTP)도 스트림 소켓을 사용합니다. 실제로, 어떤 웹사이트의 80번 포트에 텔넷으로 연결한 후 "`GET / HTTP/1.0`"을 입력하고 엔터를 두 번 치면 HTML을 돌려줄 것입니다.

여러분이 `telnet`을 설치하지 않았고 설치하고 싶지 않거나, 설치된 `telnet`이 클라이언트에 연결하는 것에 대해 까다롭게 군다면 이 안내서는 `telnet`과 유사한 프로그램인 `telneta`과 같이 제공됩니다. 이 안내서에서 필요한 일은 다 할 수 있을 것입니다. 텔넷이 사실은 명세된 네트워크 통신 프로토콜^b이며, `telnet`은 이 프로토콜을 전혀 구현하지 않는다는 사실을 기억하세요.

^a<https://beej.us/guide/bgnet/source/examples/telnet.c>

^b<https://tools.ietf.org/html/rfc854>

스트림 소켓은 어떻게 데이터를 이렇게 신뢰성 있게 전송할까요? 이를 위해 스트림 소켓은 전송 제어 프로토콜(Transmission Control Protocol), 곧 TCP (TCP에 대한 지나치게 자세한 정보는 RFC 793¹를 참고하세요)라고 불리는 프로토콜을 사용합니다. TCP는 여러분의 데이터가 순서대로 도착하고 손상되지 않도록 보장합니다. TCP를 TCP/IP의 절반으로 이미 들어보셨을 것입니다. IP는 인터넷 프로토콜(Internet Protocol)의 약어입니다. (RFC 791²을 살펴보세요) IP는 주로 인터넷 라우팅을 맡으며 데이터 무결성에는 보통 책임이 없습니다.

좋습니다. 그럼 데이터그램 소켓은 뭘까요? 왜 비연결형일까요? 뭐가 다를까요? 왜 신뢰성이 없는 것일까요? 대답은 아래와 같습니다. 데이터그램을 전송하면 도착할 수도, 도착하지 않을 수도 있습니다. 도착은 하되 순서대로 도착하지 않을 수도 있습니다. 도착한다면, 패킷 안에 있는 데이터에는 오류가 없습니다.

데이터그램 소켓도 라우팅을 위해서 IP를 사용할 것입니다. 그러나 TCP를 사용하지는 않습니다. 데이터그램 소켓은 "User Datagram Protocol" 또는 "UDP"를 사용합니다. (RFC 768³를 참고하세요)

왜 비연결형인지에 대해서 간단히 말하자면 스트림 소켓과 달리 열린 연결을 유지할 필요가 없기 때문입니다. 패킷을 만들고, 목적지 정보를 담은 IP 헤더를 붙이고 보냅니다. 연결은 필요 없습니다. 데이터그램 소켓은 일반적으로 TCP 스택을 사용할 수 없거나 패킷 몇 개가 유실되어도 큰일이 나지는 않는 상황에서 쓰입니다. 몇몇 예제 프로그램은 다음과 같습니다. `tftp` (trivial file transfer protocol, FTP의 동생), `dhcpcd` (DHCP 클라이언트), 다중 사용자 게임, 오디오 스트리밍, 화상 회의 등등

"잠시만요! `tftp`나 `dhcpcd`는 하나의 호스트에서 다른 호스트로 이진 응용프로그램을 전송하기 위해 쓰이지 않아요! 응용프로그램이 도착지에서 제대로 동작하길 바라면 데이터가 유실되면 안 되는 것 아닌가요? 데이터를 제대로 보내기 위해서 흑마법이라도 쓰나요?"

여러분, `tftp`와 유사한 프로그램들은 UDP 위에서 자신만의 프로토콜을 구현합니다. 예를 들어 `tftp` 프로토콜은 그들이 보내는 모든 패킷에 대해서 수신자가 "잘 받았습니다"라고 말하는 패킷("ACK" 패킷)을 돌려줄 것을 요구합니다. 원본 패킷의 송신자가 일정 시간, 가령 5초 안에 응답을 받지 못한다면 송신자는 ACK 응답을 받을 때까지 패킷을 재전송합니다. 이 확인 절차는 아주 중요해서 신뢰할 수 있는 `SOCK_DGRAM` 응용프로그램을 만들 때 반드시 필요합니다.

게임이나 오디오, 비디오처럼 어느 정도 패킷 유실을 허용하는 응용프로그램은 유실된 패킷을 그냥 무시하거나 혹은 그 영향을 적절히 보정하려고 합니다. (퀘이크 사용자들은 이런 유실로 인한 영향을 전문 용어로 짜증나는 렉이라고 부릅니다. 여기에서 쓰인 "짜증나는"은 아주 극단적인 욕설을 대체한 표현입니다.)

신뢰할 수 없는 기반 프로토콜을 왜 사용하는지 궁금한가요? 이유는 단 하나, 속도입니다. 보내고 잊어버리는(fire-and-forget) 방식이 어떤 데이터가 안전하게 도착했는지 추적하고 데이터가 올바른 순서로 왔는지 등을 확인하는 것보다 빠릅니다. 대화 메시지를 보낸다면 TCP는 훌륭한 선택입니다. 게임 세상 안에 있는 각각의 사용자에게 관한

¹<https://tools.ietf.org/html/rfc793>

²<https://tools.ietf.org/html/rfc791>

³<https://tools.ietf.org/html/rfc768>

초당 40번의 위치 정보 갱신을 전송한다면 한두 개의 정보가 유실되어도 상관없습니다. 그렇다면 UDP가 좋은 선택입니다.

2.2 저수준 넌센스와 네트워크 이론

프로토콜의 계층구조에 대해서 이야기했으니 네트워크가 실제로 어떻게 동작하는지 말씀드릴 차례입니다. SOCK_DGRAM 패킷이 어떻게 만들어지는지 약간의 예제를 보여드리겠습니다. 실용적으로는 이 절을 생략해도 좋습니다. 그러나 좋은 배경지식입니다.



Figure 2.1: 데이터 캡슐화(Data Encapsulation).

자, 여러분, *데이터 캡슐화*에 대해 배울 차례입니다. 이것은 아주 중요합니다. 너무 중요해서 치코 캘리포니아 주립대에서 네트워크 수업을 듣는다면 이것에 대해 배우게 될 수도 있습니다. 간단히 말하자면 이렇습니다. 패킷이 태어나면 패킷은 첫 번째 프로토콜(예를 들어 TFTP 프로토콜)에 의해 헤더로 감싸집니다("캡슐화") (드물게 푸터로도 감싸집니다). 그리고 (TFTP의 헤더가 포함되는) 전체가 다시 다음 프로토콜에 의해 감싸집니다 (예를 들어 UDP). 그리고 다시 IP로 감싸지고, 마지막으로 하드웨어(물리) 계층의 프로토콜(예를 들어 이더넷)으로 감싸집니다.

다른 컴퓨터가 패킷을 받으면 하드웨어는 이더넷 헤더를 벗겨내고, 커널이 IP와 UDP 헤더를 벗겨냅니다. 그리고 TFTP 프로그램이 TFTP 헤더를 벗겨내고, 마침내 데이터를 가지게 됩니다.

드디어 막명 높은 *계층화 네트워크 모델(Layered Network Model)*에 대해서 이야기할 수 있습니다. 이 네트워크 모델은 네트워크 기능의 체계(system)를 묘사하며 다른 모델에 비해서 많은 장점을 가지고 있습니다. 예를 들어 여러분은 물리적으로 데이터가 어떻게 전송되는지(직렬통신(Serial), Thin Ethernet(얇은 동축케이블을 쓰는 이더넷의 변형), AUI(Attachment Unit Interface), 등등)(역자 주: 여기에 언급되는 물리적 통신 단자들은 대개 현재는 특수한 산업현장이 아니면 쓰이지 않습니다. 여러분이 이런 것에 대해서 모르신다고 해도 전혀 지장이 없다는 의미입니다.)에 대해서 전혀 신경 쓰지 않고 소켓 프로그램을 완전히 똑같은 모습으로 짤 수 있습니다. 그 이유는 저수준에 있는 프로그램들이 그것을 자동으로 처리해 주기 때문입니다. 실제 네트워크 하드웨어와 네트워크 구성 방식(topology)은 소켓 프로그래머에게 투명(역자 주: 알 필요가 없거나 알 수 없음)합니다.

길게 말하지 않고 이제 전체 모델의 계층을 제시하겠습니다. 네트워크 과목 시험을 위해서 이것을 기억하세요.

- 응용(Application)
- 표현(Presentation)
- 세션(Session)
- 전송(Transport)
- 네트워크(Network)
- 데이터 링크(Data Link)
- 물리(Physical)

물리 계층은 하드웨어입니다(직렬통신, 이더넷 등). 응용 계층은 여러분이 상상할 수 있는 한 가장 물리 계층에서 먼 것입니다. 사용자가 실제로 네트워크와 상호작용하는 부분을 의미합니다.

사실 이 모델은 너무나 일반적이어서 정말로 원한다면 자동차 정비 안내서에도 쓸 수 있을 것입니다. 유닉스와 좀 더 일치하는 계층화 모델은 이렇습니다.

- 응용 계층(Application Layer) (텔넷, ftp 등.)
- 호스트 간 전송 계층(Host-to-Host Transport Layer) (TCP, UDP)
- 인터넷 계층(Internet Layer) (IP와 라우팅)
- 네트워크 접근 계층(Network Access Layer) (이더넷, 와이파이(wi-fi), 기타 등등)

이 시점까지 오면 여러분은 아마도 이 계층들이 원본 데이터의 캡슐화에 어떻게 대응하는지 아실 수 있을 듯합니다. 간단한 패킷을 만드는 데 얼마나 많은 작업이 필요한지 아시겠나요? 그리고 패킷 헤더를 "cat" 명령으로 하나하나 직접 입력해야 합니다! 농담입니다. 스트림 소켓을 위해서 할 일은 그저 `send()` 로 데이터를 보내는 것뿐입니다. 데이터그램 소켓을 위해서 할 일은 여러분이 원하는 방식으로 패킷을 캡슐화하고 `sendto()` 로 내보내는 일뿐입니다. 커널이 여러분을 위해서 전송 계층과 인터넷 계층을 만들어주고 하드웨어가 네트워크 접근 계층을 만들어줄 것입니다. 현대 기술은 정말 멋지지요!

네트워크 이론에 대한 우리의 짧은 공부는 이렇게 끝납니다. 아, 라우팅에 대해서 말씀드리는 것을 잊었습니다. 그러나 라우팅에 대해서는 아무것도 다루지 않을 생각입니다. 라우터(router)가 IP 헤더만 남기고 패킷을 벗겨낸 뒤, 라우팅 테이블 (routing table)을 조회하고, *어찌고 저찌고* 같은 내용은 하나도 이야기하지 않을 것입니다. 정말로 진정으로 알고 싶다면 IP RFC⁴을 참고하세요. 평생 모르고 살아도 문제는 없습니다.

⁴<https://tools.ietf.org/html/rfc791>

Chapter 3

IP 주소, struct, 그리고 데이터 다루기

분위기를 전환해서 코드에 대해 이야기하는 부분이 되었습니다.

그러나 코드가 아닌 이야기를 조금 더 하겠습니다. 먼저 IP 주소와 포트에 대해 이해하고 넘어가겠습니다. 그 다음 소켓 API가 IP 주소와 다른 정보를 저장하고 조작하는 방법에 대해 이야기하겠습니다.

3.1 IP 주소: 버전 4와 6

벤 케노비가 아직 오비완 케노비라고 불리던 좋은 옛 시절에는 인터넷 프로토콜 제4판 또는 IPv4라고 부르던 훌륭한 네트워크 라우팅 체계가 있었습니다. 그것은 4개의 바이트(또는 네 개의 "옥텟" (역자 주: 8비트로 이루어진 1바이트를 명시적으로 지칭하는 표현. 컴퓨터의 초기에는 7비트로 이루어진 바이트도 있었습니다.))로 이루어지고 보통 192.0.2.111 같은 "점과 숫자" 형태로 기록되던 주소를 가졌습니다.

여러분은 아마도 그것을 주변에서 보셨을 것입니다.

사실 이 글을 적는 시점에는 인터넷의 거의 모든 사이트가 IPv4를 사용합니다.

오비완을 비롯해 모두가 행복했습니다. 모든 것이 훌륭했습니다. Vint Cerf 같은 부정적인 사람이 IPv4 주소가 고갈되기 직전이라고 모두에게 경고하기 전까지 말입니다.

(다가오는 IPv4의 종말과 어둠을 모두에게 경고한 것 외에도 Vint Cerf¹는 인터넷의 아버지로 아주 잘 알려져 있습니다. 그래서 저는 그의 판단에 반기를 들 수가 없습니다.)

주소가 고갈된다니 이게 가능한 일일까요? 32비트 IPv4 주소는 수십억 개인데 정말로 가능한지 의문일 겁니다. 정말로 세상에 수십억 개의 컴퓨터가 있을까요?

있습니다.

여기에 더해서, 컴퓨터가 정말 적던 초기에는 모두가 10억은 불가능할 정도로 큰 수라고 생각했습니다. 그래서 일부 큰 조직에 관대하게도 수백만 개의 IP 주소를 할당해 주었습니다. (그런 식으로 많은 IP 주소를 할당받은 조직의 이름을 몇 개 대자면 Xerox, MIT, Ford, HP, IBM, GE, AT&T, 그리고 애플(Apple)이라고 하는 작은 회사도 있었습니다.)

사실 몇몇 임시방편이 없었다면 IP 주소는 예전에 다 떨어졌을 것입니다.

그러나 우리는 모든 사람, 모든 컴퓨터, 모든 계산기, 모든 전화기, 모든 주차 정산기, 모든 강아지(강아지들에게도 IP 주소가 필요하겠지요)에게 IP 주소가 있는 시대를 살고 있습니다.

그리고 그렇게 해서 IPv6이 태어났습니다. 그리고 Vint Cerf는 아마도 불사의 존재이겠지만(그의 물리적 형체가 사라져야 한다면 그는 이미 Internet2의 깊은 곳에서 일종의 초인공지능 ELIZA² 프로그램 같은 모습으로

¹https://en.wikipedia.org/wiki/Vint_Cerf

²<https://en.wikipedia.org/wiki/ELIZA>

존재하고 있을 것입니다.), 다시 한 번 다음 버전의 인터넷 프로토콜에서 주소가 부족해서 그가 “내가 말했지”라는 식으로 이야기하는 것을 듣고 싶어 할 사람은 없습니다.

이게 무슨 의미냐고요?

우리에게 아주 많은 주소가 필요하다는 뜻입니다. 두 배도 아니고, 십억 배도 아니고, 천조 배도 아니고, 약 7.9×10^{28} 배가 필요합니다.

“Beej, 그게 사실인가요? 저에게는 큰 수를 못 믿을 만한 이유가 많습니다.”라고 말씀하실 것입니다. 사실 32비트와 128비트 사이에는 그다지 큰 차이가 없어 보일지도 모르겠습니다. 96비트가 더 있을 뿐이니까요. 하지만 우리는 지금 거듭제곱에 대해서 생각해야 합니다. 32비트가 대략 40억 개의 숫자를 의미합니다. 128비트는 약 3.4×10^{38} 개입니다. (정확히는 2의 128승입니다.) 그것은 이 우주의 모든 별마다 IPv4 인터넷이 백만 개씩 있는 것과 비슷합니다.

IPv4의 점과 숫자 표기는 잊어버리세요. 이제 우리에게 콜론으로 구별하는 2바이트 조각으로 구성되는 16진수 표기법이 있습니다. 예시는 아래와 같습니다.

```
2001:0db8:c9d2:aee5:73e3:934a:a5ae:9551
```

그게 다가 아닙니다! 대부분의 경우에 여러분은 0이 아주 많이 들어있는 주소를 가지게 될 것입니다. 그리고 그 0들을 두 개의 콜론 사이에 압축해서 넣을 수 있습니다. 또한 각각의 바이트 쌍의 앞에 오는 0은 생략할 수 있습니다. 예를 들어 아래의 각 주소 쌍은 동일한 의미입니다.

```
2001:0db8:c9d2:0012:0000:0000:0000:0051
2001:db8:c9d2:12::51
```

```
2001:0db8:ab00:0000:0000:0000:0000:0000
2001:db8:ab00::
```

```
0000:0000:0000:0000:0000:0000:0000:0001
::1
```

::1 주소는 루프백 주소입니다. 그것은 언제나 “내가 실행 중인 이 컴퓨터”를 의미합니다. IPv4에서 루프백 주소는 127.0.0.1 입니다.

마지막으로 IPv6 주소에는 여러분이 보실지도 모르는 IPv4 호환 모드가 있습니다. 예시를 원하신다면 이렇습니다. 192.0.2.33 을 IPv6 주소로 표기한다면 아래와 같습니다. “::ffff:192.0.2.33”.

이건 아주 재미있는 일입니다.

사실 너무 재미있다고 할 수 있는데, IPv6의 제작자들이 수조, 수조 개의 주소를 거리낌 없이 잘라내어 예약된 주소로 지정했기 때문입니다. 그러나 우리에게는 그러고도 너무 많은 주소가 남아 있어서 남은 주소를 세기도 귀찮을 정도입니다. 아직도 은하계의 모든 남녀노소, 강아지, 주차 정산기에 배정할 주소가 남아 있습니다. 은하계의 모든 행성에 주차 정산기가 있다는 것은 확실합니다. 저를 믿으세요.

3.1.1 서브넷

관리적인 측면에서 때때로 “IP 주소의 이 비트까지의 첫 부분은 네트워크 부분 이고 나머지는 호스트 부분”이라고 칭하는 것이 편할 때가 있습니다.

예를 들어 IPv4 주소에서 192.0.2.12 를 가지고 있다면 우리는 첫 3바이트가 네트워크 주소이고 마지막 바이트가 호스트 주소라고 말할 수 있습니다. 조금 다르게 말하면 192.0.2.0 네트워크에 있는 호스트 12 에 대해서 말한다고 할 수 있습니다. (호스트 부분이었던 바이트를 0으로 바꾼 것에 주목하세요.)

더 오래된 정보를 알려드리겠습니다! 고대에는 이 서브넷에 “클래스”가 있었습니다. 각각 주소의 첫 번째, 두 번째, 세 번째 바이트까지가 네트워크 부분임을 의미했습니다. 여러분이 네트워크 부분에 1바이트를 받고 호스트 부분에 3바이트를 받을 정도로 운이 좋다면 여러분의 네트워크에 24비트 규모(대략 천육백만)의 호스트를 가질 수

있었습니다. 이것이 “클래스 A” 네트워크입니다. 반대쪽 끝을 “클래스 C”라고 했습니다. 여기에서는 3바이트가 네트워크 부분이고 1바이트가 호스트입니다. (256개의 호스트이고, 예약된 주소 때문에 몇 개를 더 빼야 합니다.)

그래서 보시다시피, 클래스 A는 몇 개 없고, 클래스 C는 엄청나게 많았으며, 클래스 B는 중간 정도였습니다.

IP 주소의 네트워크 부분은 *넷마스크*라는 것으로 표시되는데, IP 주소에서 네트워크 번호를 구하기 위해서 넷마스크와 비트 단위 AND 연산을 합니다. 넷마스크는 대체로 255.255.255.0 처럼 보입니다. (예를 들어 넷마스크가 저렇고 여러분의 IP가 192.0.2.12 라면 네트워크는 192.0.2.12 AND 255.255.255.0 이고 결과는 192.0.2.0 입니다.)

불행히도 이것은 나중에 드러난 인터넷의 필요에 비해 충분히 세밀하지 못했습니다. 클래스 C 네트워크는 꽤 빠르게 고갈되고 있었고 클래스 A는 이미 다 소진되었으니 물어볼 것도 없었습니다. 이 상황을 타개하기 위해서 8이나 16, 24개의 비트뿐 아니라 임의의 비트를 넷마스크로 쓸 수 있어야 했습니다. (예를 들어 255.255.255.252 같은 넷마스크로 30비트의 네트워크 부분과 2비트의 호스트(4개의 호스트)를 허용할 필요가 있었습니다.) (넷마스크는 언제나 여러 개의 1비트 뒤에 여러 개의 0비트가 따라오는 형태임을 기억하세요.)

그러나 255.192.0.0 같은 긴 문자열을 넷마스크로 쓰는 것은 불편했습니다. 첫 번째로 사람들이 보기에 그것이 몇 비트인지 알기가 힘들었고 두 번째로 너무 길었습니다. 그래서 새로운 방식이 등장했고 훨씬 좋아졌습니다. IP 주소 뒤에 빗금(역자 주: 슬래시 또는 /)을 적고 네트워크 비트의 수를 십진수로 적는 것입니다. 예시는 이렇습니다: 192.0.2.12/30

IPv6에서는 이렇게 할 것입니다: 2001:db8::/32 또는 2001:db8:5413:4028::9db9/64

3.1.2 포트 번호

친절하게도 아직 기억해주시다면, 계층화 네트워크 모델에서 호스트 대 호스트 전송 계층(TCP와 UDP)과 분리된 인터넷 계층(IP)이 있음을 아실 것입니다. 다음 문단으로 가기 전에 한 번 더 살펴보셔도 좋습니다.

(IP 계층이 사용하는) IP 주소 말고도 TCP(스트림 소켓)와 UDP(데이터그램 소켓)는 우연히도 한 가지 주소를 더 사용하는 것을 알 수 있습니다. 그것은 바로 *포트 번호*입니다. 이것은 16비트 숫자이며 연결을 위한 로컬 주소 같은 것입니다.

IP 주소를 호텔의 도로명 주소라고 생각하고, 포트 번호를 방 번호라고 생각하세요. 이것은 꽤 적절한 비유입니다. 어쩌면 나중에 자동차 산업과 관련된 다른 비유를 떠올릴 수 있을지도 모르겠습니다.

여러분이 이메일 수신과 웹서비스를 처리하는 컴퓨터를 가지고 있다고 해봅시다. 하나의 IP 주소로 어떻게 그 두 일을 구분할 수 있을까요?

인터넷의 서로 다른 서비스들은 서로 다른 “잘 알려진” 포트 번호를 가지고 있습니다. 전체 목록은 거대한 IANA 포트 목록³ 또는 여러분이 유닉스 시스템을 사용한다면 /etc/services 파일에서 볼 수 있습니다. HTTP(웹)는 80번 포트를 사용하고, telnet은 23번을, SMTP는 25를 쓰고 게임인 DOOM⁴은 666번(역자 주: 둠은 지옥의 악마와 싸우는 내용의 게임이며, 666은 기독교에서 악마의 숫자로 알려져 있습니다) 포트를 사용했습니다. 1024번 아래의 포트는 흔히 특별한 것으로 취급되어, 사용하기 위해서는 보통 운영체제 특권이 필요합니다.

포트 번호에 대해서는 이 정도로 하겠습니다.

3.2 바이트 순서

왕국의 명령으로, 앞으로는 “엔터리”와 “큰 것 먼저”인 두 개의 바이트 순서가 있을 것입니다

농담입니다. 그러나 한쪽이 다른 쪽보다 더 좋습니다. :-)

³<https://www.iana.org/assignments/port-numbers>

⁴https://en.wikipedia.org/wiki/Doom_%281993_video_game%29

사실 이것에 대해 딱 잘라 말할 방법은 없습니다. 그러니 허풍을 좀 떨겠습니다: 여러분의 컴퓨터는 뒤에서 바이트들을 여러분이 생각하는 것과 반대되는 순서로 저장하고 있었을 수도 있습니다. 아무도 이것을 여러분에게 알려주고 싶어하지 않았을 것입니다.

중요한 것은 인터넷 세계의 모든 사람들이 `b34f` 같은 16진수 2바이트 수를 표현하고자 할 때 `b3` 이 앞에 오고 `4f` 이 뒤에 오는 연속된 바이트로 저장하는 데 대체로 동의했다는 사실입니다. 이것은 말이 되기도 하고, Wilford Brimley⁵ 라면 이것에 대해서 “마땅히 해야 할 일”이라고 할 것입니다. 큰 쪽이 앞에 저장되는 이 방식을 *Big-Endian*(빅엔디언)이라고 합니다.

안타깝게도 전 세계 이곳저곳에 흩어진 일부 컴퓨터들, 그러니까 인텔 혹은 인텔 호환 프로세서 컴퓨터들은 바이트를 반대 순서로 저장합니다. 그래서 `b34f` 는 `4f` 뒤에 `b3` 이 이어지는 연속된 바이트로 저장됩니다. 이런 저장법을 *Little-Endian*(리틀 엔디언)이라고 합니다.

아직 용어 해설이 조금 남았습니다! 둘 중에서 좀 더 상식적으로 보이는 빅엔디언은 *네트워크 바이트 순서(Network Byte Order)*라고도 합니다. 네트워크에서 우리가 그렇게 바이트를 전송하기 때문입니다.

여러분의 컴퓨터는 숫자를 *호스트 바이트 순서(Host Byte Order)*로 저장합니다. 인텔 80x86이라면 그것은 리틀엔디언입니다. 모토롤라 68k라면 호스트 바이트 순서는 빅엔디언입니다. PowerPC라면 호스트 바이트 순서는.. 상황에 따라 다릅니다! (역자 주: 현재 널리 쓰이는 x86-64 프로세서들은 리틀 엔디언이며, 이것은 흔히 쓰이는 ARM 프로세서와 애플 실리콘 등의 ARM 변종에서도 동일합니다. 리틀/빅엔디언 여부에 관계없이 한 바이트 내에서는 무조건 MSB가 앞에 온다는 것도 기억해야 합니다. 결론적으로 대부분의 컴퓨터의 호스트 바이트 오더가 네트워크 바이트 순서와 다릅니다.)

패킷을 만들거나 데이터 구조를 채우는 많은 경우에 여러분은 여러분의 2바이트 또는 4바이트 숫자들이 네트워크 바이트 순서로 확실히 기록되도록 해야 합니다. 하지만 원시 호스트 바이트 순서를 모른다면 어떻게 이런 작업을 할 수 있을까요?

좋은 소식입니다! 그냥 호스트 바이트 순서가 늘 틀렸다고 가정하고, 그것을 네트워크 바이트 순서로 재정렬하는 함수에 넣으세요. 그 함수가 필요하다면 마법 같은 변환 과정을 처리하고, 여러분의 코드는 서로 다른 바이트 정렬 방식을 가진 컴퓨터에서도 호환될 것입니다.

여러분이 변환할 수 있는 숫자에는 두 가지 종류가 있습니다: `short` (2바이트) 과 `long` (4바이트)입니다. 위에서 말한 처리 함수들은 `unsigned` 변종에도 쓰일 수 있습니다. (역자 주: C에서 `short` 와 `long` 의 크기는 플랫폼마다 다릅니다. 여기서는 각각 16비트와 32비트 값을 가리키며, 실제 인수 타입은 구현의 헤더 선언을 확인해야 합니다.) 호스트 바이트 순서로 기록된 `short` 를 네트워크 바이트 순서로 변환하고 싶다면, “host”의 “h”로 시작하고 “to”를 이어서 적고 “network”의 “n”을 적고 “short”의 “s”를 적으세요. 다 붙이면 `htons()` 가 됩니다. (읽는 법: Host to Network Short)

정말 쉽지요...

“n”과 “h”, “s”, “l”의 모든 조합을 원하는 대로 쓸 수 있습니다. 정말로 바보 같은 것만 제외하고 말입니다. 예를 들어 `stoh()` 그러니까 “Short to Long Host”는 없습니다. 대신 이런 것들이 있습니다:

함수	설명
<code>htons()</code>	h ost to n etwork s hort
<code>htonl()</code>	h ost to n etwork l ong
<code>ntohs()</code>	n etwork to h ost s hort
<code>ntohl()</code>	n etwork to h ost l ong

⁵https://en.wikipedia.org/wiki/Wilford_Brimley

간단히 말하자면 숫자를 네트워크로 내보내기 전에 네트워크 바이트 순서로 변환해야 하며 네트워크에서 들어올 때는 호스트 바이트 순서로 변환해야 합니다.

소켓 API에는 표준 64비트 변종이 없지만, 다른 선택지는 `htons()` 참조 페이지에서 이야기합니다. 그리고 부동 소수점 숫자를 다루고 싶다면 한참 아래에 있는 직렬화 절을 참조하세요.

달리 말하지 않는 이상 이 문서의 숫자들은 호스트 바이트 순서라고 생각하세요.

3.3 struct

마침내 여기까지 왔습니다. 프로그래밍에 대해서 말할 차례입니다. 이 절에서는 소켓 인터페이스가 사용하는 다양한 데이터 형식에 대해서 논할 것이며 그중 몇몇은 정말로 어렵습니다.

쉬운 것부터 시작하겠습니다: 소켓 설명자입니다. 소켓 설명자는 아래의 형식입니다.

```
int
```

그냥 평범한 `int` 입니다.

여기서부터 이상해집니다. 저와 함께 꼭 참고 따라오세요.

나의 첫 번째 구조체™— `struct addrinfo`. 이 구조체는 꽤나 최근의 발명품입니다. 이 구조체는 나중에 사용할 소켓 주소 구조체를 준비하는 데 쓰입니다. 또한 호스트 이름 찾거나 서비스 이름 찾기도 사용됩니다. 나중에 실제 사용 예시를 보면 이해가 되겠지만, 지금은 연결을 만들 때 가장 먼저 다루게 될 것 중 하나라고 알아두세요.

```
struct addrinfo {
    int          ai_flags;      // AI_PASSIVE, AI_CANONNAME 등.
    int          ai_family;    // AF_INET, AF_INET6, AF_UNSPEC
    int          ai_socktype;  // SOCK_STREAM, SOCK_DGRAM
    int          ai_protocol;  // "미지정"을 위해서 0을 쓰세요
    size_t       ai_addrlen;   // ai_addr의 바이트 단위 크기
    struct sockaddr *ai_addr;   // sockaddr_in 또는 sockaddr_in6 구조체
    char         *ai_canonname; // 완전한 정규화된 호스트 이름

    struct addrinfo *ai_next;   // 연결 리스트의 다음 노드
};
```

이 구조체의 값을 어느 정도 설정한 후에 `getaddrinfo()` 를 호출할 것입니다. 이 함수는 필요한 정보가 채워진 새 연결 리스트의 포인터를 마지막 인수가 가리키는 곳에 설정합니다.

`ai_family` 필드에서 IPv4나 IPv6을 강제할 수 있고 무엇이든 상관없다면 `AF_UNSPEC` 으로 둘 수 있습니다. 이것은 여러분의 코드가 IP 버전에 무관해지도록 해 주는 좋은 기능입니다.

이것이 링크드 리스트임을 기억하세요: `ai_next` 는 다음 요소를 가리킵니다. 여러분이 고를 수 있는 여러 개의 결과가 돌아올 수 있다는 의미입니다. 저라면 쓸 수 있는 첫 번째 것을 쓰겠습니다. 그러나 여러분은 다른 비즈니스 요구사항이 있을지도 모릅니다. 제가 모든 것을 알 수는 없으니까요.

`struct addrinfo` 안의 `ai_addr` 이 `struct sockaddr` 에 대한 포인터임을 보실 수 있습니다. 여기서부터 IP 주소 구조체의 내부를 본격적으로 파고들기 시작합니다.

여러분은 대체로 이 구조체들에 쓰기 작업을 할 일이 없을 것입니다. 대체로 여러분의 `struct addrinfo` 를 채우기 위해서 `getaddrinfo()` 를 호출하는 것이 여러분이 해야 할 일의 전부입니다. 그러나 그 안에서 값을 꺼내오기 위해서는 그 안을 들여다봐야만 하므로 이제부터 설명하겠습니다.

(struct addrinfo 가 발명되기 전의 코드에서는 모든 정보를 손으로 직접 채워야 했습니다. 저 거친 바깥세상에는 정확히 그런 일을 하는 IPv4 코드를 많이 보실 수 있습니다. 이 안내서의 오래된 버전을 포함한 여러 곳에서 말입니다.)

몇몇 struct 는 IPv4용이고, 어떤 것은 IPv6용이며 어떤 것은 양쪽 모두에 필요합니다. 뭐가 무엇인지도 적어두겠습니다.

어쨌든 struct sockaddr 은 여러 종류의 소켓을 위한 소켓 주소 정보를 저장합니다.

```
struct sockaddr {
    unsigned short    sa_family;    // 주소 계열, AF_xxx
    char              sa_data[14];  // 14 바이트의 프로토콜 주소
};
```

sa_family 는 여러 값 중 하나가 될 수 있는데, 우리가 이 문서에서 하는 모든 일에 대해서는 AF_INET (IPv4) 이나 AF_INET6 (IPv6)가 될 것입니다. sa_data 에는 소켓의 목적지 주소와 포트 번호가 들어 있습니다. 여기에 주소를 직접 적어 넣는 일은 지루하고 불편합니다.

struct sockaddr 을 상대하기 위해서 프로그래머들은 IPv4를 위한 병렬적인 구조체인 struct sockaddr_in ("Internet"의 "in")을 만들었습니다.

그리고 이것이 중요한 부분입니다. struct sockaddr_in 에 대한 포인터는 struct sockaddr 에 대한 포인터로 형변환될 수 있고 그 반대도 가능합니다. 그래서 connect() 가 struct sockaddr* 을 원하더라도 struct sockaddr_in 을 사용할 수 있고 마지막에 형변환만 하면 되는 것입니다

```
// IPv4 전용입니다. IPv6은 struct sockaddr_in6를 참고하세요.

struct sockaddr_in {
    short int          sin_family;  // 주소 계열, AF_INET
    unsigned short int sin_port;    // 포트 번호
    struct in_addr      sin_addr;    // 인터넷 주소
    unsigned char       sin_zero[8]; // sockaddr 구조체와 같은 크기
};
```

이 구조체는 소켓 주소의 요소들을 참조하는 일을 쉽게 해 줍니다. sin_zero (struct sockaddr 와 길이를 맞추기 위해서 덧대진 것)은 memset() 을 이용해서 모두 0으로 설정해야 함을 기억하세요. 또한 sin_family 는 struct sockaddr 의 sa_family 에 대응되며 " AF_INET "으로 설정되어야 함을 기억하세요. 마지막으로 sin_port 는 반드시 네트워크 바이트 순서 로 기록해야 함을 기억하세요. (htons() 를 써야 한다는 의미입니다.)

더 깊게 파고들어 봅시다. sin_addr 필드가 struct in_addr 타입이라는 것을 볼 수 있습니다. 저것이 무엇일까요? 지나치게 과장할 필요는 없지만 저것은 지금껏 있었던 가장 무서운 공용체 (역자 주: 하나의 메모리 구역을 서로 다른 데이터 타입처럼 읽고 쓸 수 있게 해 주는 C 언어의 기능) 중 하나입니다.

```
// (IPv4 전용--IPv6을 위해서는 in6_addr 구조체를 참조하세요)

// 인터넷 주소 (역사적인 이유로 존재하는 구조체)
struct in_addr {
    uint32_t s_addr; // 32비트 정수 (4 바이트)
};
```

와! 사실 이것은 공용체였습니다. 그러나 이제 그 시절은 지났습니다. 잘된 일입니다. 그러니까 만약 여러분이 `struct sockaddr_in` 형으로 `ina`를 선언했다면 `ina.sin_addr.s_addr`이 (네트워크 바이트 순서로 적힌) 4바이트의 IP 주소를 가리킬 것입니다. 여러분의 시스템이 `struct in_addr`에 대해서 여전히 형편없는 공용체를 사용해도 여러분은 제가 위에서 한 것과 동일한 방식으로 4바이트 IP 주소를 참조할 수 있습니다. (이것은 `#define` 덕분입니다.)

IPv6에 대해서도 유사한 `struct`가 있습니다:

```
// (IPv6 전용--IPv4를 위해서는 sockaddr_in 구조체와 in_addr 구조체를 참조하세요)

struct sockaddr_in6 {
    u_int16_t    sin6_family;    // 주소 계열, AF_INET6
    u_int16_t    sin6_port;      // 포트, 네트워크 바이트 순서
    u_int32_t    sin6_flowinfo;  // IPv6 흐름 정보
    struct in6_addr sin6_addr;    // IPv6 주소
    u_int32_t    sin6_scope_id;  // 스코프 아이디
};

struct in6_addr {
    unsigned char    s6_addr[16];    // IPv6 주소
};
```

IPv4용 구조체가 그렇듯이 IPv6용 구조체도 IPv6 주소와 포트 번호를 가진다는 점에 주목하세요.

지금은 IPv6의 흐름 정보나 Scope ID 필드에 관한 내용은 다루지 않을 것입니다. 이것은 초보자를 안내하기 때문입니다. :-)

마지막으로 중요한 것은, 여기에 IPv4와 IPv6의 모든 구조체를 담기에 충분히 크게 설계된 `struct sockaddr_storage`라는 또 하나의 단순한 구조체가 있다는 사실입니다. 일부 함수 호출에서는 해당 함수가 넘겨받은 주소 구조체를 IPv4 또는 IPv6 주소로 채울지 미리 알 수 없는 경우가 있습니다. 그러므로 이 병렬 구조체를 넘기면 됩니다. 이것은 좀 더 크다는 점을 제외하면 `struct sockaddr`과 아주 비슷하며, 여러분은 이것을 여러분이 원하는 형식으로 형변환할 수 있습니다.

```
struct sockaddr_storage {
    sa_family_t    ss_family;    // 주소 계열

    // 이것들은 모두 패딩이고 구현에 따라 다른 내용입니다. 무시하세요.
    char            __ss_pad1[_SS_PAD1SIZE];
    int64_t         __ss_align;
    char            __ss_pad2[_SS_PAD2SIZE];
};
```

중요한 것은 여러분이 `ss_family` 필드에서 주소 계열을 볼 수 있다는 사실입니다. 그것이 `AF_INET` 또는 `AF_INET6`인지 확인하세요(IPv4 또는 IPv6인지 확인하기 위해서). 그 뒤 필요하다면 `struct sockaddr_in` 또는 `struct sockaddr_in6`으로 형변환할 수 있을 것입니다.

3.4 IP 주소, 파트 2

다행히 IP 주소를 다룰 수 있게 해 주는 많은 함수가 있습니다. 손으로 직접 값을 계산해서 `long`에 `<<` 연산자로 밀어 넣을 필요가 없습니다. (역자 주: `<<`은 비트를 왼쪽으로 이동하는 연산자입니다.)

우선 여러분이 `struct sockaddr_in ina` 를 가지고 있다고 합시다. 그리고 저장하고 싶은 IP 주소로 "10.12.110.57" 또는 "2001:db8:63b3:1::3490" 가 있다고 합시다. 여러분이 사용해야 하는 함수는 `inet_pton()` 입니다. 이것은 문자열로 표기한 IP 주소를 여러분이 `AF_INET` 또는 `AF_INET6` 를 지정하는 것에 따라서 `struct in_addr` 또는 `struct in6_addr` 으로 변환합니다. ("pton"은 "presentation to network"(역자 주: 표현에서 네트워크로)의 약어이며 쉽게 기억하고 싶다면 "printable to network"라고 해도 됩니다.) IPv4와 IPv6에 대해서는 아래와 같이 변환할 수 있습니다:

```
struct sockaddr_in sa;    // IPv4
struct sockaddr_in6 sa6;  // IPv6

inet_pton(AF_INET, "10.12.110.57", &(sa.sin_addr));
inet_pton(AF_INET6, "2001:db8:63b3:1::3490", &(sa6.sin6_addr));
```

(짧게 덧붙이면, 이 일을 하는 예전 방식은 `inet_addr()` 이나 `inet_aton()` 함수를 사용했습니다. 이것들은 이제 구형이고 IPv6과는 동작하지 않습니다.)

위의 코드 예제는 그다지 견고하지 않은데 오류 확인이 없기 때문입니다. `inet_pton()` 은 오류가 발생하면 -1 을 반환하고 주소가 엉망이면 0을 반환합니다. 그러니 결과를 사용하기 전에 반환값이 0보다 큰지 확인하세요. 좋습니다. 이제 여러분은 IP 주소 문자열을 이진 표현으로 바꿀 수 있습니다. 반대로는 어떻게 하는지 궁금하신가요? `struct in_addr` 구조체를 가지고 있고 그 구조체의 숫자와 점 표기법을 출력하고 싶다면 어떻게 해야 할까요? (또는 `struct in6_addr` 을 16진수와 콜론 표기법으로 출력한다면 어떻게 해야 할까요?) 이 경우 여러분은 `inet_ntop()` 을 사용해야 합니다. ("ntop"은 "network to presentation"을 의미하며 쉽게 기억하려면 "network to printable"이라고 부르셔도 됩니다.) 예제는 아래와 같습니다:

```
// IPv4:

char ip4[INET_ADDRSTRLEN]; // IPv4 문자열을 담아줄 공간
struct sockaddr_in sa;      // 여기에 무엇인가 담겨 있다고 가정합니다

inet_ntop(AF_INET, &(sa.sin_addr), ip4, INET_ADDRSTRLEN);

printf("The IPv4 address is: %s\n", ip4);

// IPv6:

char ip6[INET6_ADDRSTRLEN]; // IPv6 문자열을 담아줄 공간
struct sockaddr_in6 sa6;     // 여기에 무엇인가 담겨 있다고 가정합니다

inet_ntop(AF_INET6, &(sa6.sin6_addr), ip6, INET6_ADDRSTRLEN);

printf("The address is: %s\n", ip6);
```

이 함수를 호출하려면 주소 종류(IPv4 또는 IPv6)와 주소, 결과를 담을 문자열의 포인터, 그 문자열의 최대 길이를 인수로 넘겨야 합니다. (두 개의 매크로인 `INET_ADDRSTRLEN` 과 `INET6_ADDRSTRLEN` 이 편리하게도 가장 긴 IPv4 또는 IPv6 문자열을 담아줄 문자열의 크기를 가지고 있습니다.)

(이 일을 하는 오래된 방식에 대한 다른 이야기 : 이 변환 작업을 하는 역사적인 함수는 `inet_ntoa()` 입니다. 마찬가지로 구식이고 IPv6에는 작동하지 않습니다.)

마지막으로 이 함수들은 숫자 형태의 IP 주소에만 사용할 수 있습니다. 이 함수들은 "www.example.com" 같은

호스트 이름에 대한 네임서버 DNS 탐색을 하지 않습니다. 그 작업을 위해서는 다음에 보실 `getaddrinfo()` 를 써야 합니다.

3.4.1 사설(또는 분리된) 망

많은 곳들이 보호를 목적으로 네트워크를 외부로부터 숨기는 방화벽을 가지고 있습니다. 그리고 흔히 이 방화벽들은 *Network Address Translation* 또는 NAT이라는 절차를 통해서 “내부” IP 주소를 (세상의 다른 사람들이 아는) “외부” IP 주소로 변환합니다.

벌써 긴장되나요? “이 이상한 것들로 뭘 하려는 생각이죠?”라고 하는 것 같네요.

진정하고 무알코올(아니면 알코올이 있는) 음료를 준비하세요. NAT은 여러분을 위해서 투명하게 처리되므로(역자 주: 알 필요가 없게, 보이지 않게) 초보자는 NAT에 대해서 신경 쓸 필요도 없습니다. 그러나 저는 여러분이 보는 네트워크 숫자로 인해 헷갈릴 일이 없도록 방화벽 뒤의 네트워크에 대해서 이야기하고 싶었습니다.

예를 들어 제 집에는 방화벽이 있습니다. 저에게는 디지털 가입자 회선(DSL) 회사가 저에게 배정해 준 두 개의 정적 IPv4 주소가 있습니다. 그런데 제 네트워크에 있는 컴퓨터는 일곱 대입니다. 이것이 어떻게 가능하냐고요? 두 개의 컴퓨터가 같은 IP를 쓸 수는 없습니다. 그렇게 되면 데이터가 어디로 가야 할지 알 수 없게 됩니다.

답은 이렇습니다. 컴퓨터들은 IP 주소를 공유하지 않습니다. 그것들은 2천4백만 개의 IP 주소가 할당된 사설 네트워크에 속해 있습니다. 그 주소들이 전부 저만을 위한 것입니다. 적어도 바깥세상에서 보기에는 그렇습니다. 원리는 이렇습니다:

제가 원격 호스트에 로그인하면, 그것은 제가 192.0.2.33에서 로그인했다고 말해 줍니다. 그 주소는 제 인터넷 서비스 제공자가 저에게 준 공용 IP 주소입니다. 그러나 제가 로컬 시스템에 저의 주소를 물어보면 그것은 10.0.0.5라고 대답합니다. 누가 IP 주소를 변환해 주는 것일까요? 그렇습니다. 바로 방화벽입니다. 그것이 NAT을 수행하는 것입니다.

10.x.x.x는 완전히 외부와 차단된 네트워크 또는 방화벽 뒤의 네트워크만이 사용하도록 예약된 몇 개의 네트워크 대역 중 하나입니다. 어떤 사설 네트워크 숫자가 사용 가능한지는 RFC 1918⁶에 제시되어 있습니다. 그러나 여러분이 보실 일반적인 것들은 10.x.x.x 또는 192.168.x.x입니다. x에는 보통 0에서 255 사이의 수가 들어갑니다. 좀 덜 일반적인 것으로 172.y.x.x가 있습니다. y에는 16부터 31 사이의 수가 옵니다.

NAT 동작을 수행하는 방화벽 뒤에 있는 망(네트워크)이 이런 사설 네트워크 중 하나 여야만 하는 것은 아닙니다. 그러나 보통은 그렇습니다.

(재미있는 사실! 제 외부 IP 주소가 실제로 192.0.2.33인 것은 아닙니다. 192.0.2.x 네트워크는 문서에서 “진짜” IP 주소처럼 보이는 예시를 쓰기 위해 예약된 대역입니다. 바로 이 안내서에서 쓰인 것처럼 말입니다! 우와아아아!)

IPv6도 어떤 의미로는 사설망을 가지고 있습니다. 그것들은 fdXX:으로 시작합니다. (미래에는 fcXX:으로 시작할 수도 있습니다.) RFC 4193⁷ 문서에 따르자면 그렇습니다. 그러나 NAT과 IPv6은 일반적으로 같이 쓰이지 않습니다. (IPv6 to IPv4 게이트웨이(gateway) 같은 것을 만들지 않는다면 말입니다. 그리고 그것은 이 문서의 범위를 넘어섭니다.) 아무튼 이론적으로는 여러분에게는 너무나 많은 주소가 있어서 더 이상 NAT을 쓸 필요가 없을 것입니다. 그럼에도 여러분이 외부와 통하지 않는 주소를 할당하고 싶다면, 위에 적은 대역을 쓸 수 있다는 의미입니다.

⁶<https://tools.ietf.org/html/rfc1918>

⁷<https://tools.ietf.org/html/rfc4193>

Chapter 4

IPv4에서 IPv6으로 넘어가기

“아무튼 저는 IPv6에서 동작하게 하려면 제 코드의 어디를 바꿔야 하는지 알고 싶단 말이에요! 당장 알려주세요!”
알겠어요! 알겠어요!

여기 적을 내용의 대부분은 앞에서 이미 다룬 것이지만, 참을성 없는 분들을 위한 요약판이라고 생각하시면 됩니다. (물론 실제로는 이것보다 더 많은 내용이 있겠지만, 이 안내서에서 다루는 범위는 이 정도입니다.)

1. 우선 `struct sockaddr` 정보를 얻기 위해 수작업 대신 `getaddrinfo()` 함수를 사용하세요. 이렇게 하면 여러분은 IP 버전에 신경 쓰지 않아도 되고, 뒤따르는 여러 단계를 없앨 수 있습니다.
2. IP 버전에 관계된 것을 하드코딩하는 곳을 찾아낼 때마다 헬퍼 함수로 감싸 두세요.
3. `AF_INET` 을 `AF_INET6` 로 바꾸세요.
4. `PF_INET` 을 `PF_INET6` 로 바꾸세요.
5. `INADDR_ANY` 대입을 `in6addr_any` 대입으로 바꾸세요. 이런 차이가 있습니다:

```
struct sockaddr_in sa;  
struct sockaddr_in6 sa6;  
  
sa.sin_addr.s_addr = INADDR_ANY; // 모든 로컬 IPv4 인터페이스에 바인드합니다  
sa6.sin6_addr = in6addr_any; // 모든 로컬 IPv6 인터페이스에 바인드합니다
```

또한 `struct in6_addr` 을 선언할 때 `IN6ADDR_ANY_INIT` 을 초기값으로 사용할 수 있습니다. 아래와 같이 합니다.

```
struct in6_addr ia6 = IN6ADDR_ANY_INIT;
```

6. `struct sockaddr_in` 대신 `struct sockaddr_in6` 을 사용하고, 필요한 필드에 “6”을 적절히 덧붙이세요. (위의 `struct s`을 참고하세요) `sin6_zero` 필드는 없습니다.
7. `struct in_addr` 대신 `struct in6_addr` 를 사용하고, 필요한 필드에 “6”을 적절히 덧붙이세요. (위의 `struct s`을 참고하세요)
8. `inet_aton()` 이나 `inet_addr()` 대신에 `inet_pton()` 을 사용하세요.
9. `inet_ntoa()` 대신에 `inet_ntop()` 을 사용하세요.

10. `gethostbyname()` 대신에 더 뛰어난 `getaddrinfo()` 를 사용하세요.
11. `gethostbyaddr()` 대신에 더 뛰어난 `getnameinfo()` 를 사용하세요. (`gethostbyaddr()` 가 IPv6에서도 여전히 작동하기는 합니다).
12. `INADDR_BROADCAST` 는 더 이상 작동하지 않습니다. 대신 IPv6 멀티캐스트를 사용하세요.

자, 끝입니다!

Chapter 5

시스템 콜이 아니면 죽음을

이 절에서는 Unix 시스템이나 소켓 API를 지원하는 다른 시스템(BSD, Windows, Linux, Mac 등)에서 네트워크 기능에 접근할 수 있게 해 주는 시스템 호출과 라이브러리 호출을 다룹니다. 이런 함수 중 하나를 호출하면 커널이 넘겨받고 여러분을 위해 모든 일을 자동으로 마법같이 처리합니다.

(역자 주: 이 절의 일부 함수 원형은 원문의 오래된 표기입니다. 최신 POSIX 계열에서는 주소 길이 인수에 대개 `socklen_t` 또는 `socklen_t *`를 사용합니다.)

대부분의 사람들이 어려워하는 점은 이 함수들을 어떤 순서로 호출해야 하는가입니다. 이미 찾아보셨겠지만 그런 쪽으로는 `man` 페이지는 아무 쓸모도 없습니다. 그 끔찍한 상황을 해결하기 위해 시스템 콜들을 정확히(대략) 여러분의 프로그램에서 호출해야 하는 순서 그대로 아래에 이어지는 절들에 제시했습니다.

그러니까 여기에 있는 몇몇 예제 코드와 우유, 과자(이것들은 직접 준비하셔야 합니다) 그리고 두둑한 배짱과 용기만 있다면 여러분은 인터넷의 세계에서 존 포스텔의 아들처럼 데이터를 쏘아 보낼 수 있게 될 것입니다. (역자 주: John Postel은 인터넷 초기에 큰 기여를 한 컴퓨터 과학자 중 한 명입니다.)

(아래의 예제 코드들은 대개 간략히 보여주기 위해 필수적인 오류 확인을 생략했음을 기억하세요. 그리고 예제 코드들은 대개 `getaddrinfo()`의 호출이 성공하고 적절한 연결 리스트를 출력 매개변수에 설정한다고 가정합니다. 이런 상황은 독립 실행형 프로그램에서는 제대로 처리되어 있으니, 그것들을 지침으로 삼으세요.)

5.1 `getaddrinfo()` —발사 준비!

이것은 여러 옵션을 가진 진짜 일꾼입니다. 그러나 사용법은 사실 꽤 간단합니다. 이것은 여러분이 나중에 필요로 하는 `struct` 들을 초기화합니다.

역사 한 토막 : 예전에는 DNS 검색을 위해서 `gethostbyname()`을 호출해야 했습니다. 그리고 그 정보를 수작업으로 `struct sockaddr_in`에 담고 이후의 호출에서 사용해야 했습니다.

고맙게도 더 이상은 그럴 필요가 없습니다. (여러분이 IPv4와 IPv6 환경 모두에서 동작하는 코드를 짜고 싶다면 그래서도 안 됩니다!) 요새는 `getaddrinfo()`라는 함수가 있어서 DNS와 서비스 이름을 검색하고 필요한 `struct`를 채우는 등 여러분이 필요로 하는 모든 일을 해 줍니다.

이제 살펴봅시다!

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>
```

```
int getaddrinfo(const char *node,    // 예: "www.example.com" 또는 IP
               const char *service, // 예: "http" 또는 포트 번호
               const struct addrinfo *hints,
               struct addrinfo **res);
```

이 함수에는 입력 매개변수 세 개를 넘겨줍니다. 함수는 결과 연결 리스트의 포인터를 `res` 가 가리키는 곳에 설정합니다.

`node` 매개변수는 접속하려는 호스트 이름이나 IP 주소입니다.

다음 매개변수는 `service` 입니다. 이것은 "80"같은 포트 번호나 "http", "ftp", "telnet" 또는 "smtp" 같은 특정한 서비스 이름이 될 수 있습니다. (IANA 포트 목록¹ 혹은 Unix 시스템을 쓴다면 `/etc/services` 에서 볼 수 있습니다)

마지막으로 `hints` 매개변수는 관련 정보를 이미 채운 `struct addrinfo` 를 가리킵니다.

여기에 여러분이 호스트의 IP 주소와 포트 3490에서 리스닝하려는 서버일 때의 함수 호출 예제가 있습니다. 이것이 리스닝 작업이나 네트워크 설정을 하지는 않는다는 점을 기억하세요. 이것은 나중에 사용할 구조체를 설정할 뿐입니다.

```
int status;
struct addrinfo hints;
struct addrinfo *servinfo; // 결과를 가리킵니다

memset(&hints, 0, sizeof hints); // 구조체를 비웁니다
hints.ai_family = AF_UNSPEC;      // IPv4든 IPv6이든 상관없습니다
hints.ai_socktype = SOCK_STREAM; // TCP 스트림 소켓
hints.ai_flags = AI_PASSIVE;     // bind()에 쓸 와일드카드 주소를 요청합니다

if ((status = getaddrinfo(NULL, "3490", &hints, &servinfo)) != 0) {
    fprintf(stderr, "gai error: %s\n", gai_strerror(status));
    exit(1);
}

// servinfo는 이제 1개 혹은 그 이상의
// addrinfo 구조체에 대한 연결 리스트를 가리킵니다

// ... servinfo가 더 이상 필요 없을 때까지 모든 작업을 합니다...

freeaddrinfo(servinfo); // 연결 리스트를 해제
```

`ai_family` 를 `AF_UNSPEC` 으로 설정해서 IPv4든 IPv6이든 신경 쓰지 않음을 나타낸 것에 주목하세요. 만약 특정한 하나를 원한다면 `AF_INET` 이나 `AF_INET6` 을 쓸 수 있습니다.

`AI_PASSIVE` 도 볼 수 있습니다. `getaddrinfo()` 의 첫 번째 인수가 `NULL` 일 때 이 플래그를 지정하면, `getaddrinfo()` 는 `bind()` 에 쓸 와일드카드 주소를 반환합니다. 그러므로 특정 로컬 IP 주소를 하드코딩하지 않고도 서버를 설정할 수 있습니다. 특정한 로컬 IP 주소에만 바인드하려면 첫 번째 매개변수에 그 주소를 넣으세요.

(역자 주: 와일드카드 주소는 IPv4에서는 `INADDR_ANY (0.0.0.0)`, IPv6에서는 `in6addr_any (::)`입니다. 이 주소로 `bind()` 하면 해당 주소 계열의 모든 로컬 인터페이스에서 그 포트로 들어오는 요청을 받을 수 있습니다.

¹<https://www.iana.org/assignments/port-numbers>

AI_PASSIVE와 getaddrinfo()는 주소를 반환할 뿐 바인드 자체를 수행하지 않습니다. 첫 번째 인수에 특정 노드를 지정하면 AI_PASSIVE는 영향을 주지 않습니다.)

이렇게 함수를 호출합니다. 오류가 있다면(getaddrinfo()가 0이 아닌 값을 반환한다면) 보시다시피 gai_strerror() 함수를 통해 오류를 출력할 수 있습니다. 만약 모든 것이 제대로 동작한다면 servinfo는 우리가 나중에 쓸 수 있는 struct sockaddr이나 비슷한 것을 가진 struct addrinfo의 연결 리스트를 가리킬 것입니다. 근사하네요!

마지막으로 getaddrinfo()가 고맙게도 우리에게 할당해 준 연결 리스트를 이제 필요 없어졌다면 우리는 freeaddrinfo()를 호출해서 그것을 해제할 수 있습니다. (사실 반드시 해야 합니다.)

여기에 여러분이 특정한 주소, 예를 들어 "www.example.net"의 3490 포트에 접속하고자 하는 클라이언트일 경우의 호출 예제가 있습니다. 다시 말씀드리지만 이것으로는 실제 연결이 이루어지지 않습니다. 그러나 이것은 우리가 나중에 사용할 구조체를 설정해 줍니다.

```
int status;
struct addrinfo hints;
struct addrinfo *servinfo; // 결과를 가리킵니다

memset(&hints, 0, sizeof hints); // 구조체를 비웁니다
hints.ai_family = AF_UNSPEC;      // IPv4나 IPv6은 신경 쓰지 않음
hints.ai_socktype = SOCK_STREAM; // TCP 스트림 소켓

// 연결 준비
status = getaddrinfo("www.example.net", "3490", &hints, &servinfo);

// servinfo는 이제 1개 혹은 그 이상의
// addrinfo 구조체에 대한 연결 리스트를 가리킵니다

// 등등.
```

servinfo는 모든 종류의 주소 정보를 가진 연결 리스트라고 거듭 말했습니다. 이 정보를 보기 위한 짧은 시연 프로그램을 작성해 봅시다. 이 짧은 프로그램²은 여러분이 명령줄에 적는 호스트의 IP 주소들을 출력합니다.

```
/*
** showip.c
**
** 명령줄에서 주어진 호스트의 주소들을 출력합니다
**/

#include <stdio.h>
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>
#include <arpa/inet.h>
#include <netinet/in.h>

int main(int argc, char *argv[])
{
```

²<https://beej.us/guide/bgnet/source/examples/showip.c>

```

struct addrinfo hints, *res, *p;
int status;
char ipstr[INET6_ADDRSTRLEN];

if (argc != 2) {
    fprintf(stderr, "usage: showip hostname\n");
    return 1;
}

memset(&hints, 0, sizeof hints);
hints.ai_family = AF_UNSPEC; // IPv4와 IPv6 어느 쪽이든
hints.ai_socktype = SOCK_STREAM;

if ((status = getaddrinfo(argv[1], NULL, &hints, &res)) != 0) {
    fprintf(stderr, "getaddrinfo: %s\n", gai_strerror(status));
    return 2;
}

printf("IP addresses for %s:\n\n", argv[1]);

for(p = res; p != NULL; p = p->ai_next) {
    void *addr;
    char *ipver;
    struct sockaddr_in *ipv4;
    struct sockaddr_in6 *ipv6;

    // 주소 자체에 대한 포인터를 받습니다.
    // IPv4와 IPv6은 필드가 다릅니다.
    if (p->ai_family == AF_INET) { // IPv4
        ipv4 = (struct sockaddr_in *)p->ai_addr;
        addr = &(ipv4->sin_addr);
        ipver = "IPv4";
    } else { // IPv6
        ipv6 = (struct sockaddr_in6 *)p->ai_addr;
        addr = &(ipv6->sin6_addr);
        ipver = "IPv6";
    }

    // IP 주소를 문자열로 변환하고 출력합니다
    inet_ntop(p->ai_family, addr, ipstr, sizeof ipstr);
    printf(" %s: %s\n", ipver, ipstr);
}

freeaddrinfo(res); // 연결 리스트를 해제합니다

return 0;
}

```

보시다시피 이 코드는 명령줄에 넘긴 값을 `getaddrinfo()` 에 전달해 호출해서 `res` 가 가리키는 연결 리스트를 채웁니다. 그래서 우리는 이 목록을 순회해서 출력하거나 다른 일을 할 수 있습니다.

(저 예제 코드에는 IP 버전에 따라 다른 종류의 `struct sockaddr` 을 처리해야 한다는 점에서 흉한 부분이 있습니다. 그 점에 대해서 사과드립니다. 그러나 더 나은 방법이 있는지는 잘 모르겠습니다.)

실행 예제! 모두가 스크린샷을 좋아합니다.

```
$ showip www.example.net
IP addresses for www.example.net:

IPv4: 192.0.2.88

$ showip ipv6.example.com
IP addresses for ipv6.example.com:

IPv4: 192.0.2.101
IPv6: 2001:db8:8c00:22::171
```

이제 저것을 다룰 수 있으니, `getaddrinfo()` 에서 얻은 결과를 다른 소켓 함수에 넘겨 결과적으로 네트워크 연결을 성립시킬 수 있게 해 봅시다! 계속 읽어보세요!

5.2 `socket()` —파일 설명자를 받자!

더 이상 미룰 수가 없을 듯합니다. 이제 `socket()` 시스템 콜에 대해서 이야기해야 합니다. 개요는 이렇습니다.

```
#include <sys/types.h>
#include <sys/socket.h>

int socket(int domain, int type, int protocol);
```

그러면 이 인수들은 뭘까요? 이것들은 어떤 종류의 소켓을 원하는지 정할 수 있게 해 줍니다. (IPv4 또는 IPv6, 스트림 혹은 데이터그램, TCP 혹은 UDP)

예전에는 사람들이 그 값을 직접 적었고, 지금도 물론 그렇게 할 수 있습니다. (`domain` 은 `PF_INET` 이나 `PF_INET6` 이고, `type` 은 `SOCK_STREAM` 또는 `SOCK_DGRAM` 이며, `protocol` 은 주어진 `type` 에 적절한 값을 자동으로 선택하게 하려면 0 을 넘겨주거나 "tcp"나 "udp" 중 원하는 프로토콜의 값을 얻기 위해서 `getprotobyname()` 을 쓸 수도 있습니다.)

(이 `PF_INET` 은 `sin_family` 필드를 초기화할 때 넣어 주는 `AF_INET` 과 가까운 친척입니다. 사실 둘은 아주 가까워서 값도 같습니다. 그래서 많은 프로그래머는 `socket()` 을 호출할 때 첫 번째 인수로 `PF_INET` 대신 `AF_INET` 을 넘깁니다. 자, 우유와 과자를 준비하세요. 이제 이야기를 하나 할 시간이니깐요. 아주 먼 옛날에는 어쩌면 하나의 주소 계열(Address Family) ("AF_INET" 안에 들어 있는 "AF")이 여러 종류의 프로토콜 계열(Protocol Family)("PF_INET"의 "PF")을 지원할 것이라고 생각하던 시절이 있었습니다. 그러나 그런 일은 일어나지 않았습니다. 그리고 모두 행복하게 오래오래 잘 살았다고 합니다. 이런 이야기입니다. 그래서 할 수 있는 가장 정확한 일은 `struct sockaddr_in` 에서 `AF_INET` 을 쓰고 `socket()` 에서 `PF_INET` 을 사용하는 것입니다.

아무튼 이제 충분합니다. 여러분이 정말로 하고 싶은 일은 `getaddrinfo()` 를 호출한 결과로 돌아오는 값을 아래와 같이 `socket()` 에 직접 넘겨주는 것입니다.

```
int s;
struct addrinfo hints, *res;

// 탐색 시작
// ["hints"구조체는 이미 채운 것으로 가정합니다]
```

```
getaddrinfo("www.example.com", "http", &hints, &res);

// 다시 말하자면 원래는 (이 안내서의 예제들이 하듯이) 첫 번째 것이 좋다고
// 가정하는 대신 getaddrinfo()에 대해서 오류 확인을 하고
// "res" 연결 리스트를 순회해야 합니다.
// 클라이언트/서버 절의 진짜 예제들을 참고하세요.

s = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
```

`socket()` 은 이후의 시스템 호출에서 쓸 수 있는 소켓 설명자를 반환합니다. 오류가 있으면 `-1` 을 반환합니다. 전역 변수 `errno` 가 오류 값으로 설정됩니다. (자세한 정보와 멀티스레드 프로그램에서 `errno` 를 사용할 때의 짧은 노트는 `errno` 매 페이지를 참고하세요.)

좋습니다. 그러면 이제 이 소켓을 어디에 쓸 수 있을까요? 정답은 아직 못 쓴다는 것입니다. 실제로 쓰기 위해서는 안내서를 더 읽고 이것이 동작하게 하기 위한 시스템 호출을 더 해야 합니다.

5.3 `bind()` —나는 어떤 포트에 있나요?

소켓을 가지면 사용 중인 시스템의 포트에 바인드해야 할 수도 있습니다. (이 작업은 보통 여러분이 `listen()` 으로 특정 포트에서 들어오는 연결을 받으려고 할 때 이루어집니다. 다중 사용자 네트워크 게임들은 "192.168.5.10의 3490 포트에 연결합니다"라고 말할 때 이런 작업을 합니다.) (역자 주: 90년대부터 2000년대 초의 멀티플레이어 게임은 접속할 때 이런 문구를 흔히 보여 줬습니다.) 포트 번호는 커널이 특정 프로세스의 소켓 설명자를 들어오는 패킷과 연관 짓기 위해서 사용합니다. 만약 여러분이 `connect()` 만 할 생각이라면 `bind()` 는 대개 필요하지 않습니다. 그러나 재미를 위해 읽어봅시다.

이것이 `bind()` 시스템 콜의 개요입니다.

```
#include <sys/types.h>
#include <sys/socket.h>

int bind(int sockfd, struct sockaddr *my_addr, int addrlen);
```

`sockfd` 는 `socket()` 이 반환한 소켓 파일 설명자입니다. `my_addr` 은 여러분의 주소, 말하자면 포트와 IP 주소를 가진 `struct sockaddr` 에 대한 포인터입니다. `addrlen` 은 그 주소의 바이트 단위 길이입니다.

휴. 한 번에 받아들이기에는 조금 많습니다. 프로그램이 실행되는 호스트의 3490번 포트에 소켓을 바인드하는 예제를 봅시다.

```
struct addrinfo hints, *res;
int sockfd;

// 먼저 getaddrinfo()로 구조체에 정보를 불러옵니다

memset(&hints, 0, sizeof hints);
hints.ai_family = AF_UNSPEC; // IPv4나 IPv6 중 아무 것이나 씁니다
hints.ai_socktype = SOCK_STREAM;
hints.ai_flags = AI_PASSIVE; // bind()에 쓸 와일드카드 주소를 요청합니다

getaddrinfo(NULL, "3490", &hints, &res);
```

```
// 소켓을 만듭니다.

sockfd = socket(res->ai_family, res->ai_socktype, res->ai_protocol);

// getaddrinfo()에 넘겼던 포트에 바인드합니다.

bind(sockfd, res->ai_addr, res->ai_addrlen);
```

`AI_PASSIVE` 플래그와 `NULL` 첫 번째 인수는 `bind()` 에 쓸 와일드카드 주소를 요청합니다. 특정한 로컬 IP 주소에만 바인드하고 싶다면 `AI_PASSIVE` 를 빼고 `getaddrinfo()` 의 첫 번째 인수로 그 IP 주소를 넣으세요.

`bind()` 도 오류가 발생하면 `-1` 을 반환하고 `errno` 를 오류 값으로 설정합니다.

많은 오래된 코드들이 `bind()` 를 호출하기 전에 `struct sockaddr_in` 을 직접 채워 넣습니다. 이것은 분명히 IPv4 전용이지만 같은 일을 IPv6에 대해서도 못 할 이유는 없습니다. 단지 `getaddrinfo()` 를 쓰는 편이 일반적으로 더 쉽습니다. 어쨌든 예전 코드는 이런 방식입니다.

```
// !!! 이것은 예전 방식입니다 !!!

int sockfd;
struct sockaddr_in my_addr;

sockfd = socket(PF_INET, SOCK_STREAM, 0);

my_addr.sin_family = AF_INET;
my_addr.sin_port = htons(MYPORT); // short, 네트워크 바이트 순서
my_addr.sin_addr.s_addr = inet_addr("10.12.110.57");
memset(my_addr.sin_zero, '\0', sizeof my_addr.sin_zero);

bind(sockfd, (struct sockaddr *)&my_addr, sizeof my_addr);
```

위의 코드에서 모든 로컬 IPv4 인터페이스에 바인드하고 싶었다면(위의 `AI_PASSIVE` 처럼) `s_addr` 필드에 `INADDR_ANY` 를 대입할 수 있습니다. IPv6에서 이에 해당하는 값은 여러분의 `struct sockaddr_in6` 의 `sin6_addr` 필드에 대입해야 하는 전역 변수인 `in6addr_any` 입니다. (변수 초기화식에 쓸 수 있는 `IN6ADDR_ANY_INIT` 이라는 매크로도 있습니다.)

`bind()` 를 호출할 때 주의해야 할 점: 포트 번호는 낮은 것을 쓰지 마세요. 1024번 아래의 모든 포트는 예약되어 있습니다(슈퍼유저가 아닌 이상)! 그 위의 포트 번호는 (다른 프로그램이 이미 쓰고 있지 않다면) 65535까지 아무 것이나 쓸 수 있습니다.

눈치챌 수 있듯이 때때로 서버를 다시 실행하려고 하면 `bind()` 가 실패하고 "주소가 이미 사용 중입니다."라고 할 때가 있습니다. 그것은 연결되었던 소켓 중 일부가 여전히 커널에서 대기 중이고 포트를 사용하고 있다는 것을 의미합니다. 여러분은 그것이 정리될 때까지 1분 정도를 기다리거나 여러분의 프로그램이 포트를 재사용할 수 있도록 하는 코드를 넣을 수도 있습니다.

```
int yes=1;
//char yes='1'; // 솔라리스에서는 이것을 사용

// "주소가 이미 사용 중입니다"라는 오류 메시지를 제거
setsockopt(listener, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes);
```

`bind()`에 대해서 한마디 더 : 이 함수를 호출할 필요가 전혀 없는 경우도 있습니다. `connect()`를 호출해서 원격 호스트에 연결하려고 하고, 로컬 포트에 대해서는 신경 쓰지 않는다면(`telnet`의 경우처럼 원격지 포트만 신경 쓰는 경우) `connect()`가 자동으로 소켓이 바인드되지 않았는지 확인하고 필요하다면 사용하지 않은 로컬 포트에 `bind()`해 줄 것입니다.

5.4 `connect()` — 거기 안녕!

몇 분만 여러분이 텔넷 응용프로그램이 되었다고 생각해봅시다. 여러분의 사용자들이 소켓 파일 설명자를 얻기 위해서 여러분에게 명령을 내립니다 (영화 *트론*에서처럼요). 여러분은 그에 따라 `socket()`을 호출합니다. 다음으로 사용자가 여러분에게 "10.12.110.57"의 "23"번 포트(텔넷 표준 포트)에 연결하라고 합니다. 어떻게 해야 할까요?

프로그램 입장에서는 운이 좋게도, 지금 `connect()`에 대한 절을 읽는 중입니다! 이 절은 원격 호스트에 어떻게 연결하는지에 대해 알려줍니다. 거침없이 읽어봅시다! 낭비할 시간이 없습니다!

`connect()`에 대한 호출은 아래와 같습니다:

```
#include <sys/types.h>
#include <sys/socket.h>

int connect(int sockfd, struct sockaddr *serv_addr, int addrlen);
```

`sockfd`는 `socket()` 함수 호출이 반환하는 우리의 친근한 이웃인 소켓 파일 설명자입니다. `serv_addr`는 `struct sockaddr`이고 목적지 포트와 IP 주소를 담고 있습니다. `addrlen`은 서버 주소 구조체의 바이트 단위 길이를 담고 있습니다.

모든 정보는 멋진 `getaddrinfo()` 호출의 결과에서 추출할 수 있습니다.

이해가 되기 시작하나요? 저는 대답을 들을 수 없으니 그럴 것이라 생각하겠습니다. "`www.example.com`"의 3490 포트로 소켓 연결을 만드는 예제를 살펴봅시다:

```
struct addrinfo hints, *res;
int sockfd;

// getaddrinfo()로 주소 구조체 목록을 가져옵니다

memset(&hints, 0, sizeof hints);
hints.ai_family = AF_UNSPEC;
hints.ai_socktype = SOCK_STREAM;

getaddrinfo("www.example.com", "3490", &hints, &res);

// 소켓을 만듭니다

sockfd = socket(res->ai_family, res->ai_socktype, res->ai_protocol);

// 연결합니다!

connect(sockfd, res->ai_addr, res->ai_addrlen);
```

다시 이야기하자면 구식 프로그램들은 `connect()` 에 넘겨줄 `struct sockaddr_in` 을 직접 채워 넣었습니다. 그렇게 하고 싶다면 해도 됩니다. 위에 있는 `bind()` 절 에서 비슷한 내용을 참고하세요.

`connect()` 의 반환값을 확인하는 것을 잊지 마세요. 오류가 발생하면 `-1` 을 반환하고 `errno` 변수를 설정합니다.

우리가 `bind()` 를 호출하지 않았음에 주목하세요. 간단히 말하자면 우리의 로컬 포트 번호에 대해서는 신경 쓰지 않습니다. 우리가 어디로 가는지만 신경 씁니다 (원격지 포트). 커널이 우리 대신 로컬 포트를 고를 것입니다. 우리가 접속하는 사이트는 이 정보를 자동으로 우리에게서 얻어냅니다. 신경 쓰실 필요가 없습니다.

5.5 listen() —누가 연락 좀 해줄래요?

이제 흐름이 변할 때입니다. 우리가 원격지 호스트에 접속하고 싶지 않은 경우라면 어떻게 할까요? 간단히 말해서 들어오는 연결을 기다리고 그것을 어떤 방식으로 처리해야 한다면 어떻게 할까요? 그 과정은 두 단계입니다. 먼저 `listen()` 을 호출하고, `accept()` 를 씁니다.(아래를 참고하세요.)

`listen()` 함수 호출은 꽤 단순하지만 약간의 설명이 필요합니다:

```
int listen(int sockfd, int backlog);
```

`sockfd` 는 `socket()` 시스템 함수 호출로 얻어온 평범한 소켓 파일 설명자입니다. `backlog` 는 들어오는 큐에 허용되는 연결의 수입니다. 이것이 무슨 뜻인지 궁금한가요? 들어오는 연결들은 여러분이 `accept()` 를 해주기 전까지(아래를 참고하세요) 이 큐 안에서 기다릴 것이고 이것은 몇 개의 연결이 대기할 수 있는가를 정합니다. 대개의 시스템은 이 값을 조용히 20 정도로 제한합니다. 그러나 5 나 10 정도의 값으로 설정해도 괜찮을 것입니다.

(역자 주: 실제 `backlog` 상한은 운영체제와 커널 설정에 따라 다릅니다.)

또 평소와 다름없이 `listen()` 도 오류가 발생할 경우 `-1` 을 반환하고 `errno` 를 설정합니다.

아마도 상상하실 수 있겠지만 서버가 특정 포트에서 실행되도록 하기 위해서는 `listen()` 을 호출하기 전에 `bind()` 를 호출해야 합니다. (여러분의 친구들에게 어떤 포트로 연결해야 할지 말해줄 수 있어야 합니다.) 이런 식입니다.

```
getaddrinfo();
socket();
bind();
listen();
/* accept()는 아래에 옵니다 */
```

저 코드는 충분히 자명하므로 예제 코드를 대신하게 두겠습니다. (`accept()` 절의 코드는 좀 더 완전합니다.) 정말로 복잡한 부분은 `accept()` 을 호출하는 부분입니다.

5.6 accept() —“3490 포트에 접속해주셔서 감사합니다.”

각오하세요! `accept()` 함수는 조금 이상합니다. 지금부터 생길 일은 다음과 같습니다. 아주 먼 곳에 있는 누군가가 여러분의 컴퓨터에 `connect()` 함수로 연결하려고 합니다. 여러분은 특정 포트에서 `listen()` 을 실행하고 있습니다. 그들의 연결은 `accept()` 로 받아들여질 때까지 대기열에 쌓일 것입니다. 여러분은

`accept()` 를 호출해서 대기 중인 연결을 받아들이겠다고 알려줍니다. `accept()` 는 이 연결에만 쓸 완전히 새로운 소켓 파일 설명자를 반환합니다. 그렇습니다! 갑자기 소켓 하나 가격에 두 개의 소켓을 가지게 되었습니다. 원래의 소켓은 여전히 새 연결들을 리스닝하고 있고, 새롭게 만들어진 것은 `send()` 와 `recv()` 작업을 위해 준비되었습니다. 이제 다 됐군요!

호출은 아래와 같이 합니다.

```
#include <sys/types.h>
#include <sys/socket.h>

int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
```

`sockfd` 는 `listen()` 을 하고 있는 소켓 설명자입니다. 어렵지 않습니다. `addr` 은 대개 로컬 `struct sockaddr_storage` 에 대한 포인터입니다. 여기에 들어오는 연결의 정보가 들어가게 됩니다(그리고 그것을 통해서 어떤 호스트가 어떤 포트에서 여러분을 호출하고 있는지 알 수 있습니다.) `addrlen` 은 로컬 정수 변수로, 그 주소를 `accept()` 에 넘기기 전에 `sizeof(struct sockaddr_storage)` 으로 설정해 두어야 합니다. `accept()` 는 그보다 많은 바이트를 `addr` 에 적지 않을 것입니다. 더 적은 바이트를 적었다면, 그 사실을 반영하도록 `addrlen` 값을 바꿀 것입니다. `accept()` 도 오류가 발생하면 `-1` 을 반환하고 `errno` 를 설정합니다. 어느 정도는 예상하셨으리라 생각합니다.

전과 마찬가지로 한 번에 이해하기에는 많은 내용입니다. 그래서 여러분의 독서를 위한 예제 코드 조각이 있습니다.

```
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>

#define MYPOR "3490" // 사용자들이 접속할 포트
#define BACKLOG 10 // 대기열이 담을 수 있는 연결의 개수

int main(void)
{
    struct sockaddr_storage their_addr;
    socklen_t addr_size;
    struct addrinfo hints, *res;
    int sockfd, new_fd;

    // !! 이 호출들에 대한 오류 확인을 잊지 마세요 !!

    // getaddrinfo()로 주소 구조체 목록을 가져옵니다

    memset(&hints, 0, sizeof hints);
    hints.ai_family = AF_UNSPEC; // IPv4 또는 IPv6, 아무것이나 씁니다
    hints.ai_socktype = SOCK_STREAM;
    hints.ai_flags = AI_PASSIVE; // bind()에 쓸 와일드카드 주소를 요청합니다

    getaddrinfo(NULL, MYPOR, &hints, &res);

    // 소켓을 만들고, 바인드하고, 리스닝 시작

    sockfd = socket(res->ai_family, res->ai_socktype,
                    res->ai_protocol);
```

```

bind(sockfd, res->ai_addr, res->ai_addrlen);
listen(sockfd, BACKLOG);

// 들어오는 연결을 받습니다

addr_size = sizeof their_addr;
new_fd = accept(sockfd, (struct sockaddr *)&their_addr,
                &addr_size);

// new_fd 소켓 설명자에서 통신할 준비 완료!
.
.
.

```

다시 말씀드리지만 모든 `send()` 와 `recv()` 호출에 대해서 `new_fd` 를 사용할 것입니다. 만약 단 한 개의 연결만을 받아들이길 원하신다면 추가적인 연결이 같은 포트를 통해 들어오는 것을 막기 위해서 `sockfd` 를 `close()` 할 수 있습니다.

5.7 `send()` 와 `recv()` —말 좀 해봐요!

이 두 함수들은 스트림 소켓이나 연결된 데이터그램 소켓을 통해 통신하는 데 쓰입니다. 일반적인 연결하지 않은 데이터그램 소켓을 쓰고 싶다면 `sendto()` 와 `recvfrom()` 절을 보시면 됩니다.

새로울 수도, 아닐 수도 있는 이야기를 하나 하겠습니다. 이 함수들은 **블록(대기)** 호출입니다. 다시 말해 `recv()` 는 받을 데이터가 준비될 때까지 블록됩니다. “그런데 ‘블록’된다는 게 대체 무슨 뜻인가요?”라는 생각이 들 수 있습니다. 그 말은 누군가가 여러분에게 뭔가를 보낼 때까지 프로그램이 그 시스템 호출 지점에서 멈춰 있다는 뜻입니다. (그 문장에서 “멈춘다”에 해당하는 운영체제 쪽 전문 용어는 사실 `_잠든다`라서, 제가 이 표현들을 서로 바꿔 쓸 수도 있습니다.) `send()` 도 여러분이 보내는 내용이 어떤 식으로든 꼭 막혀 있으면 블록될 수도 있지만, 그런 경우는 더 드뭅니다. 이 개념은 나중에 다시 살펴보고, 필요할 때 어떻게 피하는지도 이야기할 것입니다.

`send()` 함수는 이렇습니다:

```
int send(int sockfd, const void *msg, int len, int flags);
```

`sockfd` 는 데이터를 보내고 싶은 소켓 설명자(`socket()` 으로 만들었든 `accept()` 로 만들었든)입니다. `msg` 는 여러분이 보낼 데이터에 대한 포인터이며, `len` 은 그 데이터의 바이트 단위 길이입니다. `flags` 는 그냥 0 으로 설정하세요. (`flags` 에 대해 더 자세히 알고 싶다면 `send()` 의 맨페이지를 보세요.)

예제 코드는 이렇게 될 수 있겠습니다.

```

char *msg = "Beej was here!";
int len, bytes_sent;
.
.
.
len = strlen(msg);
bytes_sent = send(sockfd, msg, len, 0);
.

```

.

.

`send()` 는 실제로 전송한 바이트 수를 반환합니다. 이 값은 여러분이 보내라고 말한 수보다 적을 수 있습니다! 때때로 엄청난 크기의 데이터를 전송하라고 요청하면 그것이 다 처리되지 못할 수 있는 것입니다. `send()` 는 보낼 수 있는 만큼을 보내고, 나머지는 여러분이 나중에 다시 보낼 것이라고 믿습니다. `send()` 가 반환한 값이 `len` 과 일치하지 않는다면 문자열의 나머지를 전송하는 일은 여러분에게 달렸음을 기억하세요. 이것에 관한 좋은 소식은 패킷이 작다면(1KB 미만 정도) 그것은 아마도 한 번에 모든 것을 보낼 수 있으리라는 점입니다. 다시 강조하지만 오류가 발생하면 `-1` 이 반환되고 `errno` 가 오류 번호로 설정됩니다.

`recv()` 함수는 많은 면에서 유사합니다:

```
int recv(int sockfd, void *buf, int len, int flags);
```

`sockfd` 는 읽어 들일 소켓 설명자이며, `buf` 는 정보를 읽어 들일 버퍼이고, `len` 은 버퍼의 최대 길이이고 `flags` 는 여기서도 0으로 설정될 수 있습니다. (플래그 정보에 대해서는 `recv()` 의 맨페이지를 참고하세요.)

`recv()` 는 실제로 버퍼에 읽어 들인 바이트 수를 반환하고, 오류가 발생하면 `errno` 를 적절한 값으로 설정한 뒤 `-1` 을 반환합니다.

잠깐! `recv()` 는 0을 반환할 수 있습니다. 이것은 한 가지 의미입니다. 상대방에서 여러분과의 연결을 닫은 것입니다! 반환값 0은 `recv()` 가 연결이 끊어졌음을 알려주는 방식입니다.

자, 정말 쉽지요? 이제 여러분은 스트림 소켓에서 데이터를 주고받을 수 있습니다. 와! 이제 여러분은 어엿한 유닉스 네트워크 프로그래머입니다!

5.8 `sendto()` 와 `recvfrom()` —DGRAM 방식으로 말해 봐요

“이제 다 깔끔하고 좋네요”라고 말씀하시는 소리가 들립니다. “그렇지만 연결이 없는 데이터그램 소켓은 어떻게 처리하지요?”라고도 하시는군요. 걱정 마세요, 친구여. 딱 맞는 것이 있습니다.

데이터그램 소켓은 원격지 호스트에 연결되어 있지 않으므로, 패킷을 보낼 때에 필요한 정보는 조금 다릅니다. 바로 목적지 주소가 필요합니다. 이런 식입니다.

```
int sendto(int sockfd, const void *msg, int len, unsigned int flags,
           const struct sockaddr *to, socklen_t tolen);
```

보시다시피 `send()` 와 같지만 두 개의 정보가 더 있습니다. `to` 는 목적지의 IP 주소와 포트를 담은 `struct sockaddr` 에 대한 포인터이며(아마도 여러분이 형변환해서 사용하실 `struct sockaddr_in` 이나 `struct sockaddr_in6` 또는 `struct sockaddr_storage` 일 것입니다.) `tolen` 은 내부적으로는 `int` 이며 간단하게 `sizeof *to` 나 `sizeof(struct sockaddr_storage)` 로 설정하면 됩니다.

목적지 주소 구조체는 `getaddrinfo()` 나 아래의 `recvfrom()` 으로 얻거나 수작업으로 채울 수 있습니다.

`send()` 와 마찬가지로 `sendto()` 도 실제로 보낸 바이트 수를 반환합니다. (그 말은 보내려고 한 바이트의 수보다 적은 수가 돌아올 수도 있다는 의미입니다.) 오류가 발생하면 `-1` 을 반환합니다.

이와 유사한 관계가 `recv()` 와 `recvfrom()` 입니다. `recvfrom()` 의 개요는 이렇습니다.

```
int recvfrom(int sockfd, void *buf, int len, unsigned int flags,
             struct sockaddr *from, int *fromlen);
```

또 다시 이것은 몇 개의 추가적인 필드가 있는 `recv()` 같은 것입니다. `from` 은 보낸 호스트의 IP 주소와 포트가 채워지는 로컬 `struct sockaddr_storage` 에 대한 포인터입니다. `fromlen` 은 로컬 `int` 에 대한 포인터이며 `sizeof *from` 이나 `sizeof(struct sockaddr_storage)` 으로 초기화되어야 합니다. 함수가 반환될 때 `fromlen` 은 `from` 에 실제로 저장된 주소의 길이로 설정되어 있을 것입니다.

`recvfrom()` 은 받은 바이트의 개수를 반환하며 오류가 나면 (`errno` 를 적절히 설정하고) `-1` 을 반환합니다.

여기 질문이 하나 있습니다. 왜 우리는 `struct sockaddr_storage` 를 소켓의 타입으로 사용할까요? 왜 그냥 `struct sockaddr_in` 을 쓸 수 없을까요? 이유는 보시다시피 우리가 IPv4나 IPv6 중 하나에 얽매이고 싶지 않기 때문입니다. 그래서 우리는 양쪽 모두에 충분히 크고 일반적인 `struct sockaddr_storage` 를 사용합니다.

(그럼... 여기에서 다른 질문 하나. 왜 `struct sockaddr` 을 모든 주소를 담을 수 있을 정도로 크게 만들지 않았을까요? 우리는 일반 목적의 `struct sockaddr_storage` 을 다시 일반 목적의 `struct sockaddr` 으로 형변환하고 있습니다! 이런 동작은 과하고 불필요해 보이지 않나요? 여기에 대한 대답은 그냥 이 `struct sockaddr` 은 만들어질 때부터 그렇게 크지 않았다는 것이고, 이제 와서 그것을 바꾸는 것은 문제의 소지가 있다는 것입니다. 그래서 그들은 그냥 새로운 타입을 만들었습니다.)

만약 여러분이 데이터그램 소켓을 `connect()` 하게 되면 모든 통신에 `send()` 와 `recv()` 를 쓸 수 있음을 기억하세요. 소켓 자체는 여전히 데이터그램 소켓일 것이고 패킷은 여전히 UDP를 사용할 것이지만 소켓 인터페이스가 자동으로 여러분을 위해서 목적지와 출발지 정보를 추가할 것입니다.

5.9 `close()` 와 `shutdown()` —이제 그만 가!

휴! 여러분은 하루 종일 `send()` 와 `recv()` 를 사용했고, 이제 충분합니다. 이제 여러분의 소켓 설명자를 닫을 준비가 되었습니다. 이건 쉽습니다. 그냥 평범한 유닉스 파일 설명자 닫기 함수인 `close()` 를 쓸 수 있습니다.

```
close(sockfd);
```

이것은 해당 소켓에 대한 후속 읽기와 쓰기를 방지할 것입니다. 원격지에서 이 소켓을 쓰거나 읽으려는 모든 시도는 오류를 반환할 것입니다.

소켓이 어떻게 닫히는지 좀 더 조절하고 싶은 경우에는 `shutdown()` 함수를 사용할 수 있습니다. 이것은 특정 방향으로의 통신만 끊는 일을 할 수 있으며 양쪽 모두 막을 수도 있습니다(마치 `close()` 가 하듯이). 개요는 이렇습니다.

```
int shutdown(int sockfd, int how);
```

`sockfd` 는 종료하고 싶은 소켓 파일 설명자이고, `how` 는 다음 중 하나입니다:

how	효과
0	후속 수신이 금지됩니다.
1	후속 송신이 금지됩니다.
2	후속 송수신이 금지됩니다. (<code>close()</code> 처럼)

`shutdown()` 은 성공 시에 `0` 을 반환하고, 오류가 발생하면 (`errno` 를 적절한 값으로 설정하고) `-1` 을 반환합니다.

연결하지 않은 데이터그램 소켓에 굳이 `shutdown()` 을 해주신다면, 그것은 단순히 해당 소켓에 `send()` 와 `recv()` 를 사용할 수 없도록 만들 것입니다(데이터그램 소켓에 `connect()` 를 사용하면 이 두 함수를 사용할 수 있음을 기억하세요).

`shutdown()` 이 실제로 파일 설명자를 닫지는 않음에 주목하세요. 소켓 설명자를 해제하기 위해서는 `close()` 를 호출해야 합니다.

별것 없군요.

(예외적으로 Windows에서 Winsock을 사용하는 경우 `close()` 대신 `closesocket()` 을 호출해야 합니다.)

5.10 `getpeername()` —누구세요?

이 함수는 너무 쉽습니다.

너무 쉬워서 이 함수에 별도의 절을 주지 않을 뻔했습니다. 아무튼 알려드리겠습니다.

`getpeername()` 함수는 연결된 스트림 소켓의 반대편 끝에 누가 있는지를 알려줄 것입니다. 개요입니다.

```
#include <sys/socket.h>

int getpeername(int sockfd, struct sockaddr *addr, int *addrlen);
```

`sockfd` 는 연결된 스트림 소켓의 설명자입니다. `addr` 은 연결의 반대편 끝에 대한 정보를 담은 `struct sockaddr` (또는 `struct sockaddr_in`) 에 대한 포인터입니다. `addrlen` 은 `int` 에 대한 포인터이며 `sizeof *addr` 이나 `sizeof(struct sockaddr)` 으로 초기화되어야 합니다.

이 함수는 오류가 발생하면 `-1` 을 반환하고 `errno` 를 알맞게 설정합니다.

여러분이 상대방의 주소를 가지면 그것을 `inet_ntop()`, `getnameinfo()` 또는 `gethostbyaddr()` 에 넣어서 화면에 출력하거나 추가적인 정보를 가져올 수 있습니다. 그들의 로그인 이름을 가져올 수는 없습니다. (좋습니다, 좋아요. 만약 저쪽 컴퓨터가 ident 데몬을 실행 중이라면 가능합니다. 그러나 그것은 이 문서의 범위를 넘어섭니다. 더 자세한 정보를 원한다면 RFC 1413³을 참고하세요.)

5.11 `gethostname()` —나는 누구인가?

`getpeername()` 보다 더 쉬운 것이 바로 `gethostname()` 함수입니다. 이것은 여러분의 프로그램이 실행되고 있는 컴퓨터의 이름을 반환합니다. 반환된 이름은 위에 있는 `getaddrinfo()` 을 써서 여러분의 로컬 시스템의 IP 주소를 알아내는 데 쓰일 수 있습니다.

이보다 더 재미있는 일이 있을까요? 사실 몇 가지 생각나긴 합니다만 소켓 프로그래밍에 대한 것이 아니군요. 아무튼 정리하자면 이렇습니다.

```
#include <unistd.h>

int gethostname(char *hostname, size_t size);
```

³<https://tools.ietf.org/html/rfc1413>

인수들은 단순합니다. `hostname` 은 함수가 반환하는 호스트 이름을 담을 `char` 배열에 대한 포인터입니다. `size` 는 `hostname` 배열의 바이트 단위 길이입니다.

함수는 성공적인 완료 후에 `0` 을 반환하고, 오류에 대해서는 흔히 그렇듯 `errno` 를 설정하고 `-1` 을 반환합니다.

Chapter 6

클라이언트-서버 배경지식

클라이언트-서버에 대해 이야기할 차례입니다. 통신망에 있는 거의 모든 것은 클라이언트 프로세스가 서버 프로세스와 이야기하거나 그 반대로 동작합니다. `telnet` 을 예로 들어봅시다. 여러분이 텔넷(클라이언트)로 원격지 호스트의 23번 포트에 접속할 때 그 호스트의 프로그램(`telnetd` 라고 불리는 서버)이 살아납니다. 그 프로그램이 들어오는 텔넷 요청을 처리하고 여러분에게 로그인 프롬프트를 띄워주는 등의 일을 처리합니다.

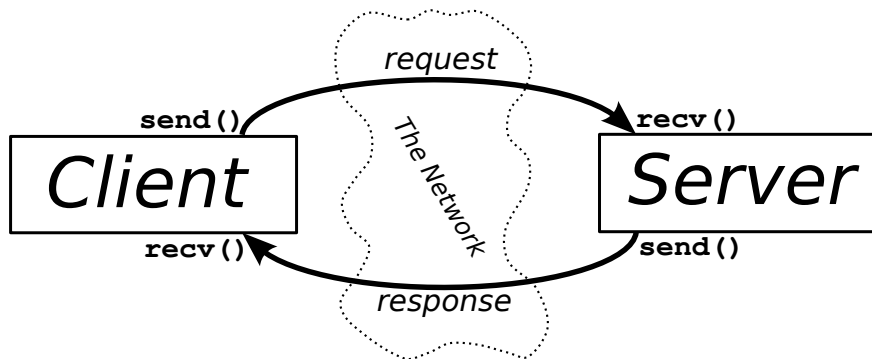


Figure 6.1: 클라이언트-서버 상호작용

위의 도표에 클라이언트와 서버의 정보 교환이 정리되어 있습니다.

클라이언트-서버 쌍은 `SOCK_STREAM` 이나 `SOCK_DGRAM` 또는 다른 어떤 방식이라도 사용할 수 있음을 기억하세요. (둘이 같은 방식으로 말하기만 한다면) 클라이언트-서버 쌍의 좋은 예시는 `telnet / telnetd`, `ftp / ftpd` 또는 `Firefox / Apache` 입니다. 여러분이 `ftp` 를 쓸 때마다 여러분의 요청을 받아들이는 원격지 프로그램인 `ftpd` 가 있습니다.

흔히 한 대의 컴퓨터에는 서버가 하나뿐이고, 그 서버는 `fork()` 를 통해 여러 클라이언트를 처리합니다. 기본적인 과정은 아래와 같습니다. 서버가 연결을 기다리고, `accept()` 한 후, 요청을 처리할 자식 프로세스를 `fork()` 합니다. 이것이 다음 절에서 우리의 예제 서버가 하는 일입니다.

6.1 단순한 스트림 서버

이 서버가 하는 일은 스트림 연결에 "Hello, world!" 을 전송하는 것입니다. 이 서버를 시험하기 위해서 할 일은 하나의 창에서 이것을 실행한 후 다른 창에서 텔넷에 아래 명령어로 접속하는 일뿐입니다.

```
$ telnet remotehostname 3490
```

(역자 주: 최신 macOS에는 `telnet` 이 기본 포함되지 않습니다. Debian과 Fedora 등의 Linux 배포판에서는 패키지로 계속 제공됩니다. `telnet` 을 쓸 수 없다면 이 책에 포함된 `telnot` 예제¹를 컴파일해 데모에 사용할 수 있습니다.)

`remotehostname` 은 서버를 실행하는 컴퓨터의 이름입니다.

서버 코드²:

```
/*
** server.c -- a stream socket server demo
*/

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <errno.h>
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>
#include <arpa/inet.h>
#include <sys/wait.h>
#include <signal.h>

#define PORT "3490" // 사용자들이 접속할 포트
#define BACKLOG 10 // 대기 중인 연결을 몇 개까지 큐에 둘 것인가

void sigchld_handler(int s)
{
    (void)s; // 사용하지 않는 변수 경고를 끕니다

    // waitpid()가 errno를 덮어쓸 수 있으므로 저장했다가 복원합니다.
    int saved_errno = errno;

    while(waitpid(-1, NULL, WNOHANG) > 0);

    errno = saved_errno;
}

// IPv4 또는 IPv6 주소를 가져옵니다.
void *get_in_addr(struct sockaddr *sa)
{
    if (sa->sa_family == AF_INET) {
        return &(((struct sockaddr_in*)sa)->sin_addr);
    }
}
```

¹<https://beej.us/guide/bgnet/source/examples/telnot.c>

²<https://beej.us/guide/bgnet/source/examples/server.c>

```

    return &(((struct sockaddr_in6*)sa)->sin6_addr);
}

int main(void)
{
    // sockfd에서 대기하고 들어오는 연결은 new_fd에 저장
    int sockfd, new_fd;
    struct addrinfo hints, *servinfo, *p;
    struct sockaddr_storage their_addr; // 접속한 쪽의 주소 정보
    socklen_t sin_size;
    struct sigaction sa;
    int yes=1;
    char s[INET6_ADDRSTRLEN];
    int rv;

    memset(&hints, 0, sizeof hints);
    hints.ai_family = AF_INET;
    hints.ai_socktype = SOCK_STREAM;
    hints.ai_flags = AI_PASSIVE; // bind()에 쓸 와일드카드 주소를 요청합니다

    if ((rv = getaddrinfo(NULL, PORT, &hints, &servinfo)) != 0) {
        fprintf(stderr, "getaddrinfo: %s\n", gai_strerror(rv));
        return 1;
    }

    // 모든 결과를 순회해 사용할 수 있는 첫 번째 결과에 바인드합니다
    for(p = servinfo; p != NULL; p = p->ai_next) {
        if ((sockfd = socket(p->ai_family, p->ai_socktype,
                           p->ai_protocol)) == -1) {
            perror("server: socket");
            continue;
        }

        if (setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, &yes,
                      sizeof(int)) == -1) {
            perror("setsockopt");
            exit(1);
        }

        if (bind(sockfd, p->ai_addr, p->ai_addrlen) == -1) {
            close(sockfd);
            perror("server: bind");
            continue;
        }

        break;
    }

    freeaddrinfo(servinfo); // 이 구조체는 이제 필요 없습니다

    if (p == NULL) {
        fprintf(stderr, "server: failed to bind\n");
        exit(1);
    }

```

```

    }

    if (listen(sockfd, BACKLOG) == -1) {
        perror("listen");
        exit(1);
    }

    sa.sa_handler = sigchld_handler; // 죽은 프로세스를 모두 거둡니다
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART;
    if (sigaction(SIGCHLD, &sa, NULL) == -1) {
        perror("sigaction");
        exit(1);
    }

    printf("server: waiting for connections...\n");

    while(1) { // 주 accept() 루프
        sin_size = sizeof their_addr;
        new_fd = accept(sockfd, (struct sockaddr *)&their_addr,
            &sin_size);
        if (new_fd == -1) {
            perror("accept");
            continue;
        }

        inet_ntop(their_addr.ss_family,
            get_in_addr((struct sockaddr *)&their_addr),
            s, sizeof s);
        printf("server: got connection from %s\n", s);

        if (!fork()) { // 자식 프로세스입니다.
            close(sockfd); // 자식은 리스너가 필요 없습니다.
            if (send(new_fd, "Hello, world!", 13, 0) == -1)
                perror("send");
            close(new_fd);
            exit(0);
        }
        close(new_fd); // 부모는 이것이 필요 없습니다.
    }

    return 0;
}

```

궁금한 독자들을 위해 덧붙이자면 구문의 명료함을 위해서 하나의 큰 `main()` 함수 안에 모든 코드를 다 적었습니다. 원한다면 더 작은 함수들로 나누어도 좋습니다.

(아마도 이 `sigaction()` 을 처음 볼 수도 있는데 괜찮습니다. 이 코드는 `fork()` 된 자식 프로세스가 종료되면서 생기는 좀비 프로세스를 거둬들이는 데 사용됩니다. 좀비 프로세스를 많이 만들고 거둬들이지 않으면 시스템 관리자가 화를 낼 것입니다.)

다음 절에 나오는 클라이언트를 사용해서 이 서버로부터 데이터를 얻을 수 있습니다.

6.2 단순한 스트림 클라이언트

이 녀석은 서버보다도 더 쉽습니다. 이 클라이언트가 하는 일은 여러분이 명령줄에 지정한 호스트의 3490번 포트로 접속하는 것입니다. 그리고 서버가 보낸 문자열을 받습니다.

클라이언트 소스³:

```
/*
** client.c -- 스트림 소켓 클라이언트의 예시
*/

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <errno.h>
#include <string.h>
#include <netdb.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <sys/socket.h>

#include <arpa/inet.h>

#define PORT "3490" // 클라이언트가 접속할 포트

#define MAXDATASIZE 100 // 한 번에 받을 수 있는 최대 바이트 개수

// IPv4 또는 IPv6 sockaddr을 얻습니다
void *get_in_addr(struct sockaddr *sa)
{
    if (sa->sa_family == AF_INET) {
        return &(((struct sockaddr_in*)sa)->sin_addr);
    }

    return &(((struct sockaddr_in6*)sa)->sin6_addr);
}

int main(int argc, char *argv[])
{
    int sockfd, numbytes;
    char buf[MAXDATASIZE];
    struct addrinfo hints, *servinfo, *p;
    int rv;
    char s[INET6_ADDRSTRLEN];

    if (argc != 2) {
        fprintf(stderr, "usage: client hostname\n");
        exit(1);
    }

    memset(&hints, 0, sizeof hints);
    hints.ai_family = AF_UNSPEC;
```

³<https://beej.us/guide/bgnet/source/examples/client.c>

```

hints.ai_socktype = SOCK_STREAM;

if ((rv = getaddrinfo(argv[1], PORT, &hints, &servinfo)) != 0) {
    fprintf(stderr, "getaddrinfo: %s\n", gai_strerror(rv));
    return 1;
}

// 모든 결과를 순회해 연결할 수 있는 첫 번째 결과를 사용합니다
for(p = servinfo; p != NULL; p = p->ai_next) {
    if ((sockfd = socket(p->ai_family, p->ai_socktype,
        p->ai_protocol)) == -1) {
        perror("client: socket");
        continue;
    }

    inet_ntop(p->ai_family,
        get_in_addr((struct sockaddr *)p->ai_addr),
        s, sizeof s);
    printf("client: attempting connection to %s\n", s);

    if (connect(sockfd, p->ai_addr, p->ai_addrlen) == -1) {
        perror("client: connect");
        close(sockfd);
        continue;
    }

    break;
}

if (p == NULL) {
    fprintf(stderr, "client: failed to connect\n");
    return 2;
}

inet_ntop(p->ai_family,
    get_in_addr((struct sockaddr *)p->ai_addr),
    s, sizeof s);
printf("client: connected to %s\n", s);

freeaddrinfo(servinfo); // 이 구조체는 이제 필요 없습니다

if ((numbytes = recv(sockfd, buf, MAXDATASIZE-1, 0)) == -1) {
    perror("recv");
    exit(1);
}

buf[numbytes] = '\0';

printf("client: received '%s'\n", buf);

close(sockfd);

return 0;

```

```
}

```

클라이언트를 실행하기 전에 서버를 실행하지 않으면 `connect()` 는 "Connection refused"를 반환한다는 점을 기억하세요. 아주 유용합니다.

6.3 데이터그램 소켓

위에서 `sendto()` 와 `recvfrom()` 에 대해 논의할 때 UDP 데이터그램 소켓의 기본에 대해서 이미 알아보았습니다. 그러므로 바로 두 개의 예제 프로그램을 제시하겠습니다. `talker.c` 와 `listener.c` 입니다.

`listener` 는 한 컴퓨터에서 포트 4950으로 들어오는 패킷을 기다립니다. `talker` 는 지정한 컴퓨터의 해당 포트에 사용자가 명령줄에 입력한 내용을 담은 패킷을 보냅니다.

데이터그램 소켓은 연결이 없고 패킷을 허공에 던진 뒤 성공 여부는 신경 쓰지 않기 때문에 클라이언트와 서버에 IPv6을 사용하도록 명시할 것입니다. 이렇게 하면 서버는 IPv6로 리스닝하는데 클라이언트는 IPv4로 전송하여, 서버가 데이터를 받지 못하는 상황을 피할 수 있을 것입니다. (우리의 TCP 스트림 소켓 세상에서도 이런 불일치가 발생할 수 있지만 `connect()` 에서 하나의 주소 계열에 대해 오류가 발생하면 다른 주소 계열로 다시 시도하게 됩니다.)

여기에 `listener.c` 의 소스 코드가 있습니다.⁴:

```
/*
** listener.c -- 데이터그램 소켓 "서버"의 예시
*/

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <errno.h>
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <netdb.h>

#define MYPORT "4950"    // 연결할 포트

#define MAXBUFLen 100

// IPv4 또는 IPv6 주소를 가져옵니다:
void *get_in_addr(struct sockaddr *sa)
{
    if (sa->sa_family == AF_INET) {
        return &(((struct sockaddr_in*)sa)->sin_addr);
    }

    return &(((struct sockaddr_in6*)sa)->sin6_addr);
}
```

⁴<https://beej.us/guide/bgnet/source/examples/listener.c>

```

int main(void)
{
    int sockfd;
    struct addrinfo hints, *servinfo, *p;
    int rv;
    int numbytes;
    struct sockaddr_storage their_addr;
    char buf[MAXBUFLEN];
    socklen_t addr_len;
    char s[INET6_ADDRSTRLEN];

    memset(&hints, 0, sizeof hints);
    hints.ai_family = AF_INET6; // IPv4를 쓰려면 AF_INET으로 설정합니다
    hints.ai_socktype = SOCK_DGRAM;
    hints.ai_flags = AI_PASSIVE; // bind()에 쓸 와일드카드 주소를 요청합니다

    if ((rv = getaddrinfo(NULL, MYPORT, &hints, &servinfo)) != 0) {
        fprintf(stderr, "getaddrinfo: %s\n", gai_strerror(rv));
        return 1;
    }

    // 모든 결과를 순회하면서 가능한 첫 번째 것에 바인드합니다
    for(p = servinfo; p != NULL; p = p->ai_next) {
        if ((sockfd = socket(p->ai_family, p->ai_socktype,
            p->ai_protocol)) == -1) {
            perror("listener: socket");
            continue;
        }

        if (bind(sockfd, p->ai_addr, p->ai_addrlen) == -1) {
            close(sockfd);
            perror("listener: bind");
            continue;
        }

        break;
    }

    if (p == NULL) {
        fprintf(stderr, "listener: failed to bind socket\n");
        return 2;
    }

    freeaddrinfo(servinfo);

    printf("listener: waiting to recvfrom...\n");

    addr_len = sizeof their_addr;
    if ((numbytes = recvfrom(sockfd, buf, MAXBUFLEN-1, 0,
        (struct sockaddr *)&their_addr, &addr_len)) == -1) {
        perror("recvfrom");
        exit(1);
    }
}

```

```

    printf("listener: got packet from %s\n",
           inet_ntop(their_addr.ss_family,
                     get_in_addr((struct sockaddr *)&their_addr),
                     s, sizeof s));
    printf("listener: packet is %d bytes long\n", numbytes);
    buf[numbytes] = '\0';
    printf("listener: packet contains \"%s\"\n", buf);

    close(sockfd);

    return 0;
}

```

`getaddrinfo()` 에서 우리가 마침내 `SOCK_DGRAM` 을 사용한다는 것에 주목하세요. 또한, `listen()` 과 `accept()` 가 필요하지 않다는 점도 기억하세요. 이것이 연결 없는 데이터그램 소켓을 사용할 때의 장점 중 하나입니다.

다음으로 `talker.c` 의 소스 코드⁵입니다.

```

/*
** talker.c -- 데이터그램 "클라이언트"의 예시
*/

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <errno.h>
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <netdb.h>

#define SERVERPORT "4950" // 사용자들이 접속할 포트

int main(int argc, char *argv[])
{
    int sockfd;
    struct addrinfo hints, *servinfo, *p;
    int rv;
    int numbytes;

    if (argc != 3) {
        fprintf(stderr, "usage: talker hostname message\n");
        exit(1);
    }

    memset(&hints, 0, sizeof hints);
    hints.ai_family = AF_INET6; // IPv4를 쓰려면 AF_INET으로 설정합니다

```

⁵<https://beej.us/guide/bgnet/source/examples/talker.c>

```

hints.ai_socktype = SOCK_DGRAM;

if ((rv = getaddrinfo(argv[1], SERVERPORT, &hints, &servinfo)) != 0) {
    fprintf(stderr, "getaddrinfo: %s\n", gai_strerror(rv));
    return 1;
}

// 모든 결과를 순회하면서 소켓을 만듭니다
for(p = servinfo; p != NULL; p = p->ai_next) {
    if ((sockfd = socket(p->ai_family, p->ai_socktype,
        p->ai_protocol)) == -1) {
        perror("talker: socket");
        continue;
    }

    break;
}

if (p == NULL) {
    fprintf(stderr, "talker: failed to create socket\n");
    return 2;
}

if ((numbytes = sendto(sockfd, argv[2], strlen(argv[2]), 0,
    p->ai_addr, p->ai_addrlen)) == -1) {
    perror("talker: sendto");
    exit(1);
}

freeaddrinfo(servinfo);

printf("talker: sent %d bytes to %s\n", numbytes, argv[1]);
close(sockfd);

return 0;
}

```

이것이 전부입니다! 하나의 장치에서 `listener`를 실행하고 `talker`를 다른 장치에서 실행하세요. 그것들이 통신하는 것을 지켜보세요.

(역자 주 : 한 장치에서도 순서만 맞게 실행하면 문제없이 시험할 수 있습니다.)

온 가족이 즐길 수 있는 전연령 프로그램입니다!

이번에는 서버를 실행할 필요도 없습니다! `talker`를 혼자 실행시키면 패킷을 신나게 날려 보내고, 아무도 반대쪽에서 `recvfrom()`을 호출하지 않는다면 그저 패킷은 사라질 뿐입니다. UDP 데이터그램 소켓으로 보낸 데이터는 도착을 보장하지 않는다는 점을 기억하세요!

전에 몇 번 말한 사소한 것 한 가지를 빼면 전부입니다: 연결된 데이터그램 소켓이 그것입니다. 그것에 대해서 여기에서 말해야 하는데, 이 문서의 데이터그램에 대한 부분이 바로 여기이기 때문입니다. 위의 `talker`가 `listener`의 주소를 지정하고 `connect()`를 호출한다고 합시다. 그 순간부터 `talker`는 `connect()`로 지정한 주소로만 데이터를 보내고 받을 수 있습니다. 이런 이유로 `sendto()`와 `recvfrom()`을 쓸 필요가 없습니다. 단순히 `send()`와 `recv()`를 쓰면 됩니다.

Chapter 7

약간 더 고급스러운 기술

이것들은 진짜로 고급 기술인 것은 아니지만 우리가 지금까지 다룬 기본적인 것들을 벗어나고 있습니다. 사실 여러분이 여기까지 왔다면 스스로가 기본적인 유닉스 네트워크 프로그래밍을 꽤 잘 한다고 생각하셔도 됩니다. 축하합니다.

이제 여러분이 소켓에 대해서 배우고 싶어 할 조금 더 난해한 것들의 세상으로 용감하게 가봅시다. 시작합니다!

7.1 블로킹

블로킹. 한 번쯤 들어보셨을 것입니다. 그게 무엇일까요? 간단히 말씀드리자면 호출이 데이터를 기다리느라 반환하지 못하는 상태입니다. 기술적으로는 이 호출이 "블록된다"고 합니다. 위에서 `listener` 를 실행하면 패킷이 도착할 때까지 그대로 기다리고 있다는 것을 눈치채셨을 것입니다. 내부적으로는 `recvfrom()` 이 호출되고, 데이터가 없으므로 `recvfrom()` 는 데이터가 도착할 때까지 "블록"된 상태인 것입니다. (즉 그대로 잠들어 있게 됩니다.)

많은 함수가 블록됩니다. `accept()` 와 모든 `recv()` 함수도 블록됩니다. 이 함수들이 블록되도록 허용되어 있기 때문입니다. `socket()` 으로 소켓 설명자를 처음 만들 때 커널이 이 소켓을 블로킹 소켓으로 설정합니다. 소켓이 블록되지 않게 하려면 `fcntl()` 을 호출해야 합니다:

```
#include <unistd.h>
#include <fcntl.h>
.
.
.
sockfd = socket(PF_INET, SOCK_STREAM, 0);
fcntl(sockfd, F_SETFL, O_NONBLOCK);
.
.
.
```

소켓을 논블로킹으로 설정하면 정보를 얻기 위해서 실질적으로 소켓을 "폴링" 할 수 있습니다. 논블로킹 소켓을 읽으려고 할 때 정보가 없다면 블록될 수 없으므로 `-1` 을 반환할 것이고 `errno` 는 `EAGAIN` 이나 `EWOULDBLOCK` 로 설정될 것입니다.

(잠깐, `EAGAIN` 이나 `EWOULDBLOCK` 를 반환한다니 무엇을 확인해야 한다는 말일까요? 명세서에는 사실 여러분의 시스템이 어떤 값을 반환해야 하는지 정의되어 있지 않습니다. 그러므로 이식성을 위해서는 둘을 모두 확인해야 합니다.)

그러나 일반적으로 말하자면 이런 방식의 조사는 좋은 생각이 아닙니다. 여러분의 프로그램이 소켓의 데이터를 기다리면서 바쁜 대기 상태가 되면 여러분의 프로그램은 보통의 프로그램보다 훨씬 CPU 시간을 많이 사용할 것입니다. (역자 주: 특별한 제한을 걸지 않으면 최대 단일 코어 하나를 100% 점유할 수 있습니다.) 읽을 데이터가 있는지 확인하는 더 나은 방법은 `poll()` 을 다루는 다음 절에 있습니다.

7.2 `poll()` — 동기 입출력 다중화

여러분이 정말로 해야 하는 일은 소켓 한 무더기를 한 번에 감시하고 그 중에 데이터가 준비된 것을 처리하는 것입니다. 이런 방식을 통해서 여러분은 모든 소켓을 지속적으로 조사하지 않아도 여러 개의 소켓 중 어떤 것이 데이터가 준비되었는지 알 수 있습니다.

경고: `poll()` 은 엄청나게 많은 수의 연결을 처리할 때 끔찍하게 느려집니다. 이런 상황에서는 `libevent`^a 같은 이벤트 라이브러리 를 사용하면 더 좋은 성능을 얻을 수 있습니다. 이런 라이브러리는 여러분의 운영체제에서 사용할 수 있는 가장 빠른 방법을 사용하려고 시도할 것입니다.

^a<https://libevent.org/>

그래서 어떻게 폴링을 피할 수 있을까요? 약간 아이러니하게도 `poll()` 시스템 함수를 사용해서 폴링을 피할 수 있습니다. 간단히 말하자면 모든 번거로운 작업을 운영체제에 맡기고, 어떤 소켓에 읽을 데이터가 준비되면 알려달라고 부탁하는 것입니다. 그 동안 우리의 프로세스는 대기 상태가 될 수 있고 시스템 자원을 아낄 수 있습니다.

전체적인 계획은 우리가 감시하고 싶은 소켓 설명자와 우리가 감시하고 싶은 이벤트의 종류에 대한 정보를 담은 `struct pollfd` 의 배열을 보관하는 것입니다. 해당하는 종류의 이벤트가 발생(예를 들어 “소켓에서 데이터를 읽을 수 있다!” 같은 이벤트)하거나 사용자가 지정한 제한 시간이 지날 때까지 `poll()` 호출은 블록됩니다.

유용하게도 `listen()` 상태인 소켓은 새로운 연결이 `accept()` 될 수 있는 상태가 되었을 때 “읽을 준비됨”을 반환할 것입니다.

이만하면 충분히 이야기했습니다. 이것을 쓰는 방법은 어떻게 봅시다.

```
#include <poll.h>

int poll(struct pollfd fds[], nfds_t nfds, int timeout);
```

`fds` 는 우리의 정보(어떤 소켓을 무엇을 위해 감시할지)의 배열입니다. `nfds` 는 배열에 담긴 요소의 개수입니다. `timeout` 은 밀리초 단위의 제한 시간입니다. 이 함수는 이벤트가 발생한 요소의 개수를 반환합니다.

위에 등장하는 구조체는 무엇인지 살펴봅시다.

```
struct pollfd {
    int fd;           // 소켓 설명자
    short events;     // 우리가 관심 있는 이벤트의 비트맵
    short revents;    // poll()이 반환하는 시점에 발생한 이벤트의 비트맵
};
```

이 구조체 타입의 배열을 하나 설정하고 각각의 `fd` 필드를 우리가 관찰하고 싶은 소켓 설명자로 설정합니다. 그리고 각각의 `events` 필드는 우리가 관심 있는 이벤트로 설정하는 것입니다.

`events` 필드는 아래 값들을 비트 단위 OR로 묶은 결과값입니다.

매크로	설명
POLLIN	이 소켓이 데이터를 <code>recv()</code> 할 준비가 되면 알려줍니다.
POLLOUT	이 소켓에 대기하지 않고 데이터를 <code>send()</code> 할 수 있으면 알려줍니다.
POLLHUP	원격 쪽이 연결을 닫으면 알려줍니다.

`struct pollfd`의 배열을 준비하면 `poll()`에 그것을 넘길 수 있습니다. 배열의 크기와 밀리초 단위의 제한 시간도 같이 넘겨야 합니다. (영원히 기다리려면 음수를 지정하면 됩니다.)

`poll()`이 반환하면 이벤트가 발생했음을 나타내는 `POLLIN`이나 `POLLOUT`이 설정되었는지 보기 위해서 `revents` 필드를 확인할 수 있습니다.

(실제로는 `poll()` 호출로 할 수 있는 것들이 더 있습니다. 자세한 내용은 아래의 `poll()` 맨페이지를 참고하세요.)

여기 표준 입력에서 데이터를 읽어 들일 수 있을 때까지(예를 들어 여러분이 줄바꿈을 입력할 때까지) 2.5초를 기다리는 예제¹가 있습니다.

```
#include <stdio.h>
#include <poll.h>

int main(void)
{
    struct pollfd pfd[1]; // 더 많은 것들을 관찰하고 싶다면 더 크게 하세요.

    pfd[0].fd = 0;          // 표준 입력
    pfd[0].events = POLLIN; // 읽을 준비가 되면 알려줍니다.

    // 만약 다른 것들도 관찰하고 싶다면
    //pfd[1].fd = some_socket; // 임의의 소켓 설명자
    //pfd[1].events = POLLIN; // 읽을 준비가 되면 알려줍니다.

    printf("엔터 키를 누르거나 시간 초과될 때까지 2.5초를 기다리세요\n");

    int num_events = poll(pfd, 1, 2500); // 2.5초 제한 시간

    if (num_events == 0) {
        printf("Poll 시간 초과!\n");
    } else {
        int pollin_happened = pfd[0].revents & POLLIN;

        if (pollin_happened) {
            printf("파일 설명자 %d를 읽을 준비가 되었습니다\n", pfd[0].fd);
        } else {
            printf("예상하지 못한 이벤트가 발생했습니다: %d\n", pfd[0].revents);
        }
    }

    return 0;
}
```

¹<https://beej.us/guide/bgnet/source/examples/poll.c>

`poll()` 이 `pfds` 배열에서 이벤트가 발생한 요소의 개수를 반환한다는 것을 다시 기억하세요. 배열의 어떤 요소에서 이벤트가 발생했는지는 알려주지 않지만 몇 개의 `revents` 필드가 0이 아닌 값으로 설정되었는지는 알려줍니다. 이것을 활용해서 반환된 숫자만큼의 0이 아닌 `revents` 를 읽은 후에는 스캔을 중단할 수 있습니다.

몇 가지 질문이 떠오를 것입니다. `poll()` 에 넘겨준 집합에 새 파일 설명자를 추가하려면 어떻게 해야 할까요? 이를 위해서 단순히 여러분의 모든 필요에 부합할 만큼 충분한 크기의 배열을 만들거나 추가적인 공간이 필요할 때 `realloc()` 을 사용하세요.

집합에서 요소를 제거하려면 어떻게 해야 할까요? 이것을 위해서 여러분은 배열의 마지막 요소를 삭제할 요소에 덮어쓰고, `poll()` 의 개수 인수에는 하나 더 적은 값을 전달하세요. (역자 주: 이것은 배열에서 임의의 요소 1개를 빠르게 제거하기 위해 일반적으로 사용하는 방법입니다.) 다른 한 가지 방법은 `fd` 필드를 음수로 설정하는 것이며 `poll()` 은 해당 요소를 무시할 것입니다.

이 모든 것을 여러분이 `telnet` 할 수 있는 하나의 채팅 서버에 합치려면 어떻게 해야 할까요?

우리가 할 일은 리스너 소켓을 시작한 후에 그것을 `poll()` 할 파일 설명자 집합에 추가하는 일입니다. (그 파일 설명자는 들어오는 연결이 있을 때 읽기 준비된 상태가 될 것입니다.)

그 후에 새로운 연결을 우리의 `struct pollfd` 배열에 추가하면 됩니다. 만약 배열의 크기가 부족하다면 동적으로 키우면 됩니다.

연결이 닫힌 후에는 그것을 배열로부터 제거합니다.

연결이 읽기 준비되면 우리는 그것에서 데이터를 읽어들이고 후 다른 모든 연결에 전송합니다. 그렇게 해서 사용자들은 서로가 입력한 내용을 볼 수 있습니다.

이제 이 폴 서버²를 한 번 시험해보세요. 이것을 하나의 창에서 실행한 후 몇 개의 다른 터미널 창에서 `telnet localhost 9034` 를 실행해 보세요. 여러분이 하나의 창에서 입력하는 것을 (여러분이 엔터 키를 누른 후에) 다른 창들에서 볼 수 있어야 합니다.

그뿐 아니라 여러분이 `CTRL-]` 를 누른 후 `quit` 을 입력해서 `telnet` 을 종료할 경우 서버는 연결 종료를 감지하고 그 연결을 파일 설명자 배열에서 제거할 것입니다.

```
/*
** pollserver.c -- 허술한 다중 사용자 대화 서버
*/

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <netdb.h>
#include <poll.h>

#define PORT "9034" // 리스닝할 포트

/*
* 소켓 주소를 IP 주소 문자열로 변환합니다.
* addr: struct sockaddr_in 또는 struct sockaddr_in6
```

²<https://beej.us/guide/bgnet/source/examples/pollserver.c>

```

/*
const char *inet_ntop2(void *addr, char *buf, size_t size)
{
    struct sockaddr_storage *sas = addr;
    struct sockaddr_in *sa4;
    struct sockaddr_in6 *sa6;
    void *src;

    switch (sas->ss_family) {
        case AF_INET:
            sa4 = addr;
            src = &(sa4->sin_addr);
            break;
        case AF_INET6:
            sa6 = addr;
            src = &(sa6->sin6_addr);
            break;
        default:
            return NULL;
    }

    return inet_ntop(sas->ss_family, src, buf, size);
}

/*
 * 리스닝 소켓을 반환합니다.
 */
int get_listener_socket(void)
{
    int listener;    // 리스닝 소켓 설명자
    int yes=1;       // 아래의 setsockopt() SO_REUSEADDR용
    int rv;

    struct addrinfo hints, *ai, *p;

    // 소켓을 얻어서 바인드합니다
    memset(&hints, 0, sizeof hints);
    hints.ai_family = AF_INET;
    hints.ai_socktype = SOCK_STREAM;
    hints.ai_flags = AI_PASSIVE;
    if ((rv = getaddrinfo(NULL, PORT, &hints, &ai)) != 0) {
        fprintf(stderr, "pollserver: %s\n", gai_strerror(rv));
        exit(1);
    }

    for(p = ai; p != NULL; p = p->ai_next) {
        listener = socket(p->ai_family, p->ai_socktype,
                          p->ai_protocol);
        if (listener < 0) {
            continue;
        }

        // 성가신 "address already in use" 오류 메시지를 피합니다

```

```

        setsockopt(listener, SOL_SOCKET, SO_REUSEADDR, &yes,
                    sizeof(int));

        if (bind(listener, p->ai_addr, p->ai_addrlen) < 0) {
            close(listener);
            continue;
        }

        break;
    }

    // 여기까지 왔다면 바인드하지 못했다는 뜻입니다
    if (p == NULL) {
        return -1;
    }

    freeaddrinfo(ai); // 이 구조체는 이제 필요 없습니다

    // 리스닝합니다
    if (listen(listener, 10) == -1) {
        return -1;
    }

    return listener;
}

/*
 * 집합에 새 파일 설명자를 추가합니다.
 */
void add_to_pfds(struct pollfd **pfds, int newfd, int *fd_count,
                int *fd_size)
{
    // 공간이 부족하면 pfds 배열 공간을 늘립니다
    if (*fd_count == *fd_size) {
        *fd_size *= 2; // 두 배로 늘립니다
        *pfds = realloc(*pfds, sizeof(**pfds) * (*fd_size));
    }

    (*pfds)[*fd_count].fd = newfd;
    (*pfds)[*fd_count].events = POLLIN; // 읽을 준비 상태를 확인합니다
    (*pfds)[*fd_count].revents = 0;

    (*fd_count)++;
}

/*
 * 집합의 지정한 인덱스에서 파일 설명자를 제거합니다.
 */
void del_from_pfds(struct pollfd pfds[], int i, int *fd_count)
{
    // 마지막 요소를 이 위치로 복사합니다
    pfds[i] = pfds[*fd_count-1];

```

```

    (*fd_count)--;
}

/*
 * 들어오는 연결을 처리합니다.
 */
void handle_new_connection(int listener, int *fd_count,
                           int *fd_size, struct pollfd **pfd)
{
    struct sockaddr_storage remoteaddr; // 클라이언트 주소
    socklen_t addrlen;
    int newfd; // 새로 accept()한 소켓 설명자
    char remoteIP[INET6_ADDRSTRLEN];

    addrlen = sizeof remoteaddr;
    newfd = accept(listener, (struct sockaddr *)&remoteaddr,
                   &addrlen);

    if (newfd == -1) {
        perror("accept");
    } else {
        add_to_pfds(pfd, newfd, fd_count, fd_size);

        printf("pollserver: new connection from %s on socket %d\n",
               inet_ntop2(&remoteaddr, remoteIP, sizeof remoteIP),
               newfd);
    }
}

/*
 * 일반 클라이언트 데이터와 연결 종료를 처리합니다.
 */
void handle_client_data(int listener, int *fd_count,
                        struct pollfd *pfd, int *pfd_i)
{
    char buf[256]; // 클라이언트 데이터용 버퍼

    int nbytes = recv(pfd[*pfd_i].fd, buf, sizeof buf, 0);

    int sender_fd = pfd[*pfd_i].fd;

    if (nbytes <= 0) { // 오류가 발생했거나 클라이언트가 연결을 닫았습니다
        if (nbytes == 0) {
            // 연결이 닫혔습니다
            printf("pollserver: socket %d hung up\n", sender_fd);
        } else {
            perror("recv");
        }

        close(pfd[*pfd_i].fd); // 잘 가!

        del_from_pfds(pfd, *pfd_i, fd_count);
    }
}

```

```

        // 방금 삭제한 슬롯을 다시 검사합니다
        (*pfd_i)--;

    } else { // 클라이언트에서 정상 데이터를 받았습니다
        printf("pollserver: recv from fd %d: %.*s", sender_fd,
               nbytes, buf);
        // 모두에게 보냅니다!
        for(int j = 0; j < *fd_count; j++) {
            int dest_fd = pfd[j].fd;

            // 리스너와 자기 자신은 제외합니다
            if (dest_fd != listener && dest_fd != sender_fd) {
                if (send(dest_fd, buf, nbytes, 0) == -1) {
                    perror("send");
                }
            }
        }
    }
}

/*
 * 기존 연결들을 모두 처리합니다.
 */
void process_connections(int listener, int *fd_count, int *fd_size,
                        struct pollfd **pfd)
{
    for(int i = 0; i < *fd_count; i++) {

        // 읽을 준비가 된 항목이 있는지 확인합니다
        if ((*pfd)[i].revents & (POLLIN | POLLHUP)) {
            // 하나 찾았습니다!!

            if ((*pfd)[i].fd == listener) {
                // 리스너라면 새 연결입니다
                handle_new_connection(listener, fd_count, fd_size,
                                      pfd);
            } else {
                // 그렇지 않으면 일반 클라이언트입니다
                handle_client_data(listener, fd_count, *pfd, &i);
            }
        }
    }
}

/*
 * 메인: 리스너와 연결 집합을 만들고 영원히 반복하면서
 * 연결을 처리합니다.
 */
int main(void)
{
    int listener;    // 리스닝 소켓 설명자

    // 5개 연결을 담을 공간으로 시작합니다

```

```

// (필요하면 realloc할 것입니다)
int fd_size = 5;
int fd_count = 0;
struct pollfd *pfd = malloc(sizeof *pfd * fd_size);

// 리스닝 소켓을 설정하고 얻습니다
listener = get_listener_socket();

if (listener == -1) {
    fprintf(stderr, "error getting listening socket\n");
    exit(1);
}

// 리스너를 집합에 추가합니다.
// 들어오는 연결을 읽을 준비 이벤트로 보고받습니다
pfd[0].fd = listener;
pfd[0].events = POLLIN;

fd_count = 1; // 리스너용

puts("pollserver: waiting for connections...");

// 메인 루프
for(;;) {
    int poll_count = poll(pfd, fd_count, -1);

    if (poll_count == -1) {
        perror("poll");
        exit(1);
    }

    // 연결들을 순회하며 읽을 데이터를 찾습니다
    process_connections(listener, &fd_count, &fd_size, &pfd);
}

free(pfd);
}

```

다음 절에서는 비슷하지만 오래된 함수인 `select()` 를 살펴볼 것입니다. `select()` 와 `poll()` 모두 비슷한 기능과 성능을 제공하고 쓰는 방식만 조금 다릅니다. `select()` 쪽이 조금 더 이식성이 좋을지도 모르나 사용하기에는 조금 더 어색할 것입니다. 여러분의 시스템에서 지원되지만 한다면 더 마음에 드는 쪽을 선택하세요.

7.3 `select()` — 동기화된 I/O 멀티플렉싱, 예전 방식

이 함수는 이상하지만 아주 유용합니다. 다음과 같은 상황을 생각해 보세요. 여러분은 서버이고 들어오는 연결을 감지함과 동시에 이미 가진 연결로부터 계속 데이터를 읽어들이고 싶습니다.

별 문제가 없다고 말씀하실지도 모릅니다. 그냥 `accept()` 와 몇 개의 `recv()` 를 쓰면 될 뿐입니다. 하지만 정말로 그럴까요? `accept()` 호출이 블록되었다면 어떻게 할까요? 어떻게 `recv()` 로 동시에 데이터를 받을 수 있을까요? “논블로킹 소켓을 써봅시다!” 역시 해결책이 아닙니다. CPU를 모조리 쓰고 싶지는 않을 것입니다. 그럼 어떻게 해야 할까요?

`select()` 가 여러 소켓을 동시에 관찰할 수 있는 능력을 줍니다. 그것이 어떤 것이 읽을 준비가 되었는지, 어떤 것이 쓸 준비가 되었는지, 그리고 정말로 관심이 있다면 어떤 것에 오류가 발생했는지까지 알려줄 것입니다.

경고 한마디: `select()` 가 이식성이 아주 좋지만 연결이 아주 많은 상황에서는 끔찍하게 느려집니다. 그런 상황에서는 여러분의 시스템에서 쓸 수 있는 가장 빠른 방법을 시도하는 `libevent`^a 같은 이벤트 라이브러리를 쓰면 더 나은 성능을 얻을 수 있습니다.

^a<https://libevent.org/>

잡담은 그만하고 `select()` 의 개요를 제시하겠습니다.

```
#include <sys/time.h>
#include <sys/types.h>
#include <unistd.h>

int select(int numfds, fd_set *readfds, fd_set *writefds,
           fd_set *exceptfds, struct timeval *timeout);
```

이 함수는 `readfds` 와 `writefds`, 그리고 `exceptfds` 라는 파일 설명자의 “집합들”을 관찰합니다. 만약 여러분이 표준 입력과 몇 개의 소켓 설명자로부터 읽어 들일 수 있는지 확인하고 싶다면 `readfds` 집합에 0과 `sockfd` 를 추가하세요. `numfds` 는 가장 큰 파일 설명자에 1을 더한 값으로 설정해야 합니다. 이 예제에서는 `sockfd+1` 이 될 것이며 이유는 당연히 그것이 표준 입력(0)보다 클 것이기 때문입니다.

`select()` 가 반환할 때 `readfds` 는 여러분이 선택한 파일 설명자 중에서 읽을 준비가 된 것들을 반영하도록 바뀌어 있을 것입니다. 여러분은 그것들을 아래에 나오는 `FD_ISSET()` 매크로로 검사할 수 있습니다.

더 진행하기 전에 이 집합들을 어떻게 조작하는지에 대해 이야기할 것입니다. 각 집합은 `fd_set` 형입니다. 이 자료형에 대해서 아래의 매크로들을 쓸 수 있습니다.

함수	설명
<code>FD_SET(int fd, fd_set *set);</code>	<code>fd</code> 를 <code>set</code> 에 더합니다.
<code>FD_CLR(int fd, fd_set *set);</code>	<code>fd</code> 를 <code>set</code> 에서 제거합니다.
<code>FD_ISSET(int fd, fd_set *set);</code>	<code>fd</code> 가 <code>set</code> 에 있다면 참을 반환합니다.
<code>FD_ZERO(fd_set *set);</code>	<code>set</code> 의 모든 요소를 제거합니다.

마지막으로 이 이상한 `struct timeval` 은 무엇일까요? 누군가 여러분에게 데이터를 보낼 때까지 무한히 기다리고 싶지 않을 때가 있습니다. 아마도 매 96초마다 실제로는 아무 일도 일어나지 않았어도 “진행 중...”이라고 출력하고 싶을 수도 있습니다. 이 시간 구조체가 제한 시간을 지정할 수 있도록 해 줍니다. 시간이 초과되고 `select()` 가 준비된 파일 설명자를 찾지 못할 경우, 그것은 반환하고 여러분은 처리를 계속할 수 있습니다.

`struct timeval` 은 아래와 같은 필드를 가지고 있습니다.

```
struct timeval {
    int tv_sec;    // 초
    int tv_usec;   // 마이크로초
};
```

단순히 `tv_sec` 을 기다리고 싶은 초로, `tv_usec` 을 기다리고 싶은 마이크로초로 설정하세요. 그렇습니다. *마이크로초*입니다. 밀리초가 아닙니다. 1밀리초는 1,000마이크로초입니다. 그리고 1,000밀리초는 1초입니다. 그러므로 1초는 1,000,000마이크로초입니다. 왜 "usec"일까요? "u"는 우리가 "마이크로"를 뜻하기 위해서 쓰는 그리스 문자 μ (뮤)와 닮았기 때문입니다. 또 함수가 반환할 때 `timeout` 은 남아있는 시간을 보여주기 위해서 업데이트 될 수도 있습니다. 이것은 여러분이 실행 중인 유닉스의 종류에 따라 다릅니다.

와! 마이크로초 해상도의 타이머를 쓸 수 있군요! 사실 별로 기대하지 않는 것이 좋습니다. 여러분이 `struct timeval` 을 아무리 작게 설정해도 여러분의 표준 유닉스 타임슬라이스 (역자 주: 커널이 프로세스 스케줄링의 최소 단위로 쓰는 시간)만큼은 기다려야 합니다.

다른 흥미로운 것들도 있습니다. `struct timeval` 을 0 으로 설정하면 `select()` 는 여러분의 집합에 있는 모든 파일 설명자를 폴링한 뒤 즉시 시간초과가 될 것입니다. 매개변수 `timeout` 을 `NULL` 로 설정하면 절대 시간초과가 되지 않으며 파일 설명자가 준비될 때까지 기다릴 것입니다. 마지막으로 만약 특정 집합을 기다릴 필요가 없다면 그 집합은 `select()` 를 호출할 때 `NULL`로 설정하면 됩니다.

아래의 코드 조각³은 표준 입력에 뭔가 나타날 때까지 2.5초를 기다립니다.

```
/*
** select.c -- select()의 예시
*/

#include <stdio.h>
#include <sys/time.h>
#include <sys/types.h>
#include <unistd.h>

#define STDIN 0 // 표준 입력의 파일 설명자

int main(void)
{
    struct timeval tv;
    fd_set readfds;

    tv.tv_sec = 2;
    tv.tv_usec = 500000;

    FD_ZERO(&readfds);
    FD_SET(STDIN, &readfds);

    // writefds와 exceptfds는 신경 쓰지 않습니다
    select(STDIN+1, &readfds, NULL, NULL, &tv);

    if (FD_ISSET(STDIN, &readfds))
        printf("키가 눌렸습니다!\n");
    else
        printf("시간이 초과되었습니다.\n");

    return 0;
}
```

만약 여러분이 줄 단위로 버퍼 처리되는 터미널을 사용한다면 엔터를 누르지 않으면 제한 시간이 초과될 것입니다.

이제 여러분 중 일부는 이것이 데이터그램 소켓의 데이터를 기다리는 아주 훌륭한 방법이라고 생각할 것입니다. 그리고 맞습니다. 맞을 수도 있습니다. 일부 유닉스에서는 `select`를 이 목적으로 쓸 수 있고, 일부에서는 그럴 수

³<https://beej.us/guide/bgnet/source/examples/select.c>

없습니다. 그 방식을 시도하고 싶다면 여러분의 로컬 매뉴얼 페이지 내용을 참고해야 합니다.

일부 유닉스는 제한 시간이 초과되기까지 남은 시간을 반영하기 위해 여러분의 `struct timeval` 를 업데이트합니다. 그러나 다른 것들은 그렇게 하지 않습니다. 만약 이식성 있는 코드를 작성하고자 한다면 그것에 의존해서는 안 됩니다. (경과한 시간을 알고 싶다면 `gettimeofday()` 을 사용하세요. 실망스럽겠지만 그것이 올바른 방법입니다.)

만약 읽기 집합에 있는 소켓이 연결을 닫는다면 어떤 일이 생길까요? 그 경우 `select()` 는 그 소켓 설명자를 “읽기 준비된 상태”로 설정할 것입니다. 실제로 그 소켓에 `recv()` 하면 `recv()` 는 0 을 반환할 것입니다. 그것이 클라이언트가 연결을 닫았음을 알아내는 방법입니다.

`select()` 에 관한 흥미로운 이야기가 하나 더 있습니다. 만약 `listen()` 작업 중인 소켓을 가지고 있을 경우, 그 소켓의 파일 설명자를 `readfds` 집합에 넣어서 새로운 연결이 있는지 알 수 있습니다.

지금까지 전능한 `select()` 함수에 대해 간단히 알아보았습니다.

그러나 대중적 요구가 있으므로 아래에 심도 있는 예제를 첨부합니다. 불행하게도 위의 아주 단순한 예제와 아래의 예제에는 상당한 차이가 있습니다. 그렇지만 한 번 살펴보고 뒤따르는 설명을 읽어보세요.

이 프로그램⁴은 단순한 다중 사용자 채팅 서버처럼 동작합니다. 하나의 창에서 이것을 실행한 후 다른 창에서 `telnet` 을 통해 접속하세요. (" `telnet hostname 9034` ") 하나의 `telnet` 세션에서 뭔가 입력하면 나머지 모두에서 그 내용이 나타나야 합니다.

```
/*
** selectserver.c -- 허술한 다중 사용자 대화 서버
*/

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <netdb.h>

#define PORT "9034" // 리스닝할 포트

/*
* 소켓 주소를 IP 주소 문자열로 변환합니다.
* addr: struct sockaddr_in 또는 struct sockaddr_in6
*/
const char *inet_ntop2(void *addr, char *buf, size_t size)
{
    struct sockaddr_storage *sas = addr;
    struct sockaddr_in *sa4;
    struct sockaddr_in6 *sa6;
    void *src;

    switch (sas->ss_family) {
        case AF_INET:
            sa4 = addr;

```

⁴<https://beej.us/guide/bgnet/source/examples/selectserver.c>

```

        src = &(sa4->sin_addr);
        break;
    case AF_INET6:
        sa6 = addr;
        src = &(sa6->sin6_addr);
        break;
    default:
        return NULL;
}

return inet_ntop(sas->ss_family, src, buf, size);
}

/*
 * 리스닝 소켓을 반환합니다.
 */
int get_listener_socket(void)
{
    struct addrinfo hints, *ai, *p;
    int yes=1;    // 아래의 setsockopt() SO_REUSEADDR용
    int rv;
    int listener;

    // 소켓을 얻어서 바인드합니다
    memset(&hints, 0, sizeof hints);
    hints.ai_family = AF_UNSPEC;
    hints.ai_socktype = SOCK_STREAM;
    hints.ai_flags = AI_PASSIVE;
    if ((rv = getaddrinfo(NULL, PORT, &hints, &ai)) != 0) {
        fprintf(stderr, "selectserver: %s\n", gai_strerror(rv));
        exit(1);
    }

    for(p = ai; p != NULL; p = p->ai_next) {
        listener = socket(p->ai_family, p->ai_socktype,
                          p->ai_protocol);
        if (listener < 0) {
            continue;
        }

        // 성가신 "address already in use" 오류 메시지를 피합니다
        setsockopt(listener, SOL_SOCKET, SO_REUSEADDR, &yes,
                   sizeof(int));

        if (bind(listener, p->ai_addr, p->ai_addrlen) < 0) {
            close(listener);
            continue;
        }

        break;
    }

    // 여기까지 왔다면 바인드하지 못했다는 뜻입니다

```

```

    if (p == NULL) {
        fprintf(stderr, "selectserver: failed to bind\n");
        exit(2);
    }

    freeaddrinfo(ai); // 이 구조체는 이제 필요 없습니다

    // 리스닝합니다
    if (listen(listener, 10) == -1) {
        perror("listen");
        exit(3);
    }

    return listener;
}

/*
 * 새로 들어온 연결을 적절한 집합에 추가합니다
 */
void handle_new_connection(int listener, fd_set *master, int *fdmax)
{
    socklen_t addrlen;
    int newfd;          // 새로 accept()한 소켓 설명자
    struct sockaddr_storage remoteaddr; // 클라이언트 주소
    char remoteIP[INET6_ADDRSTRLEN];

    addrlen = sizeof remoteaddr;
    newfd = accept(listener,
        (struct sockaddr *)&remoteaddr,
        &addrlen);

    if (newfd == -1) {
        perror("accept");
    } else {
        FD_SET(newfd, master); // 마스터 집합에 추가합니다
        if (newfd > *fdmax) { // 최댓값을 기록합니다
            *fdmax = newfd;
        }
        printf("selectserver: new connection from %s on "
            "socket %d\n",
            inet_ntop2(&remoteaddr, remoteIP, sizeof remoteIP),
            newfd);
    }
}

/*
 * 모든 클라이언트에 메시지를 방송합니다
 */
void broadcast(char *buf, int nbytes, int listener, int s,
    fd_set *master, int fdmax)
{
    for(int j = 0; j <= fdmax; j++) {
        // 모두에게 보냅니다!
    }
}

```

```

        if (FD_ISSET(j, master)) {
            // 리스너와 자기 자신은 제외합니다
            if (j != listener && j != s) {
                if (send(j, buf, nbytes, 0) == -1) {
                    perror("send");
                }
            }
        }
    }
}

/*
 * 클라이언트 데이터와 연결 종료를 처리합니다
 */
void handle_client_data(int s, int listener, fd_set *master,
                       int fdmax)
{
    char buf[256];    // 클라이언트 데이터용 버퍼
    int nbytes;

    // 클라이언트에서 온 데이터를 처리합니다
    if ((nbytes = recv(s, buf, sizeof buf, 0)) <= 0) {
        // 오류가 발생했거나 클라이언트가 연결을 닫았습니다
        if (nbytes == 0) {
            // 연결이 닫혔습니다
            printf("selectserver: socket %d hung up\n", s);
        } else {
            perror("recv");
        }
        close(s); // 잘 가!
        FD_CLR(s, master); // 마스터 집합에서 제거합니다
    } else {
        // 클라이언트에서 데이터를 받았습니다
        broadcast(buf, nbytes, listener, s, master, fdmax);
    }
}

/*
 * 메인
 */
int main(void)
{
    fd_set master;    // 마스터 파일 설명자 목록
    fd_set read_fds;  // select()용 임시 파일 설명자 목록
    int fdmax;        // 가장 큰 파일 설명자 번호

    int listener;     // 리스닝 소켓 설명자

    FD_ZERO(&master); // 마스터 집합과 임시 집합을 비웁니다
    FD_ZERO(&read_fds);

    listener = get_listener_socket();

```

```

// 리스너를 마스터 집합에 추가합니다
FD_SET(listener, &master);

// 가장 큰 파일 설명자를 기록합니다
fdmax = listener; // 지금까지는 이것입니다

// 메인 루프
for(;;) {
    read_fds = master; // 복사합니다
    if (select(fdmax+1, &read_fds, NULL, NULL, NULL) == -1) {
        perror("select");
        exit(4);
    }

    // 기존 연결을 순회하며 읽을 데이터를 찾습니다
    for(int i = 0; i <= fdmax; i++) {
        if (FD_ISSET(i, &read_fds)) { // 하나 찾았습니다!!
            if (i == listener)
                handle_new_connection(i, &master, &fdmax);
            else
                handle_client_data(i, listener, &master, fdmax);
        }
    }
}

return 0;
}

```

코드에 `master` 와 `read_fds` 두 개의 파일 설명자 집합이 있음에 주목하세요. 전자인 `master` 는 새 연결을 리스닝하는 소켓 설명자와 현재 연결된 모든 소켓의 설명자를 가집니다.

`master` 를 가지는 이유는 `select()` 가 사실 여러분이 넘기는 집합을 변형해서 읽기 준비된 소켓을 반영하기 때문입니다. 우리는 하나의 `select()` 호출과 다음 호출 사이에서 연결들을 계속 기억해야 하므로 이것들은 다른 곳에 안전하게 보관해야 하는 것입니다. 그래서 우리는 실제로 쓰기 전에 `master` 를 `read_fds` 에 복사하고 `select()` 를 호출하는 것입니다.

하지만 그것은 우리가 새로운 연결을 받을 때마다 그것을 `master` 집합에 추가해야 한다는 의미일까요? 맞습니다! 그리고 연결이 닫힐 때마다 `master` 집합에서 제거해야 할까요? 그렇습니다. 그렇게 해야 합니다.

`listener` 소켓이 읽을 준비가 되었는지 확인한다는 사실에 주목하세요. 준비가 되어 있다면 대기 중인 새로운 연결이 있다는 의미이고, `accept()` 한 후에 `master` 집합에 추가합니다. 비슷하게 클라이언트 연결이 읽을 준비가 되고 `recv()` 가 0을 반환한다면 클라이언트가 연결을 닫았다는 사실을 알 수 있고, 우리는 그 연결을 `master` 집합에서 제거해야 합니다.

클라이언트에 대한 `recv()` 가 0이 아닌 값을 반환한다면 우리는 어떤 데이터가 도착했다는 것을 알 수 있습니다. 그러므로 데이터를 받은 후에 `master` 목록을 순회하면서 모든 나머지 연결된 클라이언트들에게 그 데이터를 보냅니다.

지금까지 전능한 `select()` 함수에 대한 그다지 단순하지 않은 개관이었습니다.

모든 리눅스 팬들을 위한 짧은 이야기가 있습니다. 드문 몇몇 상황에서 때때로 리눅스의 `select()` 는 실제로는 읽을 준비가 되어 있지 않음에도 “읽을 준비가 되었다”고 하면서 반환합니다. 이것은 `select()` 가 읽기 동작에

대해 대기하지 않을 것이라고 말함에도 read() 가 블록될 것임을 의미합니다. 이런! 아무튼 해결책은 읽을 소켓에 O_NONBLOCK 플래그를 설정해서 EWOULDBLOCK 오류가 발생하도록 하는 것입니다. (이 오류가 생겨도 무시해도 됩니다.) 소켓을 논블로킹 모드로 설정하는 방법에 대해서는 fcntl() 참조 페이지를 참고하세요.

추가로 약간의 여담을 하자면 select() 와 상당히 비슷한 일을 하지만 파일 설명자 집합을 다른 방식으로 처리하는 poll() 이라는 다른 함수가 있습니다. 한 번 확인해 보세요!

7.4 부분적인 send() 처리하기

위에서 다른 send() 에 대한 절에서 send() 가 여러분이 전송 요청한 바이트들을 모두 보내지는 않을 수도 있다고 말한 것을 기억하시나요? 여러분은 512바이트를 보내길 원해도 복귀값은 412일 수 있다는 의미입니다. 남은 100바이트에는 무슨 일이 생긴 것일까요?

음, 그것들은 여전히 여러분의 작은 버퍼에 남아서 보내지길 기다리고 있습니다. 여러분의 통제 밖에 있는 상황 때문에 커널이 데이터를 한 덩어리로 전부 보내지는 않기로 결정한 것입니다. 그리고 이제 남은 데이터를 보내는 일은 여러분에게 달려 있습니다.

그 일을 하는 함수를 만드는 한 가지 방법은 이렇습니다.

```
#include <sys/types.h>
#include <sys/socket.h>

int sendall(int s, char *buf, int *len)
{
    int total = 0;          // 몇 바이트를 보냈는가
    int bytesleft = *len;   // 보내야 하는 데이터는 얼마나 남아 있는가
    int n;

    while(total < *len) {
        n = send(s, buf+total, bytesleft, 0);
        if (n == -1) { break; }
        total += n;
        bytesleft -= n;
    }

    *len = total; // 실제로 보낸 바이트 수를 기록해 반환합니다

    return n == -1 ? -1 : 0; // 실패 시에는 -1을, 성공 시에는 0을 반환합니다
}
```

이 예제에서 s 는 여러분이 데이터를 보내고 싶은 소켓이고 buf 는 데이터를 담은 버퍼입니다. len 은 버퍼에 담긴 바이트의 개수를 담은 int 에 대한 포인터입니다.

함수는 오류가 발생하면 -1 을 반환합니다(errno 는 send() 에 대한 호출로 인해 여전히 설정되어 있습니다). 또한 실제로 전송된 바이트의 개수가 len 을 통해 반환됩니다. 이 값은 오류가 발생하지 않는 이상 여러분이 전송하라고 요청한 바이트의 수와 같습니다. sendall() 은 데이터를 전송하기 위해서 최선을 다하지만 오류가 발생하면 바로 여러분에게 알려줄 것입니다.

완성도를 위해 이 함수에 대한 호출 예제를 하나 더 보겠습니다.

```
char buf[10] = "Beej!";
int len;

len = strlen(buf);
if (sendall(s, buf, &len) == -1) {
    perror("sendall");
    printf("오류가 발생해서 %d바이트만 보냈습니다!\n", len);
}
```

수신자 측에 패킷의 일부가 도착하면 무슨 일이 벌어질까요? 만약 패킷이 가변 길이일 경우 수신자는 어떻게 하나의 패킷이 끝나고 다른 하나가 시작되는 것을 알까요? 그렇습니다. 현실 세계의 시나리오는 정말 골치 아픕니다. 여러분은 아마도 **캡슐화**를 해야 할 것입니다. (시작부에 있던 데이터 캡슐화 절을 기억하시나요?) 자세한 내용을 알고 싶다면 계속 읽어보세요.

7.5 직렬화—데이터를 포장하는 방법

네트워크를 통해 문자열 데이터를 보내는 것은 꽤 쉽다는 것을 이제 알 것입니다. 하지만 만약 `int` 나 `float` 같은 “이진” 데이터를 전송하려고 하면 어떻게 해야 할까요? 몇 가지 방법이 있습니다.

1. `sprintf()` 같은 함수를 써서 수를 텍스트로 변환하고 텍스트를 전송합니다. 수신자는 `strtol()` 같은 함수를 써서 텍스트를 다시 숫자로 변환합니다.
2. `send()` 에 데이터를 가리키는 포인터를 전달해서 원시 데이터를 그대로 전송합니다.
3. 데이터를 호환성 있는 바이너리 형태로 인코딩합니다. 수신자는 디코딩합니다.

오늘 밤 한정 미리보기!

[커튼이 올라간다]

Beej가 말합니다: “저는 위의 세 번째 방법을 좋아합니다.”

[끝]

(이 절을 본격적으로 시작하기에 앞서, 이 일을 하기 위한 라이브러리들이 이미 있다는 말을 미리 해 두겠습니다. 이식성 있고 오류가 없는 여러분만의 라이브러리를 만드는 작업은 꽤 어려운 일이 될 것입니다. 그러므로 직접 구현하기로 결정하기 전에 충분히 찾아보고 조사하는 것이 좋습니다. 저는 그런 것이 어떻게 동작하는지 궁금할 독자들을 위해서 관련된 내용을 여기에 담을 뿐입니다.)

사실 위에 언급한 모든 방법에 각각의 장점과 단점이 있습니다. 그러나 위에 말한 대로, 저는 일반적으로 세 번째 방법을 선호합니다. 그러나 먼저 다른 두 가지 방법의 장단점에 대해서 조금 더 알아보시다.

수를 보내기 전에 텍스트로 인코딩하는 첫 번째 방법은 랜선을 타고 오는 정보를 출력하고 읽어 보기가 쉽다는 장점이 있습니다. Internet Relay Chat (IRC)⁵ 처럼 인간이 읽을 수 있는 프로토콜은 때때로 전송 대역폭에 민감하지 않은 상황에서 사용하기에 아주 훌륭합니다. 그러나 변환이 느리다는 단점이 있고, 결과는 거의 언제나 원본 수보다 더 많은 공간을 차지한다는 단점이 있습니다.

두 번째 방법: 원시 데이터를 전송하기. 이것은 꽤 간단(하고 위험)합니다. 보낼 데이터에 대한 포인터를 얻은 후 그것으로 `send()` 를 호출합니다.

```
double d = 3490.15926535;

send(s, &d, sizeof d, 0); /* 위험--이식성 없음! */
```

수신자는 이것을 아래와 같이 받습니다.

⁵https://en.wikipedia.org/wiki/Internet_Relay_Chat

```
double d;

recv(s, &d, sizeof d, 0); /* 위험--이식성 없음! */
```

빠르고, 간단합니다. 문제 될 것이 없어 보이지요? 사실, 모든 아키텍처들이 `double` (이나 다른 예로는 `int`)을 동일한 비트 표현이나 심지어 동일한 바이트 순서로 표시하는 것은 아니라는 문제가 있습니다! 위의 코드는 절대로 이식성이 없습니다. (잠깐—이식성이 필요 없는 상황도 있지 않을까요? 그 경우에 이 방식은 좋고 빠른 방법이 됩니다.)

정수 자료형을 포장할 때 `htons()` 계열 함수가 수를 네트워크 바이트 순서로 변환해서 이식성을 지키는 데 어떻게 도움을 주는지, 그리고 그것이 왜 필요한지 이미 살펴보았습니다. 불행하게도 `float` 자료형에 대해서는 유사한 함수가 없습니다. 희망이 없는 것일까요?

두려워하지 마세요!(잠시 무섭지 않았나요? 정말로 조금도 무섭지 않았다고요?) 우리에게 방법이 있습니다. 우리는 데이터를 원격지에서 풀어낼 수 있는 알려진 방식으로 포장(pack)(또는 “marshal”, “직렬화”, 그것도 아니면 그런 일에 대한 10억 개의 다른 이름)할 수 있습니다.

“알려진 이진 형식”은 무엇일까요? 우리는 이미 `htons()`의 예제를 보았습니다. 그것은 수를 호스트의 형식이 무엇이든 간에 네트워크 바이트 순서로 변환(또는 “인코드”, 이것이 더 이해하기 쉽다면)합니다. 수를 원래대로 돌려놓기(디코드) 위해서 수신자는 `ntohs()`를 호출해야 합니다.

하지만 제가 조금 전에 비-정수 타입에 대해서는 그런 함수가 없다고 했지요? 그렇습니다. 그리고 C 언어에서는 이것을 처리하는 표준 방법이 없기 때문에 조금 까다로운 일입니다. 하지만 파이썬에서는 누워서 피클 먹기입니다. (역자 주: 파이썬에는 직렬화와 역직렬화를 처리하는 `pickle` 모듈이 있습니다.)

필요한 작업은 데이터를 알려진 형식으로 포장하고 네트워크에 실어 보내는 것입니다. 예를 들어서 `float`를 포장하는 작업을 위한 간단하고 지저분하고 개선할 점이 많은 예제 코드⁶가 있습니다.

```
#include <stdint.h>

uint32_t htonf(float f)
{
    uint32_t p;
    uint32_t sign;

    if (f < 0) { sign = 1; f = -f; }
    else { sign = 0; }

    p = (((uint32_t)f)&0x7fff)<<16 | (sign<<31); // 정수 부분과 부호
    p |= (uint32_t)((f - (int)f) * 65536.0f)&0xffff; // 소수 부분

    return p;
}

float ntohf(uint32_t p)
{
    float f = ((p>>16)&0x7fff); // 정수 부분
    f += (p&0xffff) / 65536.0f; // 소수 부분

    if (((p>>31)&0x1) == 0x1) { f = -f; } // 부호 비트 설정

    return f;
}
```

⁶<https://beej.us/guide/bgnet/source/examples/pack.c>

```
}

```

위의 코드는 `float` 를 32비트 수에 저장하기 위한 단순한 구현입니다. 최상위 비트(31)가 수의 부호를 저장하기 위해 쓰입니다. ("1"이 음수를 의미합니다.) 다음 15비트(30-16)(역자 주: 원문에서는 7비트라고 적혀 있으나 코드의 내용상 오타로 보임)가 `float` 의 전체 수 부분을 저장하기 위해서 쓰입니다. 마지막으로 남은 비트들(15-0)이 수의 소수 부분을 기록하기 위해서 쓰입니다.

사용법은 꽤 직관적입니다.

```
#include <stdio.h>

int main(void)
{
    float f = 3.1415926, f2;
    uint32_t netf;

    netf = htonl(f); // "네트워크" 형식으로 변환
    f2 = ntohf(netf); // 시험을 위해 원래대로 변환

    printf("Original: %f\n", f);          // 3.141593
    printf(" Network: 0x%08X\n", netf); // 0x0003243F
    printf("Unpacked: %f\n", f2);        // 3.141586

    return 0;
}
```

장점을 보자면, 이 코드는 작고 간단하며 빠릅니다. 단점을 보자면 이 방식은 공간을 효율적으로 쓰지 않으며 표현 범위가 상당히 제한되어 있습니다. 32767보다 큰 수를 저장하려고 하면 이 방법은 제대로 동작하지 않을 것입니다! 또한 여러분은 위의 예제에서 소수점의 마지막 2자리가 제대로 보존되지 않은 것을 볼 수 있습니다.

이것을 해결하려면 어떻게 해야 할까요? 사실 부동 소수점 수를 저장하기 위한 표준은 IEEE-754⁷로 알려져 있습니다. 대부분의 컴퓨터는 부동 소수점 계산을 위해서 내부적으로 이 형식을 사용합니다. 그러므로 그런 경우라면 엄밀히 말하자면 변환을 수행할 필요는 없습니다. 그러나 여러분의 소스 코드가 이식성이 있기를 바란다면 반드시 그런 가정을 할 수는 없습니다.

정말 그럴까요? 여러분의 시스템은 정수에 대해 아마 2의 보수 표현을 쓰는 것처럼, 부동 소수점에 대해서도 매우 높은 확률로 IEEE-754를 쓸 것입니다. 그래서 그 사실을 알고 있다면 데이터를 그대로 네트워크로 넘길 수 있습니다. 다만 `htonl()` 이나 알맞은 함수로 엔디언은 맞춰야 합니다. `float` 에도 엔디언이 있습니다. 변환이 필요 없는 빅 엔디언 시스템에서 `htons()` 와 그 계열 함수들이 하는 일이 바로 이것입니다.

하지만 혹시 IEEE-754가 아닌 시스템을 쓰고 있을 경우를 대비해서, 여기 `float` 와 `double` 을 IEEE-754 형식으로 인코딩하는 코드가 있습니다⁸. (엄밀히는 거의 대부분을 인코딩합니다. 이 코드는 NaN이나 Infinity를 처리하지 않습니다. 그러나 그런 처리가 가능하게 수정할 수도 있습니다.)

```
#define pack754_32(f) (pack754((f), 32, 8))
#define pack754_64(f) (pack754((f), 64, 11))
#define unpack754_32(i) (unpack754((i), 32, 8))
#define unpack754_64(i) (unpack754((i), 64, 11))

uint64_t pack754(long double f, unsigned bits, unsigned expbits)
```

⁷https://en.wikipedia.org/wiki/IEEE_754

⁸<https://beej.us/guide/bgnet/source/examples/ieee754.c>

```

{
    long double fnorm;
    int shift;
    long long sign, exp, significand;
    unsigned significandbits = bits - expbits - 1; // 부호 비트를 위해 1을 뺍니다

    if (f == 0.0) return 0; // 특별한 경우의 처리

    // 부호를 확인하고 정규화를 시작합니다
    if (f < 0) { sign = 1; fnorm = -f; }
    else { sign = 0; fnorm = f; }

    // 정규화된 형태의 f를 얻어내고 지수를 추적합니다
    shift = 0;
    while(fnorm >= 2.0) { fnorm /= 2.0; shift++; }
    while(fnorm < 1.0) { fnorm *= 2.0; shift--; }
    fnorm = fnorm - 1.0;

    // 실수부의 부동 소수점이 아닌 이진 표현을 구합니다
    significand = fnorm * ((1LL<<significandbits) + 0.5f);

    // 바이어스를 더한 지수부를 구합니다
    // (역자 주: IEEE754는 실제 지수에 일정한 바이어스를 더한 값을 부호 없는
    //   정수처럼
    // 저장합니다. 이렇게 하면 음수 지수를 따로 부호화하지 않아도 되고, 지수 값의
    //   크기
    // 비교도 단순해집니다.)
    exp = shift + ((1<<(expbits-1)) - 1); // shift + bias

    // 최종 값을 반환합니다
    return (sign<<(bits-1)) | (exp<<(bits-expbits-1)) | significand;
}

long double unpack754(uint64_t i, unsigned bits, unsigned expbits)
{
    long double result;
    long long shift;
    unsigned bias;
    unsigned significandbits = bits - expbits - 1; // 부호 비트를 위해 1을 뺍니다

    if (i == 0) return 0.0;

    // 유효숫자부를 뺍아냅니다
    result = (i & ((1LL<<significandbits)-1)); // 마스크 처리
    result /= (1LL<<significandbits); // 부동 소수점으로 변환
    result += 1.0f; // 1을 다시 더합니다

    // 지수부를 처리합니다
    bias = (1<<(expbits-1)) - 1;
    shift = ((i >> significandbits) & ((1LL<<expbits)-1)) - bias;
    while(shift > 0) { result *= 2.0; shift--; }
    while(shift < 0) { result /= 2.0; shift++; }
}

```

```
// 부호 처리
result *= (i >> (bits-1)) & 1? -1.0: 1.0;

return result;
}
```

32비트(아마도 `float`)와 64비트(아마도 `double`) 수를 위한 패킹과 언패킹 매크로를 위에 넣어두었습니다. 그러나 `bits` 크기의 데이터를 인코드 하기 위해서 `pack754()` 함수를 직접 호출할 수도 있을 것입니다. (`expbits` 만큼의 지수부가 정규화된 수의 지수로 보존될 것입니다.)

여기 사용 예시가 있습니다.

```
#include <stdio.h>
#include <stdint.h> // uintN_t 형들을 정의합니다
#include <inttypes.h> // PRIx 매크로들을 정의합니다

int main(void)
{
    float f = 3.1415926, f2;
    double d = 3.14159265358979323, d2;
    uint32_t fi;
    uint64_t di;

    fi = pack754_32(f);
    f2 = unpack754_32(fi);

    di = pack754_64(d);
    d2 = unpack754_64(di);

    printf("float before : %.7f\n", f);
    printf("float encoded: 0x%08" PRIx32 "\n", fi);
    printf("float after  : %.7f\n\n", f2);

    printf("double before : %.20lf\n", d);
    printf("double encoded: 0x%016" PRIx64 "\n", di);
    printf("double after  : %.20lf\n", d2);
}
```

위의 코드는 아래의 출력을 생성합니다.

```
float before : 3.1415925
float encoded: 0x40490FDA
float after  : 3.1415925

double before : 3.14159265358979311600
double encoded: 0x400921FB54442D18
double after  : 3.14159265358979311600
```

여러분이 하실 만한 또 다른 질문은 `struct` 를 어떻게 포장하느냐입니다. 불행히도 컴파일러는 `struct` 의 모든 곳에 자유롭게 패딩을 넣을 수 있습니다. 그리고 그것은 구조체 전체를 한 번에 네트워크에 이식성 있게 전송할

수는 없다는 것을 의미합니다. (“이건 되고”, “이건 안되고”를 듣는 것이 지겨운가요? 죄송합니다. 제 친구의 말을 빌리자면 “뭔가 잘못되면 그건 전부 마이크로소프트 때문입니다.” 이 경우에는 아마도 마이크로소프트의 잘못만은 아닐 것입니다. 그러나 제 친구의 말은 완전히 옳습니다.)

다시 주제로 돌아가서, `struct` 를 전송하는 가장 좋은 방법은 각각의 필드를 독립적으로 포장한 다음 반대편에 도착하면 다시 `struct` 안에 풀어 넣는 것입니다.

여러분은 이것이 굉장히 큰 작업일 것이라 예상할 것입니다. 맞습니다. 여러분이 할 일은 데이터를 포장하는 일을 도와줄 도우미 함수를 작성하는 것입니다. 재미있을 것입니다! 정말로!!

Kernighan과 Pike가 지은 *The Practice of Programming*⁹ 에서 그들은 바로 그 일을 하도록 `printf()` 와 유사한 `pack()` 과 `unpack()` 함수를 작성했습니다. 그것에 대한 링크를 제공하고 싶지만 그 함수들과 책의 다른 소스 코드는 온라인으로 제공되지 않고 있습니다.

(*The Practice of Programming*은 아주 좋은 책입니다. 제가 그 책을 추천할 때마다 제우스가 고양이를 한 마리씩 구합니다.) (역자 주: 신이 보답을 할 만큼 아주 좋은 선행이라는 뜻입니다.)

이 시점에서 저는 프로토콜 버퍼의 C 구현체¹⁰ 에 대한 링크를 제공하려 합니다. 저는 이것을 써 본 적이 없으나 훌륭한 코드로 보입니다. 파이썬과 펄 프로그래머들은 같은 일을 하기 위해서 그들의 언어가 가진 `pack()` 과 `unpack()` 함수를 확인해 보길 바랍니다. 자바는 유사한 방식으로 사용할 수 있는 `Serializable` 인터페이스를 가지고 있습니다.

그러나 만약 여러분이 자신만의 패킹 유틸리티를 C 언어로 작성하고 싶다면, K&P의 해결책은 패킷을 만들기 위해서 가변 길이 매개변수 목록을 활용하는 `printf()` 와 유사한 함수를 만드는 것입니다. 여기 제가 직접 만든 버전이 있으며¹¹ 여러분이 그런 것이 어떻게 동작하는지 알기에 충분할 것입니다.

(이 코드는 위의 `pack754()` 함수를 참조합니다. `packi*()` 함수는 정수가 아닌 `char` 배열에 수를 담는다는 점을 제외하면 `htons()` 계열 함수와 유사하게 동작합니다.)

```
#include <stdio.h>
#include <ctype.h>
#include <stdarg.h>
#include <string.h>

/*
** packi16() -- 16비트 정수를 char 버퍼에 저장합니다. (htons()처럼)
*/
void packi16(unsigned char *buf, unsigned int i)
{
    *buf++ = i>>8; *buf++ = i;
}

/*
** packi32() -- 32비트 정수를 char 버퍼에 저장합니다. (htonl()처럼)
*/
void packi32(unsigned char *buf, unsigned long int i)
{
    *buf++ = i>>24; *buf++ = i>>16;
    *buf++ = i>>8; *buf++ = i;
}
```

⁹<https://beej.us/guide/url/tpop>

¹⁰<https://github.com/protobuf-c/protobuf-c>

¹¹<https://beej.us/guide/bgnet/source/examples/pack2.c>

```

/*
** packi64() -- 64비트 정수를 char 버퍼에 저장합니다. (htonl()처럼)
*/
void packi64(unsigned char *buf, unsigned long long int i)
{
    *buf++ = i>>56; *buf++ = i>>48;
    *buf++ = i>>40; *buf++ = i>>32;
    *buf++ = i>>24; *buf++ = i>>16;
    *buf++ = i>>8;  *buf++ = i;
}

/*
** unpacki16() -- 16비트 정수를 char 버퍼에서 풀어냅니다. (ntohs()처럼)
*/
int unpacki16(unsigned char *buf)
{
    unsigned int i2 = ((unsigned int)buf[0]<<8) | buf[1];
    int i;

    // 부호 없는 수를 부호 있는 수로 바꿉니다
    if (i2 <= 0x7ffffu) { i = i2; }
    else { i = -1 - (unsigned int)(0xffffu - i2); }

    return i;
}

/*
** unpacku16() -- 16비트 부호 없는 정수를 char 버퍼에서 풀어냅니다. (ntohs()처럼)
*/
unsigned int unpacku16(unsigned char *buf)
{
    return ((unsigned int)buf[0]<<8) | buf[1];
}

/*
** unpacki32() -- 32비트 정수를 char 버퍼에서 풀어냅니다. (ntohl()처럼)
*/
long int unpacki32(unsigned char *buf)
{
    unsigned long int i2 = ((unsigned long int)buf[0]<<24) |
                           ((unsigned long int)buf[1]<<16) |
                           ((unsigned long int)buf[2]<<8) |
                           buf[3];

    long int i;

    // 부호 없는 수를 부호 있는 수로 바꿉니다
    if (i2 <= 0xffffffffu) { i = i2; }
    else { i = -1 - (long int)(0xffffffffu - i2); }

    return i;
}

/*

```

```

** unpacku32() -- 32비트 부호 없는 정수를 char 버퍼에서 풀어냅니다. (ntohl()처럼)
*/
unsigned long int unpacku32(unsigned char *buf)
{
    return ((unsigned long int)buf[0]<<24) |
           ((unsigned long int)buf[1]<<16) |
           ((unsigned long int)buf[2]<<8) |
           buf[3];
}

/*
** unpacki64() -- 64비트 정수를 char 버퍼에서 풀어냅니다. (ntohl()처럼)
*/
long long int unpacki64(unsigned char *buf)
{
    unsigned long long int i2 = ((unsigned long long int)buf[0]<<56) |
                                ((unsigned long long int)buf[1]<<48) |
                                ((unsigned long long int)buf[2]<<40) |
                                ((unsigned long long int)buf[3]<<32) |
                                ((unsigned long long int)buf[4]<<24) |
                                ((unsigned long long int)buf[5]<<16) |
                                ((unsigned long long int)buf[6]<<8) |
                                buf[7];

    long long int i;

    // 부호 없는 수를 부호 있는 수로 바꿉니다
    if (i2 <= 0x7fffffffffffffffu) { i = i2; }
    else { i = -1 -(long long int)(0xffffffffffffffffu - i2); }

    return i;
}

/*
** unpacku64() -- 64비트 부호 없는 정수를 char 버퍼에서 풀어냅니다. (ntohl()처럼)
*/
unsigned long long int unpacku64(unsigned char *buf)
{
    return ((unsigned long long int)buf[0]<<56) |
           ((unsigned long long int)buf[1]<<48) |
           ((unsigned long long int)buf[2]<<40) |
           ((unsigned long long int)buf[3]<<32) |
           ((unsigned long long int)buf[4]<<24) |
           ((unsigned long long int)buf[5]<<16) |
           ((unsigned long long int)buf[6]<<8) |
           buf[7];
}

/*
** pack() -- 형식 문자열이 지시한 방식으로 버퍼에 데이터를 저장합니다
**
** bits |signed  unsigned  float  string
** -----+-----
**      8 |   c          C

```

```

**      16 |   h       H       f
**      32 |   l       L       d
**      64 |   q       Q       g
**      - |
**
** (16비트 부호 없는 길이가 자동으로 문자열의 앞에 붙습니다.)
**
unsigned int pack(unsigned char *buf, char *format, ...)
{
    va_list ap;

    signed char c;           // 8비트
    unsigned char C;

    int h;                   // 16비트
    unsigned int H;

    long int l;              // 32비트
    unsigned long int L;

    long long int q;         // 64비트
    unsigned long long int Q;

    float f;                 // 부동 소수점
    double d;
    long double g;
    unsigned long long int fhold;

    char *s;                 // 문자열
    unsigned int len;

    unsigned int size = 0;

    va_start(ap, format);

    for(; *format != '\0'; format++) {
        switch(*format) {
            case 'c': // 8비트
                size += 1;
                c = (signed char)va_arg(ap, int); // 자료형 승급
                *buf++ = c;
                break;

            case 'C': // 부호 없는 8비트
                size += 1;
                C = (unsigned char)va_arg(ap, unsigned int); // 자료형 승급
                *buf++ = C;
                break;

            case 'h': // 16비트
                size += 2;
                h = va_arg(ap, int);

```

```
    packi16(buf, h);
    buf += 2;
    break;

case 'H': // 부호 없는 16비트
    size += 2;
    H = va_arg(ap, unsigned int);
    packi16(buf, H);
    buf += 2;
    break;

case 'l': // 32비트
    size += 4;
    l = va_arg(ap, long int);
    packi32(buf, l);
    buf += 4;
    break;

case 'L': // 부호 없는 32비트
    size += 4;
    L = va_arg(ap, unsigned long int);
    packi32(buf, L);
    buf += 4;
    break;

case 'q': // 64비트
    size += 8;
    q = va_arg(ap, long long int);
    packi64(buf, q);
    buf += 8;
    break;

case 'Q': // 부호 없는 64비트
    size += 8;
    Q = va_arg(ap, unsigned long long int);
    packi64(buf, Q);
    buf += 8;
    break;

case 'f': // 부동 소수점 16비트
    size += 2;
    f = (float)va_arg(ap, double); // 자료형 승급
    fhold = pack754_16(f); // IEEE 754로 변환
    packi16(buf, fhold);
    buf += 2;
    break;

case 'd': // 부동 소수점 32비트
    size += 4;
    d = va_arg(ap, double);
    fhold = pack754_32(d); // IEEE 754로 변환
    packi32(buf, fhold);
    buf += 4;
```

```

        break;

    case 'g': // 부동 소수점 64비트
        size += 8;
        g = va_arg(ap, long double);
        fhold = pack754_64(g); // IEEE 754로 변환
        packi64(buf, fhold);
        buf += 8;
        break;

    case 's': // 문자열
        s = va_arg(ap, char*);
        len = strlen(s);
        size += len + 2;
        packi16(buf, len);
        buf += 2;
        memcpy(buf, s, len);
        buf += len;
        break;
    }
}

va_end(ap);

return size;
}

/*
** unpack() -- 형식 문자열이 지정하는 대로 버퍼에 데이터를 풀어놓습니다
**
**      bits | signed   unsigned   float   string
**      -----
**      8  |  c       C
**      16 |  h       H       f
**      32 |  l       L       d
**      64 |  q       Q       g
**      -  |
**
** (문자열은 저장된 길이에 근거해서 추출됩니다. 단, `s` 앞에 최대 길이를 지정할
  ⇨ 수 있습니다.)
*/
void unpack(unsigned char *buf, char *format, ...)
{
    va_list ap;

    signed char *c;           // 8비트
    unsigned char *C;

    int *h;                   // 16비트
    unsigned int *H;

    long int *l;              // 32비트
    unsigned long int *L;

```

```

long long int *q;           // 64비트
unsigned long long int *Q;

float *f;                   // 부동 소수점
double *d;
long double *g;
unsigned long long int fhold;

char *s;
unsigned int len, maxstrlen=0, count;

va_start(ap, format);

for(; *format != '\0'; format++) {
    switch(*format) {
        case 'c': // 8비트
            c = va_arg(ap, signed char*);
            if (*buf <= 0x7f) { *c = *buf;} // 부호를 다시 붙입니다
            else { *c = -1 - (unsigned char)(0xffu - *buf); }
            buf++;
            break;

        case 'C': // 부호 없는 8비트
            C = va_arg(ap, unsigned char*);
            *C = *buf++;
            break;

        case 'h': // 16비트
            h = va_arg(ap, int*);
            *h = unpacki16(buf);
            buf += 2;
            break;

        case 'H': // 부호 없는 16비트
            H = va_arg(ap, unsigned int*);
            *H = unpacku16(buf);
            buf += 2;
            break;

        case 'l': // 32비트
            l = va_arg(ap, long int*);
            *l = unpacki32(buf);
            buf += 4;
            break;

        case 'L': // 부호 없는 32비트
            L = va_arg(ap, unsigned long int*);
            *L = unpacku32(buf);
            buf += 4;
            break;

        case 'q': // 64비트

```

```

        q = va_arg(ap, long long int*);
        *q = unpacki64(buf);
        buf += 8;
        break;

    case 'Q': // 부호 없는 64비트
        Q = va_arg(ap, unsigned long long int*);
        *Q = unpacku64(buf);
        buf += 8;
        break;

    case 'f': // 부동 소수점
        f = va_arg(ap, float*);
        fhold = unpacku16(buf);
        *f = unpack754_16(fhold);
        buf += 2;
        break;

    case 'd': // 32비트 부동 소수점
        d = va_arg(ap, double*);
        fhold = unpacku32(buf);
        *d = unpack754_32(fhold);
        buf += 4;
        break;

    case 'g': // 64비트 부동 소수점
        g = va_arg(ap, long double*);
        fhold = unpacku64(buf);
        *g = unpack754_64(fhold);
        buf += 8;
        break;

    case 's': // 문자열
        s = va_arg(ap, char*);
        len = unpacku16(buf);
        buf += 2;
        if (maxstrlen > 0 && len > maxlen) count = maxlen - 1;
        else count = len;
        memcpy(s, buf, count);
        s[count] = '\0';
        buf += len;
        break;

    default:
        if (isdigit(*format)) { // 최대 문자열 길이를 기록합니다
            maxlen = maxlen * 10 + (*format - '0');
        }
    }

    if (!isdigit(*format)) maxlen = 0;
}

va_end(ap);

```

```
}

```

위 코드의 시연 프로그램¹²입니다. 이 프로그램은 `buf`에 데이터를 조금 포장한 후 다시 변수에 풀어놓습니다. `unpack()`을 문자열 매개변수로 호출하는 경우(형식 지정자 "`s`")에는 버퍼 오버런을 막기 위해 "`96s`"처럼 최대 길이를 앞에 붙이는 것이 좋습니다. 네트워크를 통해 받은 데이터를 풀어놓을 때에는 주의해야 합니다. 악의적인 사용자가 여러분의 시스템을 공격하려고 잘못 구성된 패킷을 보낼 수 있습니다!

```
#include <stdio.h>
#include <stdint.h>
#include <inttypes.h>

// C23 컴파일러가 있다면
#if __STDC_VERSION__ >= 202311L
#include <stdfloat.h>
#else
// 아니면 직접 정의합니다.
// 아키텍처에 따라 다릅니다! 하지만 아마 다음과 같을 것입니다.
typedef float float32_t;
typedef double float64_t;
#endif

int main(void)
{
    uint8_t buf[1024];
    int8_t magic;
    int16_t monkeycount;
    int32_t altitude;
    float32_t absurdityfactor;
    char *s = "Great unmitigated Zot! You've found the Runestaff!";
    char s2[96];
    int16_t packetsize, ps2;

    packetsize = pack(buf, "chhlsf", (int8_t)'B', (int16_t)0,
                      (int16_t)37, (int32_t)-5, s, (float32_t)-3490.6677);
    packi16(buf+1, packetsize); // 재미삼아 패킷 크기를 넣어 둡니다

    printf("packet is %" PRIu32 " bytes\n", packetsize);

    unpack(buf, "chhl96sf", &magic, &ps2, &monkeycount, &altitude,
           s2, &absurdityfactor);

    printf("'%'c' %" PRIu32 " %" PRIu16 " %" PRIu32
           " \"%s\" %f\n", magic, ps2, monkeycount,
           altitude, s2, absurdityfactor);

    return 0;
}
```

여러분이 직접 만든 코드를 쓰건 다른 사람이 작성한 것을 쓰건, 매번 각 비트를 수동으로 포장하기보다는 버그를 통제하기 위해 일반적인 데이터 패킹 루틴을 사용하는 것이 좋은 습관입니다.

¹²<https://beej.us/guide/bgnet/source/examples/pack2.c>

데이터를 포장할 때에 쓰기 좋은 형식은 무엇일까요? 아주 좋은 질문입니다. 다행히도 RFC 4506¹³, 외부 데이터 표현 표준이 부동 소수점과 정수, 배열 등 다양한 자료형에 대해서 이진 형식을 정의합니다. 만약 데이터를 직접 처리할 생각이라면 이것을 준수하는 것을 권장합니다. 그러나 반드시 그래야 하는 것은 아닙니다. 패킷 경찰들이 문 앞까지 찾아오는 것은 아닙니다. 최소한 저는 그렇지 않을 것이라고 생각합니다.

어떤 경우에도, 데이터를 보내기 전에 어떤 식으로든 인코딩하는 것이 올바른 방식입니다.

7.6 망할 데이터 캡슐화

아무튼 데이터 캡슐화가 정말로 의미하는 것은 무엇일까요? 가장 단순한 경우 그것은 여러분이 데이터에 약간의 식별 정보나 패킷 길이 혹은 둘 모두를 담은 헤더를 붙여둔다는 뜻이 됩니다.

헤더가 어떤 모양을 하고 있어야 할까요? 사실 여러분의 프로젝트를 끝내기 위해서 필요하다고 느끼는 어떤 이진 데이터면 됩니다.

와. 정말 막연한 이야기입니다.

좋습니다. 예를 들자면 `SOCK_STREAM` 을 사용하는 다중 사용자 대화 프로그램이 있다고 합시다. 한 사용자가 뭔가 입력한다면, 두 조각의 정보가 서버에 전달되어야 합니다. 무엇을 말했는지, 그리고 누가 말했는지 그것입니다.

여기까지는 좋아 보입니다. “그럼 무엇이 문제인가요?”라고 여러분은 질문할 것입니다.

문제는 메시지가 가변 길이일 수 있다는 점입니다. “Tom”이라는 사용자가 “Hi”라고 말할 수 있고 “Benjamin”이라는 또 다른 사용자가 “Hey guys what is up?”이라고 말할 수도 있습니다.

그것이 들어오는 대로 클라이언트에게 `send()` 한다고 합시다. 여러분의 송신 데이터 스트림은 아래와 같을 것입니다.

```
t o m H i B e n j a m i n H e y g u y s w h a t i s u p ?
```

이런 식일 것입니다. 클라이언트가 어떻게 하면 메시지의 시작과 끝을 알 수 있을까요? 원한다면 모든 메시지가 같은 길이를 갖도록 하고 우리가 구현한 `sendall()` 함수를 그냥 호출할 수 있을 것입니다. 그러나 그렇게 하면 대역폭을 낭비하게 됩니다! “tom”이 “Hi”라고 말하는 일을 위해서 1024바이트를 `send()` 하고 싶지는 않을 것입니다.

그래서 우리는 데이터를 작은 헤더와 패킷 구조에 캡슐화합니다. 클라이언트와 서버 모두 이 데이터를 어떻게 포장하고 풀어내는지(때때로 “marshal”과 “unmarshal”이라고 부릅니다) 알고 있습니다. 어느새 클라이언트와 서버가 어떻게 통신하는지를 설명하는 프로토콜을 정의하기 시작한 셈입니다!

지금은 사용자의 이름이 `'\0'` 으로 패딩된 고정된 8개의 문자라고 가정합니다. 데이터는 최대 128개 문자로 구성되는 가변 길이 형태라고 합시다. 이 상황에서 쓸 수 있는 예제 패킷 구조를 살펴봅시다.

1. `len` (1바이트, 부호 없음)—패킷의 전체 길이, 8바이트의 사용자 이름과 대화 데이터의 길이를 센다.
2. `name` (8바이트)—사용자의 이름, 필요한 경우 NUL로 덧댐니다.
3. `chatdata` (*n*바이트)—데이터 자체, 최대 128바이트. 패킷의 길이는 이 데이터의 길이에 8을 더한 값으로 계산되어야 합니다.(위에서 언급한 이름 필드의 길이)

필자가 8바이트와 128바이트를 필드의 길이 제한으로 선택한 이유가 궁금한가요? 특별한 이유는 없고, 충분히 길 것이라고 생각했습니다. 그러나 아마도 8바이트는 여러분의 필요에는 조금 못 미칠 수도 있습니다. 그런 경우에는 이름 필드의 길이를 30바이트나 다른 값으로 설정할 수 있습니다. 선택은 여러분의 몫입니다.

위의 패킷 정의를 사용하는 첫 번째 패킷은 아래와 같은 정보로 구성될 수 있습니다. (16진수와 아스키 코드로 표시되었습니다.)

¹³<https://tools.ietf.org/html/rfc4506>

```

    0A      74 6F 6D 00 00 00 00 00      48 69
(길이)  T o m      (패딩)      H i

```

두 번째 패킷도 비슷합니다.

```

    18      42 65 6E 6A 61 6D 69 6E      48 65 79 20 67 75 79 73 20 77 ...
(길이)  B e n j a m i n      H e y      g u y s      w ...

```

(길이는 물론 네트워크 바이트 순서로 기록되어 있습니다. 이 경우 길이가 단일 바이트이므로 그것이 중요하지는 않지만, 일반적으로는 여러분의 패킷이 가지는 모든 이진 정수가 네트워크 바이트 순서로 기록되기를 원할 것입니다.)

이 데이터를 보낼 때 여러분은 위에서 제시된 `sendall()` 과 비슷한 함수를 쓸 수 있습니다. 그렇게 하면 데이터를 모두 전송하기 위해서 `send()` 를 여러 번 호출하는 한이 있어도 모든 데이터가 전송되는 것을 확인할 수 있습니다.

마찬가지로 이 데이터를 받을 때에도 약간의 추가적인 작업이 필요합니다. 이 데이터를 받을 때에도 부분적인 패킷(예를 들어 위의 벤자민으로부터 `recv()` 로 받은 것이 " 18 42 65 6E 6A "뿐일 수도 있습니다.)을 받을 가능성을 염두에 두어야 합니다. 전체 패킷을 받을 때까지 `recv()` 를 반복적으로 호출해야 합니다.

그러나 어떻게 해야 할까요? 우리는 패킷이 완성되기 위해서 받아야 하는 바이트의 총 개수를 알고 있습니다. 개수가 패킷의 앞쪽에 붙어 있기 때문입니다. 우리는 또한 패킷의 최대 크기가 $1 + 8 + 128$, 즉 137바이트라는 것을 알고 있습니다. (우리가 그렇게 정의했기 때문입니다.)

여기에서는 몇 가지 방식으로 일을 할 수 있습니다. 모든 패킷이 길이 정보로 시작한다는 것을 알고 있으므로 패킷의 길이를 얻기 위해서 `recv()` 를 호출할 수 있습니다. 그리고 길이를 가지고 있으면 전체 패킷을 받을 때까지 남은 길이를 명시하면서 `recv()` 를(아마도 반복적으로) 호출하는 것입니다. 이 방식의 장점은 하나의 패킷을 담기에 충분한 크기의 버퍼만 있으면 된다는 것이고, 단점은 모든 데이터를 받기 위해서 `recv()` 를 최소 두 번 호출해야 한다는 것입니다.

다른 옵션은 `recv()` 를 호출할 때 한 패킷의 최대 크기만큼을 받겠다고 지정하는 것입니다. 그 후에 받은 데이터를 버퍼의 뒤쪽에 쌓아두고, 패킷이 완성되었는지 확인합니다. 물론 다음 패킷의 일부를 받을 수 있으므로 그것을 위한 여분의 공간이 필요합니다.

이를 위해서 두 개의 패킷을 담기에 충분한 배열을 선언하면 됩니다. 이것은 패킷이 도착하는 대로 재구성하는 일에 사용할 작업 공간입니다.

데이터를 `recv()` 로 받을 때마다 그것을 작업 버퍼에 덧붙이고 패킷이 완성되었는지 확인합니다. 버퍼에 담긴 바이트의 개수가 헤더에 명시된 길이보다 많거나 같은지 확인한다는 뜻입니다(사실은 헤더에 헤더 자신의 길이가 포함되지 않으므로 +1을 해야 합니다). 만약 버퍼의 바이트 수가 1보다 적다면 물론 패킷은 완성되지 않은 것입니다. 또한 이 경우 버퍼의 첫 바이트를 읽어들이는 것도 그것은 쓰레기값이므로 그것을 감안한 처리를 해야 합니다.

패킷이 완성되면 여러분은 그것으로 여러분이 원하는 일을 할 수 있습니다. 패킷을 사용하거나, 그것을 여러분의 작업 버퍼에서 제거할 수 있습니다.

휴! 머릿속으로 아직 잘 따라오고 있나요? 여기 두 번째 문제가 있습니다. 한 번의 `recv()` 호출로 한 패킷을 넘어서는 분량을 읽어들이 수가 있습니다. 즉 작업 버퍼에 하나의 완전한 패킷과 다음 패킷의 불완전한 부분이 있을 수 있다는 것입니다. 제기랄. (그러나 이런 경우를 처리하기 위해서 여러분의 작업 버퍼를 두 개의 패킷을 담기에 충분한 크기로 만들어둔 것입니다.)

첫 패킷의 길이를 헤더를 통해 알고 있고 작업 버퍼에 있는 바이트의 수를 추적하고 있으므로, 뺄셈을 해서 작업 버퍼에 있는 바이트 중 몇 개가 다음(미완성된) 패킷에 속해 있는지 계산할 수 있습니다. 첫 번째 패킷을 처리한 후에는 그것을 작업 버퍼에서 제거하고 부분적인 두 번째 패킷을 버퍼의 앞쪽으로 옮겨서 다음 `recv()` 를 처리할 준비를 할 수 있습니다.

(독자 여러분 중 일부는 부분적인 두 번째 패킷을 작업 버퍼의 앞쪽으로 옮기는 것에 시간이 걸리고, 한형 버퍼를 사용하면 그 작업이 필요하지 않다는 것에 주목할 것입니다. 다른 독자들에게는 불행하게도, 한형 버퍼에 대한

논의는 이 글의 범위를 벗어납니다. 흥미가 있다면 자료 구조 책을 집어 들고 거기서부터 시작하면 됩니다.)

쉽다고 한 적은 없습니다. 사실 앞에서 쉽다고 말했습니다. 또 실제로도 그렇습니다. 단지 연습이 필요하고, 머지않아 자연스럽게 익숙해질 것입니다. 엑스칼리버에 맹세합니다!

7.7 브로드캐스트(Broadcast) 패킷 — Hello, World!

지금까지 이 안내서에서는 데이터를 하나의 호스트에서 다른 하나의 호스트로 보내는 일에 대해서 이야기했습니다. 그러나 적절한 권한이 있다면 *한 번에* 여러 호스트에게 데이터를 보낼 수도 있습니다!

UDP(TCP는 안 됩니다)와 표준 IPv4에서 이것은 *브로드캐스팅(Broadcasting)*이라는 메커니즘으로 가능합니다. IPv6에서 브로드캐스팅은 지원되지 않으며, 보통은 더 나은 기술인 *멀티캐스팅(Multicasting)*을 사용해야 합니다. 그러나 이번에는 그것에 대해서 다루지 않을 것입니다. 눈부신 미래에 대해서는 그만 이야기합시다. 우리는 32비트의 현재에 갇혀 있습니다.

잠깐! 그러나 무작정 브로드캐스팅을 시작할 수는 없습니다. 네트워크에 브로드캐스트 패킷을 전송하기 전에 `SO_BROADCAST` 소켓 옵션을 설정해야 합니다. 이것은 미사일 발사 스위치에 달아두는 플라스틱 덮개 같은 것입니다. 그만큼 강력한 도구라는 뜻입니다.

아무튼 진지하게 말하자면 브로드캐스트 패킷을 쓰는 일에는 위험이 따릅니다. 브로드캐스트 패킷을 받는 모든 시스템은 반드시 그 데이터가 어떤 포트를 목적으로 삼는지 알아내기 위해서 패킷의 데이터 캡슐화 계층이라는 양파 껍질을 벗겨내야 한다는 점이 바로 그것입니다. 그렇게 하고 난 후에야 시스템은 데이터를 포트에 건네줄지 아니면 무시할지를 결정합니다. 어떤 경우건 그것은 브로드캐스트 패킷을 받는 각 시스템에 큰 작업이고, 상당히 많은 시스템이 불필요한 작업을 할 수 있습니다. 게임 둠이 처음 세상에 나왔을 때 그것의 네트워크 코드에 대한 불평이 이런 것이었습니다.

자, 고양이 가족을 벗기는 방법은 하나만 있는 게 아니라는 말도 있죠¹⁴... 잠깐만요. 정말 고양이 가족을 벗기는 방법이 여러 가지라는 말을 하고 있는 건가요? 대체 그런 표현은 왜 있는 걸까요? 아무튼 마찬가지로, 브로드캐스트 패킷을 보내는 방법도 하나만 있는 것은 아닙니다. 핵심을 말하자면 이것입니다. 어떻게 브로드캐스트 메시지의 목적지 주소를 지정할 수 있을까요? 두 개의 일반적인 방법이 있습니다.

1. 데이터를 특정 서브넷의 브로드캐스트 주소로 보냅니다. 이것은 주소의 모든 호스트 부분 비트가 1로 설정된 서브넷 네트워크 주소입니다. 예를 들어 저의 네트워크는 집에서 `192.168.1.0` 이고 넷마스크는 `255.255.255.0` 입니다. 그러므로 주소의 마지막 바이트가 호스트 번호입니다(넷마스크에 따라 첫 세 바이트가 네트워크 주소이기 때문입니다). 그러므로 저의 브로드캐스트 주소는 `192.168.1.255` 입니다. 유닉스에서는 `ifconfig` 명령이 이 모든 정보를 줄 것입니다. (궁금한 분을 위해 적자면 브로드캐스트 주소를 얻기 위한 비트단위 논리연산은 **네트워크 번호 OR (NOT 넷마스크)**입니다.) 여러분은 이 종류의 브로드캐스트 패킷을 로컬 네트워크뿐 아니라 원격 네트워크에도 보낼 수 있습니다. 그러나 이 경우 목적지의 라우터가 패킷을 무시할 가능성이 존재합니다. (이런 패킷을 무시하지 않으면 공격자가 브로드캐스트 통신을 과다하게 발송할 수 있습니다.)
2. 데이터를 "전역" 브로드캐스트 주소로 보냅니다. 이것은 `255.255.255.255`, 통칭 `INADDR_BROADCAST` 입니다. 많은 시스템은 이것을 자동으로 여러분의 네트워크 번호와 비트단위 AND연산 해서 네트워크 브로드캐스트 주소로 변환할 것입니다. 그러나 일부는 그렇게 하지 않을 것입니다. 이는 시스템마다 다릅니다. 역설적이게도 라우터들은 이 종류의 브로드캐스트 패킷을 로컬 네트워크 너머로 전송하지 않습니다.

`SO_BROADCAST` 소켓 옵션을 지정하지 않고 브로드캐스트 주소에 데이터를 보내려고 하면 어떤 일이 생길까요?

오래됐지만 유용한 `talker` 와 `listener` 를 실행해보고 무슨 일이 생기는지 봅시다.

```
$ talker 192.168.1.2 foo
sent 3 bytes to 192.168.1.2
```

¹⁴기록 차원에서 말해 두자면, 저는 고양이를 좋아합니다. 최고죠. 평생 여러 마리를 사랑하며 함께 살았습니다. 기원을 알 수 없는 이 음울한 비유 표현을 싫어하는 분들이 있다는 것도 알지만, 이 안내서의 이 부분에는 이 표현이 가장 잘 맞는다고 생각합니다. (역자 주: 각주 번호 6178은 숫자를 180도 돌려 읽으면 "glib"처럼 보이는 말장난입니다. "glib"은 말만 번지르르하고 깊이가 없다는 뜻입니다.)

```
$ talker 192.168.1.255 foo
sendto: Permission denied
$ talker 255.255.255.255 foo
sendto: Permission denied
```

별로 좋지 않은 상황입니다. `SO_BROADCAST` 을 설정하지 않았기 때문입니다. 설정을 한 뒤에는 원하는 곳 어디에든 `sendto()` 를 할 수 있습니다.

사실 그것이 브로드캐스트를 할 수 있는 UDP 응용프로그램과 그렇지 않은 응용프로그램의 유일한 차이입니다. 그러니 오래된 `talker` 응용프로그램에 `SO_BROADCAST` 소켓 옵션을 설정하는 부분을 하나 추가해봅시다. 이 프로그램을 `broadcaster.c`¹⁵라고 부를 것입니다.

```
/*
** broadcaster.c -- talker.c와 같은 데이터그램 클라이언트, 다만
**                이 프로그램은 브로드캐스트를 할 수 있습니다.
*/

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <errno.h>
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <netdb.h>

#define SERVERPORT 4950 // 사용자들이 연결할 포트

int main(int argc, char *argv[])
{
    int sockfd;
    struct sockaddr_in their_addr; // 연결 대상의 주소 정보
    struct hostent *he;
    int numbytes;
    int broadcast = 1;
    //char broadcast = '1'; // 동작하지 않으면 이것을 써 보세요

    if (argc != 3) {
        fprintf(stderr, "usage: broadcaster hostname message\n");
        exit(1);
    }

    if ((he=gethostbyname(argv[1])) == NULL) { // 호스트 정보를 받아옵니다
        perror("gethostbyname");
        exit(1);
    }

    if ((sockfd = socket(PF_INET, SOCK_DGRAM, 0)) == -1) {
        perror("socket");
```

¹⁵<https://beej.us/guide/bgnet/source/examples/broadcaster.c>

```

    exit(1);
}

// 이 호출이 브로드캐스트 패킷을 보낼 수 있게 만듭니다
if (setsockopt(sockfd, SOL_SOCKET, SO_BROADCAST, &broadcast,
    sizeof broadcast) == -1) {
    perror("setsockopt (SO_BROADCAST)");
    exit(1);
}

their_addr.sin_family = AF_INET;    // IPv4 주소 계열
their_addr.sin_port = htons(SERVERPORT); // short, 네트워크 바이트 순서
their_addr.sin_addr = *((struct in_addr *)he->h_addr);
memset(their_addr.sin_zero, '\0', sizeof their_addr.sin_zero);

numbytes = sendto(sockfd, argv[2], strlen(argv[2]), 0,
    (struct sockaddr *)&their_addr, sizeof their_addr);

if (numbytes == -1) {
    perror("sendto");
    exit(1);
}

printf("sent %d bytes to %s\n", numbytes,
    inet_ntoa(their_addr.sin_addr));

close(sockfd);

return 0;
}

```

이 프로그램과 “평범한” UDP 클라이언트/서버의 상황에는 어떤 차이가 있을까요? 아무것도 없습니다! (이 경우에는 클라이언트가 브로드캐스트 패킷을 보낼 수 있다는 점을 빼면) 앞서와 마찬가지로 이전에 언급한 UDP `listener`를 한 창에 실행하고 다른 창에 `broadcaster`를 실행하세요. 위에서 실패한 전송이 성공할 것입니다.

```

$ broadcaster 192.168.1.2 foo
sent 3 bytes to 192.168.1.2
$ broadcaster 192.168.1.255 foo
sent 3 bytes to 192.168.1.255
$ broadcaster 255.255.255.255 foo
sent 3 bytes to 255.255.255.255

```

`listener`가 수신한 패킷에 반응하는 것을 볼 수 있어야 합니다. (만약 `listener`가 반응하지 않는다면 그것이 IPv6 주소에 바인드되어서 그럴 수 있습니다. `listener.c`의 `AF_INET6`를 `AF_INET`으로 바꿔서 IPv4를 강제해 보세요.)

자, 여기까지도 조금 재미있었습니다. 그러나 같은 네트워크에 있는 다른 컴퓨터에서 `listener`를 실행해서 각 컴퓨터에서 하나씩 실행되게 한 후에 `broadcaster`에 브로드캐스트 주소를 넣고 다시 실행해봅시다. `sendto()`를 한 번만 실행했음에도 두 개의 `listener` 모두가 패킷을 받습니다! 멋지네요!

만약 `listener`가 실행 중인 컴퓨터의 IP 주소를 목적으로 하여 전송된 데이터는 받는데 브로드캐스트 주소로 보낸 데이터는 받지 못한다면 아마도 방화벽이 컴퓨터에서 패킷을 막고 있을 것입니다. (그래요, Pat, Bapper. 이게 제 샘플 코드가 동작하지 않았던 이유라는 것을 저보다 먼저 깨달아 줘서 고마워요. 안내서에 여러분을

언급하겠다고 했지요. 바로 이 부분에 적었습니다.)

다시 말하지만 브로드캐스트 패킷을 다룰 때에는 주의하세요. LAN에 있는 모든 시스템이 `recvfrom()` 수행 여부와 상관없이 패킷을 처리해야 하므로 전체 컴퓨터 네트워크에 상당한 부하를 줄 수 있습니다. 브로드캐스트 패킷은 반드시 가끔씩만, 그리고 적절한 상황에서만 쓰여야 합니다.

Chapter 8

일반적인 질문들

이 헤더 파일들은 어디서 찾을 수 있을까요?

여러분의 시스템에 헤더 파일이 없다면 아마도 그것이 필요하지 않을 것입니다. 사용 중인 플랫폼의 설명서를 참고하세요. Windows를 위해 작업하고 있다면 `#include <winsock.h>` 만 하면 됩니다. (역자 주: `winsock.h` 는 Winsock 1.1용입니다. 현재 Windows 소켓 프로그램에서는 보통 `winsock2.h` 를 사용하며, `getaddrinfo()` 계열을 사용한다면 `ws2tcpip.h` 도 포함합니다.)

`bind()` 가 “Address already in use” 를 보고하면 어떻게 해야 하나요?

리스닝 소켓에 `setsockopt()` 를 `SO_REUSEADDR` 옵션과 함께 사용해야 합니다. 예제가 필요하다면 `bind()` 에 관한 절과 `select()` 에 관한 절을 참고하세요.

시스템에 열린 소켓의 목록을 얻으려면 어떻게 해야 하나요?

`netstat` 를 사용하세요. 자세한 정보는 `man` 을 참고해야 하지만 아래와 같이 입력해도 약간의 유용한 출력을 얻을 수 있습니다.

```
$ netstat
```

비결은 어떤 소켓이 어떤 프로그램과 연결되어 있는지 알아내는 것입니다. :-)

라우팅 테이블을 보려면 어떻게 해야 하나요?

`route` 명령(대개의 Linux 시스템에서 `/sbin` 에 있습니다) 을 실행하세요. 아니면 `netstat -r` 명령을 실행하세요. 혹은 `ip route` 명령일 수도 있습니다.

컴퓨터가 하나밖에 없다면 어떻게 클라이언트와 서버 프로그램을 실행하나요? 네트워크 프로그램을 작성하려면 네트워크가 있어야 하는 것 아닌가요?

여러분에게는 다행히도 사실상 모든 컴퓨터가 커널에 자리 잡고 네트워크 카드인 척하는 루프백 네트워크 “장치”를 구현합니다. (이것은 라우팅 테이블에서 “`lo`”라는 이름으로 표시되는 인터페이스입니다.)

여러분이 “`goat`”라는 이름의 컴퓨터에 로그인했다고 합시다. 클라이언트를 하나의 창에서 실행하고 서버를 다른 창에서 실행합시다. 아니면 서버를 백그라운드에서 실행하고 (“`server &`”) 클라이언트를 같은 창에서 실행합시다. 루프백 장치는 여러분이 `client goat` 와 `client localhost` (“`localhost`”는 여러분의 `/etc/hosts` 파일에 정의되어 있을 것입니다.) 중 어떤 것이든 할 수 있게 해 주므로 네트워크 없이도 서버와 대화하는 클라이언트 프로그램을 시험할 수 있습니다.

간단히 말하자면 네트워크 없는 단일 컴퓨터에서 코드를 실행하기 위해 코드를 변경할 필요는 없습니다. 만세!

원격 쪽에서 연결을 닫았는지 어떻게 알 수 있을까요?

`recv()` 가 0 을 반환하는 것으로 알 수 있습니다.

“ping” 유틸리티를 만들려면 어떻게 해야 하나요? ICMP는 무엇인가요? raw 소켓과 SOCK_RAW 에 대해서는 어디에서 더 알아볼 수 있을까요?

raw 소켓에 대한 모든 질문은 W. Richard Stevens' UNIX Network Programming books 에서 답을 얻을 수 있습니다. 또한 온라인으로 사용 가능한¹ Stevens' UNIX Network Programming source code에서 `ping/` 하위 디렉터리를 살펴보세요.

`connect()` 에 대한 제한 시간을 변경하거나 단축할 수 있을까요?

W. Richard Stevens가 여러분에게 줄 수 있는 답과 동일한 답을 드리는 대신, UNIX Network Programming source code의 `lib/connect_nonb.c`² 를 안내해드리겠습니다.

요점은 `socket()` 으로 소켓 설명자를 만든 후 논블로킹으로 설정한 뒤 `connect()` 를 호출할 때 모든 것이 잘 돌아간다면 `connect()` 는 즉시 -1 을 반환할 것이고 `errno` 는 `EINPROGRESS` 로 설정될 것이라는 것입니다. 그 후 `select()` 를 호출할 때 소켓 설명자를 읽기와 쓰기 집합에 모두 넣으면서 여러분이 원하는 제한 시간을 지정하면 됩니다. 시간 초과가 발생하지 않으면 `connect()` 이 완료되었다는 의미입니다. 이 시점에서 `getsockopt()` 를 `SO_ERROR` 옵션과 함께 호출해서 `connect()` 호출의 반환값을 얻을 수 있고, 오류가 없었다면 그 값은 0이어야 합니다.

마지막으로 여러분은 아마도 해당 소켓에 데이터를 전송하기 전에 소켓을 다시 블로킹 모드로 설정하고 싶을 것입니다.

이 방식은 프로그램이 연결을 시작하는 동안 다른 일을 할 수 있게 해 주는 장점도 있음에 주목하세요. 예를 들어 500ms 정도의 짧은 제한 시간을 설정한 후 시간 초과가 일어날 때마다 화면의 표시를 갱신하고, `select()` 를 다시 호출할 수 있습니다. `select()` 가 20번 정도 시간 초과를 일으킨다면 연결을 포기할 때가 되었음을 알 수 있습니다.

위에서도 말씀드렸지만 완벽하게 훌륭한 예제가 필요하다면 Stevens의 코드를 참고하세요.

Windows를 위해 빌드하려면 어떻게 하나요?

먼저 윈도우를 삭제한 후 리눅스나 BSD를 설치하세요. `};-)` . 사실 그럴 필요는 없고, 도입부의 Windows에서 빌드하기에 관한 절을 살펴보세요.

Solaris/SunOS에서 빌드하려면 어떻게 하나요? 컴파일을 시도하면 계속 링커 오류가 발생합니다!

링커 오류는 Sun 시스템이 자동으로 소켓 라이브러리를 링크하지 않기 때문에 발생합니다. 컴파일을 위해서는 도입부의 Solaris/SunOS를 위한 절 을 참고하세요.

왜 `select()` 가 시그널을 받으면 실패하나요?

시그널은 블로킹 중인 시스템 콜이 `errno` 를 `EINTR` 로 설정하고 -1 을 반환하게 만듭니다. `sigaction()` 으로 시그널 핸들러를 설정하면 `SA_RESTART` 플래그를 설정할 수 있는데 이것은 중단된 시스템 콜을 다시 시작하게 해 줄 것입니다.

물론 이런 방식이 늘 작동하는 것은 아닙니다.

제가 선호하는 해결책은 `goto` 문을 쓰는 방법입니다. 이것이 교수님들을 아주 짜증나게 할 수 있다는 것도 잘 알 테니, 한번 해 보세요!

¹<http://www.unpbook.com/src.html>

²<http://www.unpbook.com/src.html>

```
select_restart:
if ((err = select(fdmax+1, &readfds, NULL, NULL, NULL)) == -1) {
    if (errno == EINTR) {
        // 어떤 시그널이 우리에게 인터럽트를 걸었습니다. 그러니 재시작합니다
        goto select_restart;
    }
    // 진짜 오류는 여기서 처리합니다
    perror("select");
}
```

물론 여기에서 `goto` 를 쓸 필요는 없습니다. 다른 구조로 제어할 수도 있습니다. 그러나 저는 이 경우 `goto` 문이 사실 더 깔끔하다고 생각합니다.

`recv()` 호출에 시간 제한을 적용하려면 어떻게 해야 하나요?

`select()` 를 사용하세요! `select()` 는 읽어들이려는 소켓 설명자에 시간 제한 매개변수를 지정할 수 있게 해 줍니다. 아니면 모든 기능을 아래와 같이 하나의 함수에 감쌀 수 있습니다.

```
#include <unistd.h>
#include <sys/time.h>
#include <sys/types.h>
#include <sys/socket.h>

int recvtimeout(int s, char *buf, int len, int timeout)
{
    fd_set fds;
    int n;
    struct timeval tv;

    // 파일 설명자 집합을 설정합니다
    FD_ZERO(&fds);
    FD_SET(s, &fds);

    // 시간 제한을 위한 struct timeval을 설정합니다
    tv.tv_sec = timeout;
    tv.tv_usec = 0;

    // 시간이 초과되거나 데이터를 받을 때까지 기다립니다
    n = select(s+1, &fds, NULL, NULL, &tv);
    if (n == 0) return -2; // 시간 초과!
    if (n == -1) return -1; // 오류

    // 여기에 데이터가 있어야 하므로 일반 recv()를 수행합니다
    return recv(s, buf, len, 0);
}

.
.
.
// recvtimeout() 호출 예시
n = recvtimeout(s, buf, sizeof buf, 10); // 10초 제한 시간

if (n == -1) {
    // 오류가 발생했습니다
```

```

    perror("recvtimeout");
}
else if (n == -2) {
    // 시간이 초과되었습니다
} else {
    // buf에 데이터가 들어 있습니다
}
.
.
.

```

`recvtimeout()` 이 시간 초과 상황에서 `-2` 를 반환한다는 점에 주목하세요. 왜 `0` 이 아닌지 궁금한가요? `recv()` 가 원격 연결이 닫혔을 때 `0` 을 반환한다는 점을 떠올려봅시다. 그러니 그 값은 쓸 수가 없고, `-1` 은 “오류”를 의미합니다. 그래서 저는 시간 초과를 가리키는 값으로 `-2` 를 썼습니다.

소켓에 데이터를 보내기 전에 암호화하거나 압축하려면 어떻게 해야 하나요?

데이터를 암호화하기 위한 간편한 방법 중 하나는 SSL(secure sockets layer)를 사용하는 것입니다. 그러나 그것은 이 안내서의 범위를 벗어납니다. (더 많은 정보가 필요하면 OpenSSL 프로젝트³를 확인하세요.) (역주: 현재는 SSL이라는 명칭 대신 TLS를 사용하며, 새 구현에서는 TLS 1.2 이상을 사용하는 것이 일반적입니다.)

그러나 만약 여러분이 자신만의 압축기나 암호화 체계를 만들거나 써 보고 싶다면, 여러분의 데이터가 양 끝 사이에서 정해진 단계를 거친다고 생각해 보세요. 각 단계는 데이터를 특정한 방법으로 바꿉니다.

1. 서버가 데이터를 파일(또는 다른 것)에서 읽어옵니다
2. 서버가 데이터를 암호화/압축합니다(여러분이 이 단계를 추가합니다)
3. 서버가 암호화된 데이터를 `send()` 합니다

반대편은 이렇습니다.

1. 클라이언트가 암호화된 데이터를 `recv()` 합니다
2. 클라이언트가 데이터를 복호화/압축 해제합니다(여러분이 이 단계를 추가합니다)
3. 클라이언트가 데이터를 파일(또는 다른 것)에 씁니다

만약 압축과 암호화를 모두 할 생각이라면 압축을 먼저 해야 한다는 점을 기억하세요. :-)

클라이언트가 서버가 했던 작업을 제대로 거꾸로 수행하기만 한다면 여러분이 얼마나 많은 단계를 추가하든 데이터는 결국 무사할 것입니다.

그러므로 제 예제 코드를 바탕으로 작업하려면, 데이터를 읽어 온 뒤 `send()` 로 네트워크에 보내기 전에 실행되는 부분을 찾아 그 사이에 암호화 코드를 끼워넣으면 됩니다.

“PF_INET”이 계속 등장하는데 무엇인가요? AF_INET 과 관계가 있을까요?

그렇습니다. 관계가 있습니다. 자세한 사항은 `socket()` 에 대한 절을 참고하세요.

클라이언트에게서 셸 명령을 받아서 실행하는 서버는 어떻게 만드나요?

단순함을 위해서 클라이언트가 `connect()` 와 `send()` 후에 연결을 `close()` 처리한다고 가정합니다. (즉 클라이언트가 다시 연결하지 않는 한 후속 시스템 콜은 없다는 의미입니다.)

클라이언트의 과정은 이렇습니다.

1. 서버에 `connect()`
2. `send("/sbin/ls > /tmp/client.out")`
3. 연결을 `close()` 로 닫음

³<https://www.openssl.org/>

한편 서버는 데이터를 받아서 실행합니다.

1. 클라이언트의 연결 요청에 대한 `accept()`
2. 명령 문자열에 대한 `recv(str)`
3. 연결을 `close()` 로 닫음
4. 명령을 실행하기 위해서 `system(str)`

주의하세요! 클라이언트가 말하는 것을 서버가 실행한다는 것은 원격 셸 접근 권한을 주는 것과 비슷한 일이고, 사람들이 서버에 접속할 때 여러분의 계정으로 무엇이든 할 수 있다는 의미입니다. 위의 예제에서 클라이언트가 "`rm -rf ~`"를 보내면 어떻게 될까요? 여러분의 계정이 가진 모든 것을 삭제할 것입니다!

그러나 여러분이 현명하다면 안전하다고 확신하는 몇 개의 유틸리티, 예를 들어 `foobar` 외의 것을 클라이언트가 실행하지 못하도록 하는 것이 좋습니다.

```
if (!strcmp(str, "foobar", 6)) {
    sprintf(sysstr, "%s > /tmp/server.out", str);
    system(sysstr);
}
```

그러나 불행히도 이것만으로는 여전히 위험합니다. 클라이언트가 "`foobar; rm -rf ~`"를 입력한다면 어떻게 될까요? 가장 안전한 방식은 명령의 매개변수에 들어가는 숫자나 영문자가 아닌 모든 문자(필요하다면 공백 문자도) 앞에 이스케이프("`\`") 문자를 붙이는 것입니다.

보시다시피 보안은 클라이언트가 보낸 것을 서버가 실행할 때에 큰 문제가 됩니다.

제가 꽤 큰 데이터를 보내는데 `recv()` 를 해보면 한 번에 536바이트나 1460바이트 씩만 받아옵니다. 그러나 이것을 로컬 컴퓨터에서 실행하면 한 번에 모든 데이터를 받아옵니다. 왜 이런 것인가요?

MTU에 도달한 것입니다. 이것은 물리 매체가 처리할 수 있는 최대 크기입니다. 로컬 컴퓨터에서는 루프백 장치를 쓰기 때문에 8K나 그 이상의 크기도 문제없이 다룰 수 있습니다. 그러나 이더넷에서는 헤더를 포함해 1500바이트가 한계입니다. 모뎀을 쓴다면 (마찬가지로 헤더를 포함해) 576바이트가 한계입니다.

일단 모든 데이터가 전송되었음을 확실히 해야 합니다. (자세한 정보는 `sendall()` 함수의 구현을 확인하세요.) 전송이 잘 되었음이 확실하다면 모든 데이터를 읽어들이 때까지 `recv()` 를 반복문 내부에서 호출해야 합니다.

여러 번의 `recv()` 호출을 통해 완전한 패킷을 수신하는 작업에 대해 자세한 정보가 필요하다면 망할 데이터 캡슐화 절을 참고하세요.

저는 Windows 시스템을 써서 `fork()` 시스템 호출이 없고 `struct sigaction` 같은 것도 없습니다. 어떻게 해야 하나요?

이것이 있다면 그것은 컴파일러와 함께 있는 POSIX 라이브러리에 있을 것입니다. 저는 Windows 시스템을 가지고 있지 않으므로 그에 대해 정확한 답을 줄 수 없습니다. 그러나 기억하기로는 마이크로소프트가 POSIX 호환성 계층을 만들었고 `fork()` 도 거기에 있을 것입니다. (어쩌면 `sigaction` 도 있을 것입니다.) (역자 주: 최신 Windows 환경에서는 보통 POSIX 호환 계층에 기대기보다 Windows의 고유 API나 WSL 같은 별도 실행 환경을 검토하는 편이 더 나을 수 있습니다.)

VC++에 달려오는 도움말에서 "`fork`"나 "`POSIX`"를 검색하고 도움이 될 만한 것이 있는지 살펴보세요. (역자 주: 여기서 VC++는 Visual Studio 및 Microsoft C/C++ 개발 도구의 예전 이름입니다. 이 안내서가 처음 쓰인 시점을 감안해서 읽는 편이 좋습니다.)

그것이 전혀 작동하지 않는다면, `fork()` / `sigaction` 과 관련된 것들을 떼어내고 Win32에 대응하는 `CreateProcess()` 로 교체하세요. 저는 `CreateProcess()` 를 어떻게 쓰는지 모릅니다. 그것은 인수를 엄청나게 많이 받지만 아마도 VC++과 함께 제공되는 문서에 설명이 있을 것입니다.

저는 방화벽 뒤에 있습니다. 방화벽 너머의 사람들이 저의 IP 주소를 알고 저의 시스템에 접근하게 하려면 어떻게 해야 하나요?

불행히도 방화벽의 목적은 방화벽 바깥의 사람들이 방화벽 안의 시스템에 접근하는 것을 막는 것입니다. 그러므로 그것을 허용하는 것은 보안에 구멍을 내는 일로 여겨집니다.

그러나 아무 방법도 없다고 말하려고 이 이야기를 꺼낸 것은 아닙니다. 방화벽이 마스커레이딩이나 NAT 처리 같은 것을 한다면 여전히 `connect()` 로 방화벽 너머에 접근할 수 있습니다. 여러분의 프로그램이 언제나 연결을 개시하는 쪽이 되도록 한다면 문제는 없을 것입니다.

만약 그것으로는 충분하지 않다면, 시스템 관리자에게 부탁해서 방화벽에 구멍을 내서 여러분에게 연결할 수 있도록 해야 합니다. 방화벽은 자체 NAT 소프트웨어나 프록시 등을 써서 여러분에게 연결을 전달 해줄 수 있습니다.

방화벽의 구멍은 가볍게 볼 것이 아니라는 점을 기억하세요. 나쁜 사람들에게 내부 네트워크에 대한 접근 권한을 주지 않도록 해야 합니다. 초보자라면 소프트웨어를 안전하게 만드는 것이 생각보다 어렵다는 것을 알아야 합니다.

여러분의 시스템 관리자가 저를 탓하는 일이 없게 해주세요. ;-)

패킷 스니퍼는 어떻게 작성하나요? 어떻게 하면 제 이더넷 인터페이스를 프로미스큐어스 모드로 설정할 수 있을까요?

모르는 이들을 위해 설명하자면, 네트워크 카드가 “프로미스큐어스 모드(promiscuous mode)” 일 때 목적지 주소가 현재 컴퓨터가 아닌 패킷까지 전부 운영체제에 전달합니다. (우리는 IP 주소가 아닌 이더넷 계층 주소에 대해서 이야기하는 것입니다. 그러나 이더넷은 IP보다 낮은 계층이므로, 사실상 모든 IP 주소에 대한 통신이 전달됩니다. 더 자세한 내용은 저수준 센스와 네트워크 이론을 참고하세요.)

이것이 패킷 스니퍼 동작의 기본 원리입니다. 패킷 스니퍼는 인터페이스를 프로미스큐어스 모드로 만들고, 운영체제는 그 인터페이스를 통해 전달되는 모든 패킷을 받게 됩니다. 여러분은 이런 데이터를 읽을 수 있는 몇 가지 종류의 소켓을 쓸 수 있습니다.

불행히도 질문에 대한 답은 플랫폼에 따라 다릅니다. 그러나 인터넷을 찾아보면, 예를 들어 “windows promiscuous ioctl”을 검색한다면 도움이 되는 정보를 얻을 수 있을 것입니다. Linux를 위해서는 유용해 보이는 Stack Overflow 스레드⁴도 있습니다.

어떻게 하면 TCP나 UDP 소켓에 대해서 사용자 정의한 제한 시간 값을 사용할 수 있을까요?

시스템에 따라 다릅니다. 여러분의 시스템이 어떤 기능을 지원하는지 알아내기 위해서 (그리고 그것을 `setsockopt()` 에 쓰기 위해서) `SO_RCVTIMEO` 나 `SO_SNDTIMEO` 같은 것을 인터넷에서 찾아봐야 할 것입니다.

Linux 매뉴얼지는 `alarm()` 이나 `setitimer()` 를 대체재로 쓸 것을 권합니다.

어떤 포트가 사용 가능한 상태인지는 어떻게 알아내나요? “공식적인” 포트 번호 목록 같은 것이 있을까요?

보통 이것은 문제가 되지 않습니다. 여러분이 웹 서버를 작성한다고 하면, 80번같이 잘 알려진 포트를 쓰는 것이 좋습니다. 여러분만의 특별한 목적의 서버를 작성한다면 무작위의 포트 번호(그러나 1023보다 큰 것으로)를 고르고 시도해보세요.

만약 포트가 이미 사용 중이라면 `bind()` 를 시도할 때 “Address already in use” 오류가 발생할 것입니다. 다른 포트를 고르세요. (여러분의 소프트웨어의 사용자가 설정 파일이나 명령줄 스위치로 대체 포트를 지정할 수 있게 하는 것이 좋습니다.)

인터넷 할당 번호 관리 기관(the Internet Assigned Numbers Authority, IANA) 이 관리하는 공식 포트 번호⁵ 가 있습니다. (1023보다 큰) 어떤 번호가 저 목록에 있다고 해서 그 포트를 쓸 수 없는 것은 아닙니다. 예를 들어 Id Software의 DOOM은 “mdqs”(그게 무엇이든) 와 같은 포트를 씁니다. 중요한 것은 같은 컴퓨터의 누구도 여러분이 그 포트를 쓰고 싶을 때 그 포트를 쓰고 있지 않으면 된다는 것입니다.

⁴<https://stackoverflow.com/questions/21323023/>

⁵<https://www.iana.org/assignments/port-numbers>

Chapter 9

맨페이지

유닉스의 세상에는 많은 설명서들이 있습니다. 거기에는 여러분이 쓸 수 있는 개별 함수에 대한 짧은 절이 있습니다.

물론 `manual` 은 타자로 치기에 너무 길 것입니다. 그 말인즉 저를 포함해 유닉스 세상의 누구도 그만큼을 치고 싶어하지 않습니다. 물론 저는 제가 얼마나 간결한 것을 선호하는지에 대해서 이렇게 저렇게 길게 늘어서 말할 수 있을 것입니다. 그러나 저는 제가 얼마나 완전히 놀라우리만치 사실상 총체적인 모든 상황에서 간결하기를 선호하는지에 대해서 긴 웅변을 늘어놓는 대신 짧게 말하도록 하겠습니다.

[박수 소리]

(박수쳐주셔서) 감사합니다. 제가 말하고자 하는 바는 이런 페이지들이 유닉스 세상에서 "맨페이지"라고 불린다는 것입니다. 그리고 독자 여러분의 읽는 재미를 위해서 저의 개인적인 축약판을 여기에 포함해두었습니다. 그 말인즉 이 함수들 대부분은 저의 사용법보다 훨씬 더 범용적이지만 저는 인터넷 소켓 프로그래밍에 연관된 부분만을 제시할 것이라는 의미입니다.

그러나 잠깐! 저의 맨페이지에서 잘못된 부분은 이것만이 아닙니다.

- 이 문서들은 불완전하고 안내서의 기본적인 부분만을 보여줍니다.
- 현실 세계에는 여기에 있는 것보다 훨씬 많은 맨페이지가 있습니다.
- 여러분의 시스템에 있는 것과 다를 수 있습니다.
- 여러분의 시스템에 있는 특정 함수에 대해서는 헤더 파일이 다를 수 있습니다.
- 여러분의 시스템에 있는 특정 함수에 대해서는 함수 매개변수가 다를 수 있습니다.

진짜 정보를 원한다면 `man whatever` 를 입력해서 여러분의 로컬 유닉스 맨페이지를 참고하세요. "whatever"는 예를 들자면 "accept" 처럼 여러분이 큰 관심을 갖는 사안입니다. (마이크로소프트 비주얼 스튜디오도 그들의 도움말 부분에 이것과 유사한 것을 가지고 있다고 생각합니다. 그러나 "man"이 "help"보다 1바이트 더 간결하므로 더 좋습니다. 이번에도 유닉스가 이겼습니다!)

그래서 이 정보에 흥미 있다면 애초에 안내서에 첨부한 이유는 무엇이냐고요? 말하자면 몇 가지 이유가 있습니다. 그러나 그 중 가장 좋은 이유는 바로 (a) 이 버전들은 네트워크 프로그래밍을 위해 특별히 재구성되었고 원본보다 이해하기 쉬우며 (b) 이 버전들에는 예제가 있다는 것입니다!

예제 이야기가 나왔으니 더 이야기하자면 코드의 길이가 너무 길어지는 점을 염려해서 오류 검사 코드를 모두 넣지는 않았습니다. 그러나 여러분은 여러분의 시스템 콜이 100% 성공할 것이라고 가정하지 않는 이상 오류 확인을 언제나 해야 합니다. 그리고 사실 그런 확신이 있어도 검사를 하는 것이 좋습니다!

9.1 `accept()`

리스닝 소켓에서 들어오는 연결을 받아들입니다

개요

```
#include <sys/types.h>
#include <sys/socket.h>

int accept(int s, struct sockaddr *addr, socklen_t *addrlen);
```

설명

`SOCK_STREAM` 소켓을 얻고 `listen()` 으로 들어오는 연결을 받을 준비를 마쳤다면, 그 다음에는 `accept()` 를 호출해서 새로 연결된 클라이언트와 이후 통신에 사용할 새 소켓 설명자를 실제로 얻습니다.

리스닝에 사용하던 기존 소켓은 여전히 남아 있으며, 이후 들어오는 연결에 대해 계속 `accept()` 를 호출하는 데 쓰입니다.

매개변수 설명

`s` `listen()` 중인 소켓 설명자입니다.
`addr` 여러분에게 연결하는 쪽의 주소로 채워집니다.
`addrlen` 호출 전에는 `addr` 가 가리키는 버퍼의 크기를 넣고, 호출 뒤에는 실제 피어 주소의 길이가 저장됩니다.
 피어 주소가 필요 없다면 `addr` 와 `addrlen` 을 모두 `NULL` 로 넘길 수 있습니다.

`accept()` 는 보통 블록됩니다. 미리 `select()` 를 사용해서 리스닝 소켓 설명자가 “읽을 준비”가 되었는지 살펴볼 수 있습니다. 그렇다면 `accept()` 가 받을 새 연결이 기다리고 있다는 뜻입니다! 야호! 다른 방법으로는 `fcntl()` 을 사용해서 리스닝 소켓에 `O_NONBLOCK` 플래그를 설정할 수도 있습니다. 그러면 이 소켓은 블록되지 않고, 대신 `errno` 를 `EWOULDBLOCK` 으로 설정한 채 `-1` 을 반환하게 됩니다.

`accept()` 가 반환하는 소켓 설명자는 원격 호스트에 연결되어 열린 상태인 진짜 소켓 설명자입니다. 사용이 끝나면 `close()` 해야 합니다.

반환값

`accept()` 는 새로 연결된 소켓 설명자를 반환합니다. 오류가 발생하면 `-1` 을 반환하고 `errno` 를 적절히 설정합니다.

예제

```
struct sockaddr_storage their_addr;
socklen_t addr_size;
struct addrinfo hints, *res;
int sockfd, new_fd;

// 먼저 getaddrinfo()로 주소 구조체 목록을 가져옵니다:

memset(&hints, 0, sizeof hints);
hints.ai_family = AF_UNSPEC; // IPv4와 IPv6 어느 쪽이든 사용
hints.ai_socktype = SOCK_STREAM;
hints.ai_flags = AI_PASSIVE; // bind()에 쓸 와일드카드 주소를 요청합니다
```

```

getaddrinfo(NULL, MYPORT, &hints, &res);

// 소켓을 만들고, 바인드하고, 리스닝합니다:

sockfd = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
bind(sockfd, res->ai_addr, res->ai_addrlen);
listen(sockfd, BACKLOG);

// 이제 들어오는 연결을 받아들입니다:

addr_size = sizeof their_addr;
new_fd = accept(sockfd, (struct sockaddr *)&their_addr, &addr_size);

// new_fd 소켓 설명자로 통신할 준비가 끝났습니다!

```

함께 보기

socket(), getaddrinfo(), listen(), struct sockaddr_in

9.2 bind()

소켓을 IP 주소 및 포트 번호와 연결합니다

개요

```

#include <sys/types.h>
#include <sys/socket.h>

int bind(int sockfd, struct sockaddr *my_addr, socklen_t addrlen);

```

설명

원격 호스트가 여러분의 서버 프로그램에 연결하려면 두 가지 정보가 필요합니다. IP 주소와 포트 번호입니다. bind() 호출은 바로 그 일을 할 수 있게 해 줍니다.

먼저 getaddrinfo() 를 호출해서 로컬 주소와 포트 정보가 담긴 struct sockaddr 를 불러옵니다. 그런 다음 socket() 을 호출해서 소켓 설명자를 얻고, 그 소켓과 주소를 bind() 에 넘깁니다. 그러면 IP 주소와 포트가 마법처럼(진짜 마법으로) 소켓에 묶입니다!

특정 로컬 IP 주소를 지정하지 않고 서버를 설정하려면 getaddrinfo() 의 hints 매개변수에 AI_PASSIVE 플래그를 넣고 첫 번째 인수에는 NULL 을 넘기면 됩니다. 그러면 결과 struct sockaddr 의 IP 주소 부분에는 bind() 에 쓸 와일드카드 값이 들어갑니다.

뭐라고요? 모든 로컬 인터페이스에 바인드하려면 struct sockaddr 의 IP 주소에 어떤 특별한 값을 넣어야 하느냐고요? 알려드리겠습니다. 하지만 이것은 여러분이 struct sockaddr 를 손으로 직접 채우는 경우에만 해당합니다. 그렇지 않다면 위에서 말한 대로 getaddrinfo() 의 결과를 쓰세요. IPv4에서는 struct sockaddr_in 구조체의 sin_addr.s_addr 필드를 INADDR_ANY 로 설정합니다. IPv6에서는

`struct sockaddr_in6` 구조체의 `sin6_addr` 필드에 전역 변수 `in6addr_any` 를 대입합니다. 또는 새 `struct in6_addr` 를 선언하는 중이라면 `IN6ADDR_ANY_INIT` 으로 초기화할 수 있습니다.

마지막으로 `addrlen` 매개변수는 `my_addr` 가 가리키는 주소 구조체의 크기로 설정되어야 합니다. 예를 들어 IPv4 주소 구조체라면 `sizeof(struct sockaddr_in)` 을 사용할 수 있습니다.

반환값

성공하면 0을 반환합니다. 오류가 발생하면 `-1` 을 반환하고 `errno` 를 적절히 설정합니다.

예제

```
// getaddrinfo()를 사용하는 현대적인 방식

struct addrinfo hints, *res;
int sockfd;

// 먼저 getaddrinfo()로 주소 구조체 목록을 가져옵니다:

memset(&hints, 0, sizeof hints);
hints.ai_family = AF_UNSPEC; // IPv4와 IPv6 어느 쪽이든 사용
hints.ai_socktype = SOCK_STREAM;
hints.ai_flags = AI_PASSIVE; // bind()에 쓸 와일드카드 주소를 요청합니다

getaddrinfo(NULL, "3490", &hints, &res);

// 소켓을 만듭니다:
// (실제로는 "res" 연결 리스트를 순회하고 오류 확인도 해야 합니다!)

sockfd = socket(res->ai_family, res->ai_socktype, res->ai_protocol);

// getaddrinfo()에 넘긴 포트에 바인드합니다:

bind(sockfd, res->ai_addr, res->ai_addrlen);
```

```
// 구조체를 직접 채우는 예제, IPv4

struct sockaddr_in myaddr;
int s;

myaddr.sin_family = AF_INET;
myaddr.sin_port = htons(3490);

// IP 주소를 지정할 수도 있고:
inet_pton(AF_INET, "63.161.169.137", &(myaddr.sin_addr));

// 모든 로컬 IPv4 인터페이스에서 받도록 설정할 수도 있습니다:
myaddr.sin_addr.s_addr = INADDR_ANY;

s = socket(PF_INET, SOCK_STREAM, 0);
bind(s, (struct sockaddr*)&myaddr, sizeof myaddr);
```

함께 보기

getaddrinfo(), socket(), struct sockaddr_in, struct in_addr

9.3 connect()

소켓을 서버에 연결합니다

개요

```
#include <sys/types.h>
#include <sys/socket.h>

int connect(int sockfd, const struct sockaddr *serv_addr,
            socklen_t addrlen);
```

설명

socket() 호출로 소켓 설명자를 만들었다면, 이름도 딱 맞는 connect() 시스템 호출을 사용해서 그 소켓을 원격 서버에 연결할 수 있습니다. 여러분이 해야 할 일은 소켓 설명자와, 좀 더 친해지고 싶은 서버의 주소를 넘기는 것뿐입니다. (아, 그리고 이런 함수에 흔히 넘기는 주소의 길어도 필요합니다.)

보통 이 정보는 getaddrinfo() 호출의 결과로 얻습니다. 하지만 원한다면 직접 struct sockaddr를 채워도 됩니다.

아직 그 소켓 설명자에 대해 bind()를 호출하지 않았다면, 그 소켓은 여러분의 IP 주소와 무작위 로컬 포트에 자동으로 바인드됩니다. 서버가 아니라면 보통 이것으로 충분합니다. 여러분은 로컬 포트가 무엇인지에는 별 관심이 없고, serv_addr 매개변수에 넣어야 할 원격 포트가 무엇인지만 신경 쓰기 때문입니다. 클라이언트 소켓이 특정 IP 주소와 포트에 있어야 한다면 bind()를 호출할 수는 있지만, 그런 일은 꽤 드뭅니다.

소켓이 connect()되면, 마음껏 그 소켓에서 데이터를 send()하고 recv()할 수 있습니다.

특별 참고: SOCK_DGRAM UDP 소켓을 원격 호스트에 connect()하면 sendto()와 recvfrom()뿐 아니라 send()와 recv()도 사용할 수 있습니다. 원한다면요.

반환값

성공하면 0을 반환합니다. 오류가 발생하면 -1을 반환하고 errno를 적절히 설정합니다.

예제

```
// www.example.com의 80번 포트(http)에 연결합니다

struct addrinfo hints, *res;
int sockfd;

// 먼저 getaddrinfo()로 주소 구조체 목록을 가져옵니다:
```

```
memset(&hints, 0, sizeof hints);
hints.ai_family = AF_UNSPEC; // IPv4와 IPv6 어느 쪽이든 사용
hints.ai_socktype = SOCK_STREAM;

// 다음 줄에서 "http" 대신 "80"을 넣을 수도 있습니다:
getaddrinfo("www.example.com", "http", &hints, &res);

// 소켓을 만듭니다:

sockfd = socket(res->ai_family, res->ai_socktype, res->ai_protocol);

// getaddrinfo()에 넘겼던 주소와 포트에 연결합니다:

connect(sockfd, res->ai_addr, res->ai_addrlen);
```

함께 보기

socket(), bind()

9.4 close()

소켓 설명자를 닫습니다

개요

```
#include <unistd.h>

int close(int s);
```

설명

여러분이 꾸며낸 어떤 미친 계획에든 소켓을 다 사용했고, 더 이상 그 소켓으로 `send()` 나 `recv()` 를 하고 싶지 않거나, 사실 그 어떤 일도 하고 싶지 않다면 `close()` 할 수 있습니다. 그러면 그 소켓은 해제되어 다시는 쓰이지 않습니다.

원격 쪽은 이런 일이 일어났는지 두 가지 방법 중 하나로 알 수 있습니다. 첫째: 원격 쪽이 `recv()` 를 호출하면 0 이 반환됩니다. 둘째: 원격 쪽이 `send()` 를 호출하면 SIGPIPE 신호를 받고, `send()` 는 -1 을 반환하며 `errno` 는 EPIPE 로 설정됩니다.

Windows 사용자: 여러분이 사용해야 하는 함수는 `close()` 가 아니라 `closesocket()` 입니다. 소켓 설명자에 `close()` 를 쓰려고 하면 Windows가 화를 낼 수도 있습니다... 그리고 화난 Windows는 별로 마음에 들지 않을 것입니다.

반환값

성공하면 0을 반환합니다. 오류가 발생하면 -1을 반환하고 `errno` 를 적절히 설정합니다.

예제

```
s = socket(PF_INET, SOCK_DGRAM, 0);
.
.
.
// 아주 많은 일들...*부우우우웅!*
.
.
.
close(s); // 정말로 별것 없습니다.
```

함께 보기

`socket()`, `shutdown()`

9.5 `getaddrinfo()`, `freeaddrinfo()`, `gai_strerror()`

호스트 이름 및/또는 서비스에 대한 정보를 얻고 그 결과로 `struct sockaddr` 를 채웁니다.

개요

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>

int getaddrinfo(const char *nodename, const char *servname,
                const struct addrinfo *hints, struct addrinfo **res);

void freeaddrinfo(struct addrinfo *ai);

const char *gai_strerror(int ecode);

struct addrinfo {
    int     ai_flags;           // AI_PASSIVE, AI_CANONNAME, ...
    int     ai_family;         // AF_xxx
    int     ai_socktype;        // SOCK_xxx
    int     ai_protocol;        // 0(자동) 또는 IPPROTO_TCP, IPPROTO_UDP

    socklen_t ai_addrlen;       // ai_addr의 길이
    char *ai_canonname;         // nodename의 정규 이름
    struct sockaddr *ai_addr;    // 이진 주소
    struct addrinfo *ai_next;    // 연결 리스트의 다음 구조체
};
```

설명

`getaddrinfo()` 는 특정 호스트 이름에 대한 정보(예를 들면 IP 주소)를 반환하고, 그 지저분한 세부사항(IPv4인지 IPv6인지 같은 것)을 알아서 처리해 `struct sockaddr` 를 채워 주는 훌륭한 함수입니다. 이 함수는 오래된 `gethostbyname()` 과 `getservbyname()` 을 대체합니다. 아래 설명에는 조금 겁먹을 만큼 많은 정보가 있지만, 실제 사용법은 꽤 단순합니다. 예제를 먼저 살펴보는 것도 좋습니다.

관심 있는 호스트 이름은 `nodename` 매개변수에 들어갑니다. 주소는 “www.example.com” 같은 호스트 이름일 수도 있고, 문자열로 넘긴 IPv4 또는 IPv6 주소일 수도 있습니다. `AI_PASSIVE` 플래그를 사용한다면(아래를 보세요) 이 매개변수는 `NULL` 일 수도 있습니다.

`servname` 매개변수는 기본적으로 포트 번호입니다. 포트 번호(“80”처럼 문자열로 넘깁니다)일 수도 있고, “http”, “tftp”, “smtp”, “pop” 같은 서비스 이름일 수도 있습니다. 잘 알려진 서비스 이름은 IANA 포트 목록¹이나 여러분의 `/etc/services` 파일에서 찾을 수 있습니다.

마지막 입력 매개변수로는 `hints` 가 있습니다. 사실상 여기에서 `getaddrinfo()` 함수가 무엇을 할지 정합니다. 사용하기 전에 `memset()` 으로 구조체 전체를 0으로 비우세요. 사용 전에 설정해야 하는 필드들을 살펴봅시다.

`ai_flags` 에는 여러 가지 값을 설정할 수 있지만, 여기 중요한 몇 가지가 있습니다. (여러 플래그는 `!` 연산자로 비트 OR해서 지정할 수 있습니다.) 전체 플래그 목록은 여러분의 맨페이지를 확인하세요.

`AI_CANONNAME` 은 결과의 `ai_canonname` 을 호스트의 정규(진짜) 이름으로 채우게 합니다. 첫 번째 인수가 `NULL` 일 때 `AI_PASSIVE` 는 결과의 IP 주소를 `INADDR_ANY` (IPv4) 또는 `in6addr_any` (IPv6)로 채웁니다. 이 와일드카드 주소로 `bind()` 하면 해당 주소 계열의 모든 로컬 인터페이스에서 그 포트로 들어오는 요청을 받을 수 있습니다. 주소를 하드코딩하고 싶지 않은 서버 설정에 아주 좋습니다.

`AI_PASSIVE` 플래그를 사용할 때 `nodename` 에 `NULL` 을 넘기면 위 와일드카드 주소를 얻습니다. 특정 `nodename` 을 넘기면 그 주소를 해석하며 `AI_PASSIVE` 는 영향을 주지 않습니다.

입력 매개변수를 계속 보자면, `ai_family` 는 아마 `AF_UNSPEC` 으로 설정하고 싶을 것입니다. 이것은 `getaddrinfo()` 에게 IPv4와 IPv6 주소를 모두 찾아보라고 말합니다. `AF_INET` 이나 `AF_INET6` 으로 둘 중 하나만 쓰게 제한할 수도 있습니다.

다음으로 `socktype` 필드는 원하는 소켓의 종류에 따라 `SOCK_STREAM` 또는 `SOCK_DGRAM` 으로 설정해야 합니다.

마지막으로 프로토콜 종류를 자동 선택하게 하려면 `ai_protocol` 은 그냥 `0` 으로 두세요.

이제 그 모든 내용을 채우고 나면 마침내 `getaddrinfo()` 를 호출할 수 있습니다!

물론 재미는 여기서 시작됩니다. 이제 `res` 는 `struct addrinfo` 들의 연결 리스트를 가리키고, 여러분은 이 목록을 순회하면서 `hints` 에 넘긴 조건과 맞는 모든 주소를 얻을 수 있습니다.

어떤 이유로든 동작하지 않는 주소가 일부 나올 수 있으므로, Linux 맨페이지의 방식은 목록을 순회하면서 성공할 때까지 `socket()` 과 `connect()` 를 호출하는 것입니다. (`AI_PASSIVE` 플래그로 서버를 설정하는 중이라면 `connect()` 대신 `bind()` 를 호출합니다.)

마지막으로 연결 리스트를 다 썼다면 `freeaddrinfo()` 를 호출해서 메모리를 해제해야 합니다. (그렇지 않으면 메모리가 새고, 어떤 사람들은 화를 낼 것입니다.)

¹<https://www.iana.org/assignments/port-numbers>

반환값

성공하면 0을 반환하고 오류가 발생하면 0이 아닌 값을 반환합니다. 0이 아닌 값이 반환되면 `gai_strerror()` 함수를 사용해서 반환값 안의 오류 코드를 출력 가능한 형태로 얻을 수 있습니다.

예제

```
// 서버에 연결하는 클라이언트 코드
// 구체적으로는 www.example.com의 80번 포트(http)에 대한 스트림 소켓
// IPv4와 IPv6 어느 쪽이든

int sockfd;
struct addrinfo hints, *servinfo, *p;
int rv;

memset(&hints, 0, sizeof hints);
hints.ai_family = AF_UNSPEC; // IPv6만 강제하려면 AF_INET6 사용
hints.ai_socktype = SOCK_STREAM;

rv = getaddrinfo("www.example.com", "http", &hints, &servinfo);
if (rv != 0) {
    fprintf(stderr, "getaddrinfo: %s\n", gai_strerror(rv));
    exit(1);
}

// 모든 결과를 순회하면서 연결 가능한 첫 번째 것에 연결합니다
for(p = servinfo; p != NULL; p = p->ai_next) {
    if ((sockfd = socket(p->ai_family, p->ai_socktype,
        p->ai_protocol)) == -1) {
        perror("socket");
        continue;
    }

    if (connect(sockfd, p->ai_addr, p->ai_addrlen) == -1) {
        perror("connect");
        close(sockfd);
        continue;
    }

    break; // 여기까지 왔다면 연결에 성공한 것입니다
}

if (p == NULL) {
    // 연결하지 못한 채 목록 끝까지 순회했습니다
    fprintf(stderr, "failed to connect\n");
    exit(2);
}

freeaddrinfo(servinfo); // 이 구조체는 이제 필요 없습니다
```

```
// 연결을 기다리는 서버 코드
// 구체적으로는 이 호스트의 IP에 있는 3490번 포트의 스트림 소켓
```

```
// IPv4와 IPv6 어느 쪽이든.

int sockfd;
struct addrinfo hints, *servinfo, *p;
int rv;

memset(&hints, 0, sizeof hints);
hints.ai_family = AF_UNSPEC; // IPv6만 강제하려면 AF_INET6 사용
hints.ai_socktype = SOCK_STREAM;
hints.ai_flags = AI_PASSIVE; // bind()에 쓸 와일드카드 주소를 요청합니다

if ((rv = getaddrinfo(NULL, "3490", &hints, &servinfo)) != 0) {
    fprintf(stderr, "getaddrinfo: %s\n", gai_strerror(rv));
    exit(1);
}

// 모든 결과를 순회하면서 바인드 가능한 첫 번째 것에 바인드합니다
for(p = servinfo; p != NULL; p = p->ai_next) {
    if ((sockfd = socket(p->ai_family, p->ai_socktype,
        p->ai_protocol)) == -1) {
        perror("socket");
        continue;
    }

    if (bind(sockfd, p->ai_addr, p->ai_addrlen) == -1) {
        close(sockfd);
        perror("bind");
        continue;
    }

    break; // 여기까지 왔다면 바인드에 성공한 것입니다
}

if (p == NULL) {
    // 성공적으로 바인드하지 못한 채 목록 끝까지 순회했습니다
    fprintf(stderr, "failed to bind socket\n");
    exit(2);
}

freeaddrinfo(servinfo); // 이 구조체는 이제 필요 없습니다
```

함께 보기

gethostbyname(), getnameinfo()

9.6 gethostname()

시스템의 이름을 반환합니다

개요

```
#include <unistd.h>

int gethostname(char *name, size_t len);
```

설명

여러분의 시스템에는 이름이 있습니다. 모두 그렇습니다. 이것은 지금까지 이야기한 네트워크다운 것들보다는 약간 더 유닉스다운 일이지만, 그래도 쓸모가 있습니다.

예를 들어 호스트 이름을 얻은 다음 `gethostbyname()` 을 호출해서 IP 주소를 알아낼 수 있습니다.

`name` 매개변수는 호스트 이름을 담은 버퍼를 가리켜야 하고, `len` 은 그 버퍼의 바이트 단위 크기입니다. `gethostname()` 은 버퍼 끝을 넘어 덮어쓰지 않습니다. (오류를 반환하거나, 그냥 쓰기를 멈출 수 있습니다.) 그리고 버퍼 안에 공간이 있다면 문자열을 `NUL` 로 끝냅니다.

반환값

성공하면 0을 반환합니다. 오류가 발생하면 `-1` 을 반환하고 `errno` 를 적절히 설정합니다.

예제

```
char hostname[128];

gethostname(hostname, sizeof hostname);
printf("My hostname: %s\n", hostname);
```

함께 보기

`gethostbyname()`

9.7 `gethostbyname()`, `gethostbyaddr()`

호스트 이름에 대한 IP 주소를 얻거나, 그 반대 작업을 합니다

개요

```
#include <sys/socket.h>
#include <netdb.h>

struct hostent *gethostbyname(const char *name); // 더 이상 권장되지 않습니다!
struct hostent *gethostbyaddr(const char *addr, int len, int type);
```

설명

주의하세요: 이 두 함수는 `getaddrinfo()` 와 `getnameinfo()` 로 대체되었습니다! 특히 `gethostbyname()` 은 IPv6과 잘 동작하지 않습니다.

이 함수들은 호스트 이름과 IP 주소를 서로 변환합니다. 예를 들어 “www.example.com”이 있다면 `gethostbyname()` 을 사용해서 그 IP 주소를 얻고 `struct in_addr` 에 저장할 수 있습니다.

반대로 `struct in_addr` 나 `struct in6_addr` 가 있다면 `gethostbyaddr()` 를 사용해서 호스트 이름을 되찾을 수 있습니다. `gethostbyaddr()` 는 IPv6과 호환되기는 하지만, 대신 더 새롭고 반짝이는 `getnameinfo()` 를 쓰는 편이 좋습니다.

(점과 숫자 형식의 IP 주소 문자열이 있고 그 호스트 이름을 찾고 싶다면, `AI_CANONNAME` 플래그와 함께 `getaddrinfo()` 를 쓰는 편이 낫습니다.)

`gethostbyname()` 은 “www.yahoo.com” 같은 문자열을 받아 IP 주소를 포함한 많은 정보가 들어 있는 `struct hostent` 를 반환합니다. (다른 정보로는 공식 호스트 이름, 별칭 목록, 주소 타입, 주소 길이, 주소 목록이 있습니다. 사용법을 보고 나면 우리의 구체적인 목적에는 꽤 쉽게 쓸 수 있는 범용 구조체입니다.)

`gethostbyaddr()` 는 `struct in_addr` 나 `struct in6_addr` 를 받아 그에 대응하는 호스트 이름이 있다면 가져옵니다. 그러니까 일종의 `gethostbyname()` 반대입니다. 매개변수를 보자면, `addr` 은 `char*` 이지만 실제로는 `struct in_addr` 에 대한 포인터를 넘기고 싶을 것입니다. `len` 은 `sizeof(struct in_addr)` 이어야 하고, `type` 은 `AF_INET` 이어야 합니다.

그러면 반환되는 `struct hostent` 는 무엇일까요? 여기에는 해당 호스트에 대한 정보를 담은 여러 필드가 있습니다.

필드	설명
<code>char *h_name</code>	실제 정규 호스트 이름입니다.
<code>char **h_aliases</code>	이름처럼 접근할 수 있는 별칭 목록입니다. 마지막 요소는 <code>NULL</code> 입니다.
<code>int h_addrtype</code>	주요 주소 타입입니다. 우리의 목적에서는 사실상 <code>AF_INET</code> 이어야 합니다.
<code>int length</code>	주소의 바이트 단위 길이입니다. IP 버전 4 주소에서는 4입니다.
<code>char **h_addr_list</code>	이 호스트의 IP 주소 목록입니다. <code>char**</code> 이지만 실제로는 <code>struct in_addr*</code> 배열이 변장한 것입니다. 마지막 배열 요소는 <code>NULL</code> 입니다.
<code>h_addr</code>	흔히 정의되어 있는 <code>h_addr_list[0]</code> 의 별칭입니다. 이 호스트의 아무 IP 주소나 원한다면(네, 둘 이상 있을 수 있습니다) 이 필드를 쓰면 됩니다.

반환값

성공하면 결과 `struct hostent` 에 대한 포인터를 반환하고, 오류가 발생하면 `NULL` 을 반환합니다.

오류 보고에 보통 쓰는 `perror()` 같은 것들 대신, 이 함수들은 `h_errno` 변수에 별도의 결과를 둡니다. 이것은 `herror()` 나 `hstrerror()` 함수를 사용해서 출력할 수 있습니다. 이 함수들은 여러분에게 익숙한 고전적인 `errno`, `perror()`, `strerror()` 함수와 비슷하게 동작합니다.

예제

```
// 이것은 호스트 이름을 얻는 구식 방법입니다
// 대신 getaddrinfo()를 쓰세요!

#include <stdio.h>
#include <errno.h>
#include <netdb.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

int main(int argc, char *argv[])
{
    int i;
    struct hostent *he;
    struct in_addr **addr_list;

    if (argc != 2) {
        fprintf(stderr, "usage: ghbn hostname\n");
        return 1;
    }

    if ((he = gethostbyname(argv[1])) == NULL) { // 호스트 정보 얻기
        perror("gethostbyname");
        return 2;
    }

    // 이 호스트에 대한 정보를 출력합니다:
    printf("Official name is: %s\n", he->h_name);
    printf("    IP addresses: ");
    addr_list = (struct in_addr **)he->h_addr_list;
    for(i = 0; addr_list[i] != NULL; i++) {
        printf("%s ", inet_ntoa(*addr_list[i]));
    }
    printf("\n");

    return 0;
}
```

```
// 이것은 대체되었습니다
// 대신 getnameinfo()를 쓰세요!

struct hostent *he;
struct in_addr ipv4addr;
struct in6_addr ipv6addr;

inet_pton(AF_INET, "192.0.2.34", &ipv4addr);
he = gethostbyaddr(&ipv4addr, sizeof ipv4addr, AF_INET);
printf("Host name: %s\n", he->h_name);
```

```
inet_pton(AF_INET6, "2001:db8:63b3:1::beef", &ipv6addr);
he = gethostbyaddr(&ipv6addr, sizeof ipv6addr, AF_INET6);
printf("Host name: %s\n", he->h_name);
```

함께 보기

`getaddrinfo()`, `getnameinfo()`, `gethostname()`, `errno`, `perror()`, `strerror()`, `struct in_addr`

9.8 getnameinfo()

주어진 `struct sockaddr` 에 대한 호스트 이름과 서비스 이름 정보를 찾습니다.

개요

```
#include <sys/socket.h>
#include <netdb.h>

int getnameinfo(const struct sockaddr *sa, socklen_t salen,
                char *host, size_t hostlen,
                char *serv, size_t servlen, int flags);
```

설명

이 함수는 `getaddrinfo()` 의 반대입니다. 다시 말해 이미 채워진 `struct sockaddr` 를 받아 그에 대해 이름과 서비스 이름 조회를 수행합니다. 이 함수는 오래된 `gethostbyaddr()` 와 `getservbyport()` 함수를 대체합니다.

`sa` 매개변수에는 `struct sockaddr` 에 대한 포인터를 넘겨야 합니다. 실제로는 아마 형변환한 `struct sockaddr_in` 이나 `struct sockaddr_in6` 일 것입니다. 그리고 `salen` 에는 그 `struct` 의 길이를 넘깁니다.

결과 호스트 이름과 서비스 이름은 `host` 와 `serv` 매개변수가 가리키는 영역에 쓰입니다. 물론 `hostlen` 과 `servlen` 으로 이 버퍼들의 최대 길이를 지정해야 합니다.

마지막으로 넘길 수 있는 플래그가 몇 가지 있는데, 여기 괜찮은 것 몇 개가 있습니다. `NI_NOFQDN` 은 `host` 에 전체 도메인 이름이 아니라 호스트 이름만 들어가게 합니다. `NI_NAMEREQD` 는 DNS 조회로 이름을 찾을 수 없을 때 함수가 실패하게 만듭니다. (이 플래그를 지정하지 않았고 이름을 찾을 수 없다면 `getnameinfo()` 는 대신 IP 주소의 문자열 버전을 `host` 에 넣습니다.)

늘 그렇듯 전체 이야기는 여러분의 로컬 맨페이지를 확인하세요.

반환값

성공하면 0을 반환하고 오류가 발생하면 0이 아닌 값을 반환합니다. 반환값이 0이 아니라면 `gai_strerror()` 에 넘겨 사람이 읽을 수 있는 문자열을 얻을 수 있습니다. 더 자세한 내용은 `getaddrinfo()` 를 보세요.

예제

```
struct sockaddr_in6 sa; // 원한다면 IPv4일 수도 있습니다
char host[1024];
char service[20];

// sa에 호스트와 포트에 대한 좋은 정보가 가득하다고 가정합니다...

getnameinfo((const struct sockaddr *)&sa, sizeof sa, host, sizeof host, service,
            sizeof service, 0);

printf("  host: %s\n", host);    // 예: "www.example.com"
printf("service: %s\n", service); // 예: "http"
```

함께 보기

getaddrinfo(), gethostbyaddr()

9.9 getpeername()

연결의 원격 쪽에 대한 주소 정보를 반환합니다

개요

```
#include <sys/socket.h>

int getpeername(int s, struct sockaddr *addr, socklen_t *len);
```

설명

원격 연결을 `accept()` 했거나 서버에 `connect()` 했다면, 여러분에게는 이제 `_피어_`라고 부르는 것이 생깁니다. 피어는 간단히 말해 여러분이 연결된 컴퓨터이며, IP 주소와 포트로 식별됩니다. 그래서...

`getpeername()` 은 여러분이 연결된 피어의 주소 정보를 `addr` 이 가리키는 주소 구조체에 채웁니다.

왜 “name”이라고 부를까요? 음, 이 안내서에서 사용하는 인터넷 소켓뿐 아니라 여러 종류의 소켓이 있기 때문에, “name”은 모든 경우를 포괄하는 괜찮은 일반 용어였습니다. 하지만 우리의 경우 피어의 “name”은 IP 주소와 포트입니다.

이 함수는 결과 주소의 크기를 `len` 에 반환하지만, 호출 전에는 `len` 에 `addr` 의 크기를 미리 넣어 두어야 합니다.

반환값

성공하면 0을 반환합니다. 오류가 발생하면 `-1` 을 반환하고 `errno` 를 적절히 설정합니다.

예제

```
// s가 연결된 소켓이라고 가정합니다

socklen_t len;
struct sockaddr_storage addr;
char ipstr[INET6_ADDRSTRLEN];
int port;

len = sizeof addr;
getpeername(s, (struct sockaddr*)&addr, &len);

// IPv4와 IPv6을 모두 처리합니다:
if (addr.ss_family == AF_INET) {
    struct sockaddr_in *s = (struct sockaddr_in *)&addr;
    port = ntohs(s->sin_port);
    inet_ntop(AF_INET, &s->sin_addr, ipstr, sizeof ipstr);
} else { // AF_INET6
    struct sockaddr_in6 *s = (struct sockaddr_in6 *)&addr;
    port = ntohs(s->sin6_port);
    inet_ntop(AF_INET6, &s->sin6_addr, ipstr, sizeof ipstr);
}

printf("Peer IP address: %s\n", ipstr);
printf("Peer port      : %d\n", port);
```

함께 보기

gethostname(), gethostbyname(), gethostbyaddr()

9.10 errno

마지막 시스템 호출의 오류 코드를 담습니다

개요

```
#include <errno.h>

int errno;
```

설명

이 변수는 많은 시스템 호출의 오류 정보를 담습니다. 기억하시겠지만 `socket()` 이나 `listen()` 같은 함수는 오류가 발생하면 `-1` 을 반환하고, 정확히 어떤 오류가 일어났는지 알려주기 위해 `errno` 값을 설정합니다.

헤더 파일 `errno.h` 에는 `EADDRINUSE`, `EPIPE`, `ECONNREFUSED` 같은 오류에 대한 상수 심볼 이름이 많이 나열되어 있습니다. 여러분의 로컬 맨페이지는 어떤 코드가 오류로 반환될 수 있는지 알려줄 것이고, 여러분은 런타임에 이 값들을 사용해서 서로 다른 오류를 서로 다른 방식으로 처리할 수 있습니다.

또는 더 흔하게는 `perror()` 나 `strerror()` 를 호출해서 사람이 읽을 수 있는 오류 표현을 얻을 수 있습니다.

멀티스레딩 애호가들을 위해 한 가지 적자면, 대부분의 시스템에서 `errno` 는 스레드 안전한 방식으로 정의됩니다. (다시 말해 실제로는 전역 변수가 아니지만, 단일 스레드 환경에서는 전역 변수처럼 동작합니다.)

반환값

이 변수의 값은 가장 최근에 일어난 오류입니다. 마지막 동작이 성공했다면 "성공"을 나타내는 코드일 수도 있습니다.

예제

```
s = socket(PF_INET, SOCK_STREAM, 0);
if (s == -1) {
    perror("socket"); // 또는 strerror()를 사용
}

tryagain:
if (select(n, &readfds, NULL, NULL, NULL) == -1) {
    // 오류가 발생했습니다!!

    // 단순히 인터럽트된 것뿐이라면 select() 호출을 다시 시작합니다:
    if (errno == EINTR) goto tryagain; // 으아아! goto!!!

    // 그렇지 않다면 더 심각한 오류입니다:
    perror("select");
    exit(1);
}
```

함께 보기

`perror()`, `strerror()`

9.11 fcntl()

소켓 설명자를 제어합니다

개요

```
#include <unistd.h>
#include <fcntl.h>

int fcntl(int s, int cmd, long arg);
```

설명

이 함수는 보통 파일 잠금이나 다른 파일 중심 작업에 사용되지만, 가끔 보거나 사용하게 될 소켓 관련 기능도 몇 가지 가지고 있습니다.

`s` 매개변수는 작업하려는 소켓 설명자입니다. `cmd` 는 `F_SETFL` 로 설정해야 하고, `arg` 에는 아래 파일 상태 플래그를 지정할 수 있습니다. (말했듯이 `fcntl()` 에는 제가 여기에서 드러낸 것보다 더 많은 기능이 있지만, 저는 소켓 중심으로 이야기하려고 합니다.)

arg	설명
<code>O_NONBLOCK</code>	소켓을 논블로킹으로 설정합니다. 더 자세한 내용은 블로킹 절을 보세요.
<code>O_ASYNC</code>	소켓이 비동기 I/O를 하도록 설정합니다. 소켓에서 <code>recv()</code> 할 데이터가 준비되면 <code>SIGIO</code> 신호가 발생합니다. 보기 드문 방식이고 이 안내서의 범위를 벗어납니다. 또한 특정 시스템에서만 사용할 수 있는 것으로 생각합니다.

반환값

성공하면 0을 반환합니다. 오류가 발생하면 `-1` 을 반환하고 `errno` 를 적절히 설정합니다.

`fcntl()` 시스템 호출은 실제로 사용법에 따라 서로 다른 반환값을 가지지만, 소켓과 관련되지 않은 내용이므로 여기에서는 다루지 않았습니다. 더 자세한 정보는 여러분의 로컬 `fcntl()` 맨페이지를 보세요.

예제

```
int s = socket(PF_INET, SOCK_STREAM, 0);

fcntl(s, F_SETFL, O_NONBLOCK); // 논블로킹으로 설정
fcntl(s, F_SETFL, O_ASYNC);    // 비동기 I/O로 설정
```

함께 보기

Blocking, `send()`

9.12 `htons()`, `htonl()`, `ntohs()`, `ntohl()`

여러 바이트 정수 타입을 호스트 바이트 순서와 네트워크 바이트 순서 사이에서 변환합니다

개요

```
#include <netinet/in.h>

uint32_t htonl(uint32_t hostlong);
uint16_t htons(uint16_t hostshort);
uint32_t ntohl(uint32_t netlong);
uint16_t ntohs(uint16_t netshort);
```

설명

여러분을 정말로 불행하게 만들기 위해, 서로 다른 컴퓨터들은 여러 바이트 정수(즉, `char` 보다 큰 정수)를 내부적으로 서로 다른 바이트 순서로 사용합니다. 그 결과 Intel 컴퓨터에서 Mac으로(그 Mac들도 Intel 컴퓨터가 되기 전을 말합니다) 2바이트 `short int` 를 `send()` 하면, 한 컴퓨터가 숫자 1 이라고 생각하는 것을 다른 컴퓨터는 256 이라고 생각할 수 있고 그 반대도 가능합니다.

이 문제를 피하는 방법은 모두가 차이를 내려놓고 Motorola와 IBM이 옳았으며 Intel이 이상한 방식으로 했다는 데 동의한 뒤, 내보내기 전에 모두 바이트 순서를 “빅엔디언”으로 변환하는 것입니다. Intel은 “리틀엔디언” 시스템이므로, 우리가 선호하는 바이트 순서를 “네트워크 바이트 순서”라고 부르는 편이 훨씬 정치적으로 올바릅니다. 그래서 이 함수들은 여러분의 네이티브 바이트 순서에서 네트워크 바이트 순서로, 그리고 그 반대로 변환합니다.

(이 말은 Intel에서는 이 함수들이 모든 바이트를 뒤집고, PowerPC에서는 바이트가 이미 네트워크 바이트 순서이므로 아무 일도 하지 않는다는 뜻입니다. 하지만 누군가가 Intel 시스템에서 빌드해도 올바르게 동작하기를 원할 수 있으므로, 여러분은 코드에서 늘 이 함수들을 사용해야 합니다.)

여기에서 다루는 타입은 32비트(4바이트, 아마 `int`)와 16비트(2바이트, 거의 확실히 `short`) 숫자라는 점에 주목하세요.

여러 시스템에는 64비트 변종이 있습니다. 사용할 수 있다면 `<endian.h>` 안의 `htobe64()`² 함수와 그 친척들을 확인해 보세요. (MacOS에는 없는 것 같습니다.) 그리고 GCC에는 128비트까지 다루는 바이트 스와핑 내장 함수³도 있습니다. 아니면 직접 만들 수도 있습니다⁴. 단, 실제 스왑은 리틀엔디언 시스템에서만 하세요!

어쨌든 이 함수들이 동작하는 방식은 이렇습니다. 먼저 호스트(여러분 시스템의) 바이트 순서에서 변환하는지, 아니면 네트워크 바이트 순서에서 변환하는지를 결정합니다. “host”라면 호출할 함수의 첫 글자는 “h”입니다. 그렇지 않으면 “network”의 “n”입니다. 함수 이름의 가운데는 언제나 “to”입니다. 하나에서 다른 하나로 변환하기 때문입니다. 그리고 끝에서 두 번째 글자는 무엇으로 변환하는지를 보여줍니다. 마지막 글자는 데이터의 크기로, `short`는 “s”, `long`은 “l”입니다. 그래서:

함수	설명
<code>htons()</code>	h ost to n etwork s hort
<code>htonl()</code>	h ost to n etwork l ong
<code>ntohs()</code>	n etwork to h ost s hort
<code>ntohl()</code>	n etwork to h ost l ong

반환값

각 함수는 변환된 값을 반환합니다.

예제

```
uint32_t some_long = 10;
uint16_t some_short = 20;

uint32_t network_byte_order;

// 변환하고 전송
```

²<https://man.archlinux.org/man/htobe64>

³<https://gcc.gnu.org/onlinedocs/gcc/Byte-Swapping-Builtins.html>

⁴<https://beej.us/guide/bgnet/source/examples/htonl.c>

```
network_byte_order = htonl(some_long);
send(s, &network_byte_order, sizeof(uint32_t), 0);

some_short == ntohs(htons(some_short)); // 이 표현식은 참입니다
```

9.13 inet_ntoa(), inet_aton(), inet_addr

IP 주소를 점과 숫자로 된 문자열에서 `struct in_addr` 로, 또는 그 반대로 변환합니다

개요

```
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

// 이것들은 모두 구식입니다!
// 대신 inet_pton()이나 inet_ntop()을 쓰세요!

char *inet_ntoa(struct in_addr in);
int inet_aton(const char *cp, struct in_addr *inp);
in_addr_t inet_addr(const char *cp);
```

설명

이 함수들은 IPv6을 처리하지 못하기 때문에 구식입니다! 대신 `inet_ntop()` 이나 `inet_pton()` 을 쓰세요! 여기 포함한 이유는 여전히 현장에서 발견될 수 있기 때문입니다.

이 함수들은 모두 `struct in_addr` (아마 여러분의 `struct sockaddr_in` 의 일부일 것입니다)에서 점과 숫자 형식의 문자열(예: "192.168.5.10")로, 또는 그 반대로 변환합니다. 명령줄 같은 곳에서 IP 주소를 넘겨받았다면, `connect()` 등에 쓸 `struct in_addr` 를 얻는 가장 쉬운 방법입니다. 더 강력한 기능이 필요하다면 `gethostbyname()` 같은 DNS 함수들을 시도하거나, 여러분 나라에서 _쿠데타_를 시도해 보세요.

`inet_ntoa()` 함수는 `struct in_addr` 안의 네트워크 주소를 점과 숫자 형식의 문자열로 변환합니다. "ntoa"의 "n"은 network를 뜻하고, "a"는 역사적인 이유로 ASCII를 뜻합니다. (그러니까 "Network To ASCII"입니다. "toa" 접미사에는 C 라이브러리에 `atoi()` 라는 비슷한 친구가 있는데, 이것은 ASCII 문자열을 정수로 변환합니다.)

`inet_aton()` 함수는 그 반대로, 점과 숫자 문자열을 이진 IPv4 주소 형식으로 변환해 두 번째 인수로 전달한 `struct in_addr` 에 저장합니다.

마지막으로 `inet_addr()` 함수는 기본적으로 `inet_aton()` 과 같은 일을 하는 더 오래된 함수입니다. 이론적으로는 구식이지만, 많이 보게 될 것이고 사용한다고 경찰이 잡으려 오지는 않을 것입니다.

반환값

`inet_aton()` 은 주소가 유효하면 0이 아닌 값을 반환하고, 주소가 유효하지 않으면 0을 반환합니다.

`inet_ntoa()` 는 점과 숫자 문자열을 정적 버퍼에 담아 반환합니다. 이 버퍼는 함수를 호출할 때마다 덮어씌웁니다.

`inet_addr()` 는 주소를 `in_addr_t` 로 반환하고, 오류가 발생하면 `-1` 을 반환합니다. (그 값은 유효한 IP 주소인 " 255.255.255.255 " 문자열을 변환하려고 했을 때와 같은 결과입니다. 그래서 `inet_aton()` 이 더 낫습니다.)

예제

```
struct sockaddr_in antelope;
char *some_addr;

inet_aton("10.0.0.1", &antelope.sin_addr); // antelope에 IP 저장

some_addr = inet_ntoa(antelope.sin_addr); // IP 반환
printf("%s\n", some_addr); // "10.0.0.1" 출력

// 그리고 이 호출은 위의 inet_aton() 호출과 같습니다:
antelope.sin_addr.s_addr = inet_addr("10.0.0.1");
```

함께 보기

`inet_ntop()`, `inet_pton()`, `gethostbyname()`, `gethostbyaddr()`

9.14 `inet_ntop()`, `inet_pton()`

IP 주소를 사람이 읽을 수 있는 형식으로, 또는 그 반대로 변환합니다.

개요

```
#include <arpa/inet.h>

const char *inet_ntop(int af, const void *src,
                      char *dst, socklen_t size);

int inet_pton(int af, const char *src, void *dst);
```

설명

이 함수들은 사람이 읽을 수 있는 IP 주소를 다루고, 그것을 여러 함수와 시스템 호출에서 쓸 수 있도록 이진 표현으로 변환합니다. "n"은 "network"를, "p"는 "presentation"을 뜻합니다. 또는 "text presentation"입니다. 하지만 "printable"이라고 생각해도 됩니다. "ntop"은 "network to printable"입니다. 보이지요?

가끔은 IP 주소를 볼 때 이진 숫자 더미를 보고 싶지 않습니다. `192.0.2.180` 이나 `2001:db8:8714:3a90::12` 처럼 보기 좋게 출력 가능한 형식으로 보고 싶습니다. 그런 경우를 위한 것이 `inet_ntop()` 입니다.

`inet_ntop()` 은 `af` 매개변수로 주소 계열(`AF_INET` 또는 `AF_INET6`)을 받습니다. `src` 매개변수는 문자열로 변환하려는 주소가 들어 있는 `struct in_addr` 또는 `struct in6_addr` 에 대한 포인터여야 합니다. 마지막으로 `dst` 와 `size` 는 목적지 문자열에 대한 포인터와 그 문자열의 최대 길이입니다.

`dst` 문자열의 최대 길이는 얼마여야 할까요? IPv4와 IPv6 주소의 최대 길이는 얼마일까요? 다행히 여러분을 도와줄 매크로가 몇 개 있습니다. 최대 길이는 `INET_ADDRSTRLEN` 과 `INET6_ADDRSTRLEN` 입니다.

다른 경우에는 읽을 수 있는 형식의 IP 주소 문자열이 있고, 그것을 `struct sockaddr_in` 또는 `struct sockaddr_in6` 안에 넣고 싶을 수 있습니다. 그런 경우에는 반대 함수인 `inet_pton()` 이 여러분이 찾는 것입니다.

`inet_pton()` 도 `af` 매개변수로 주소 계열(`AF_INET` 또는 `AF_INET6`)을 받습니다. `src` 매개변수는 출력 가능한 형식의 IP 주소가 담긴 문자열에 대한 포인터입니다. 마지막으로 `dst` 매개변수는 결과를 저장할 곳을 가리키며, 아마 `struct in_addr` 또는 `struct in6_addr` 일 것입니다.

이 함수들은 DNS 조회를 하지 않습니다. 그 작업에는 `getaddrinfo()` 가 필요합니다.

반환값

`inet_ntop()` 은 성공하면 `dst` 매개변수를 반환합니다. 실패하면 `NULL` 을 반환하고 `errno` 를 설정합니다.

`inet_pton()` 은 성공하면 `1` 을 반환합니다. 오류가 있으면 `-1` 을 반환하고(`errno` 가 설정됩니다), 입력이 유효한 IP 주소가 아니면 `0` 을 반환합니다.

예제

```
// inet_ntop()과 inet_pton()의 IPv4 데모

struct sockaddr_in sa;
char str[INET_ADDRSTRLEN];

// 이 IP 주소를 sa에 저장합니다:
inet_pton(AF_INET, "192.0.2.33", &(sa.sin_addr));

// 이제 다시 꺼내 출력합니다
inet_ntop(AF_INET, &(sa.sin_addr), str, INET_ADDRSTRLEN);

printf("%s\n", str); // "192.0.2.33" 출력
```

```
// inet_ntop()과 inet_pton()의 IPv6 데모
// (기본적으로 6이 여기저기 붙는 것 말고는 같습니다)

struct sockaddr_in6 sa;
char str[INET6_ADDRSTRLEN];

// 이 IP 주소를 sa에 저장합니다:
inet_pton(AF_INET6, "2001:db8:8714:3a90::12", &(sa.sin6_addr));

// 이제 다시 꺼내 출력합니다
inet_ntop(AF_INET6, &(sa.sin6_addr), str, INET6_ADDRSTRLEN);
```

```
printf("%s\n", str); // "2001:db8:8714:3a90::12" 출력
```

```
// 사용할 수 있는 도우미 함수:

// struct sockaddr 주소를 문자열로 변환합니다. IPv4와 IPv6:

char *get_ip_str(const struct sockaddr *sa, char *s, size_t maxlen)
{
    switch(sa->sa_family) {
        case AF_INET:
            inet_ntop(AF_INET, &(((struct sockaddr_in *)sa)->sin_addr),
                      s, maxlen);
            break;

        case AF_INET6:
            inet_ntop(AF_INET6, &(((struct sockaddr_in6 *)sa)->sin6_addr),
                      s, maxlen);
            break;

        default:
            strncpy(s, "Unknown AF", maxlen);
            return NULL;
    }

    return s;
}
```

함께 보기

getaddrinfo()

9.15 listen()

소켓에게 들어오는 연결을 리스닝하라고 알려줍니다

개요

```
#include <sys/socket.h>

int listen(int s, int backlog);
```

설명

여러분은 (socket() 시스템 호출로 만든) 소켓 설명자에게 들어오는 연결을 리스닝하라고 말할 수 있습니다. 여러분, 이것이 서버와 클라이언트를 구분하는 것입니다.

`backlog` 매개변수는 사용 중인 시스템에 따라 몇 가지 다른 뜻을 가질 수 있지만, 대략적으로는 커널이 새 연결을 거부하기 전에 보류 중인 연결을 몇 개까지 둘 수 있는지를 의미합니다. 그러므로 새 연결이 들어오면 `backlog` 가 꽉 차지 않도록 빠르게 `accept()` 해야 합니다. 10 정도로 설정해 보고, 부하가 클 때 클라이언트가 "Connection refused"를 받기 시작하면 더 크게 설정하세요.

`listen()` 을 호출하기 전에 서버는 `bind()` 를 호출해서 자신을 특정 포트 번호에 붙여야 합니다. 그 포트 번호(서버의 IP 주소에 있는)가 클라이언트가 연결할 대상이 됩니다.

반환값

성공하면 0을 반환합니다. 오류가 발생하면 `-1` 을 반환하고 `errno` 를 적절히 설정합니다.

예제

```
struct addrinfo hints, *res;
int sockfd;

// 먼저 getaddrinfo()로 주소 구조체 목록을 가져옵니다:

memset(&hints, 0, sizeof hints);
hints.ai_family = AF_UNSPEC; // IPv4와 IPv6 어느 쪽이든 사용
hints.ai_socktype = SOCK_STREAM;
hints.ai_flags = AI_PASSIVE; // bind()에 쓸 와일드카드 주소를 요청합니다

getaddrinfo(NULL, "3490", &hints, &res);

// 소켓을 만듭니다:

sockfd = socket(res->ai_family, res->ai_socktype, res->ai_protocol);

// getaddrinfo()에 넘긴 포트에 바인드합니다:

bind(sockfd, res->ai_addr, res->ai_addrlen);

listen(sockfd, 10); // sockfd를 서버 소켓으로 설정합니다

// 그런 다음 이 아래 어딘가에 accept() 루프를 둡니다
```

함께 보기

`accept()`, `bind()`, `socket()`

9.16 perror(), strerror()

오류를 사람이 읽을 수 있는 문자열로 출력합니다

개요

```
#include <stdio.h>
#include <string.h> // strerror()용

void perror(const char *s);
char *strerror(int errnum);
```

설명

많은 함수가 오류가 발생하면 -1 을 반환하고 `errno` 변수의 값을 어떤 숫자로 설정하므로, 그것을 여러분이 이해할 수 있는 형태로 쉽게 출력할 수 있다면 분명 좋을 것입니다.

고맙게도 `perror()` 가 그 일을 합니다. 오류 앞에 더 많은 설명을 출력하고 싶다면 `s` 매개변수가 그 문자열을 가리키게 하면 됩니다. (또는 `s` 를 `NULL` 로 두면 아무 추가 내용도 출력되지 않습니다.)

간단히 말해 이 함수는 `ECONNRESET` 같은 `errno` 값을 받아 "Connection reset by peer." 처럼 보기 좋게 출력합니다.

`strerror()` 함수는 `perror()` 와 매우 비슷하지만, 주어진 값(보통 `errno` 변수를 넘깁니다)에 대한 오류 메시지 문자열의 포인터를 반환한다는 점이 다릅니다.

반환값

`strerror()` 는 오류 메시지 문자열에 대한 포인터를 반환합니다.

예제

```
int s;

s = socket(PF_INET, SOCK_STREAM, 0);

if (s == -1) { // 어떤 오류가 발생했습니다
    // "socket error: " + 오류 메시지를 출력합니다:
    perror("socket error");
}

// 비슷하게:
if (listen(s, 10) == -1) {
    // 이것은 "an error: " + errno의 오류 메시지를 출력합니다:
    printf("an error: %s\n", strerror(errno));
}
```

함께 보기

`errno`

9.17 poll()

여러 소켓의 이벤트를 동시에 검사합니다

개요

```
#include <sys/poll.h>

int poll(struct pollfd *ufds, unsigned int nfds, int timeout);
```

설명

이 함수는 `select()` 와 매우 비슷합니다. 둘 다 파일 설명자 집합을 감시하면서 `recv()` 할 준비가 된 수신 데이터, 데이터를 `send()` 할 준비가 된 소켓, `recv()` 할 준비가 된 out-of-band 데이터, 오류 같은 이벤트를 확인합니다.

기본 개념은 `ufds` 에 `nfds` 개의 `struct pollfd` 배열을 넘기고, 밀리초 단위의 타임아웃도 함께 넘기는 것입니다. (1초는 1000밀리초입니다.) 영원히 기다리고 싶다면 `timeout` 은 음수일 수 있습니다. 타임아웃까지 어떤 소켓 설명자에서도 이벤트가 일어나지 않으면 `poll()` 은 반환합니다.

`struct pollfd` 배열의 각 요소는 소켓 설명자 하나를 나타내며, 아래 필드들을 포함합니다:

```
struct pollfd {
    int fd;           // 소켓 설명자
    short events;     // 관심 있는 이벤트의 비트맵
    short revents;    // 반환 시점에 발생한 이벤트의 비트맵
};
```

`poll()` 을 호출하기 전에 `fd` 에 소켓 설명자를 넣으세요. (`fd` 를 음수로 설정하면 이 `struct pollfd` 는 무시되고 `revents` 필드는 0으로 설정됩니다.) 그런 다음 아래 매크로들을 비트 OR해서 `events` 필드를 구성합니다:

매크로	설명
<code>POLLIN</code>	이 소켓에서 <code>recv()</code> 할 데이터가 준비되면 알려줍니다.
<code>POLLOUT</code>	이 소켓으로 대기하지 않고 데이터를 <code>send()</code> 할 수 있으면 알려줍니다.
<code>POLLPRI</code>	이 소켓에서 <code>recv()</code> 할 out-of-band 데이터가 준비되면 알려줍니다.

`poll()` 호출이 반환되면 `revents` 필드는 위 필드들을 비트 OR한 값으로 구성되어, 어떤 설명자에서 실제로 이벤트가 발생했는지 알려줍니다. 추가로 아래 필드들이 나타날 수도 있습니다:

매크로	설명
<code>POLLERR</code>	이 소켓에서 오류가 발생했습니다.
<code>POLLHUP</code>	연결의 원격 쪽이 끊었습니다.
<code>POLLNVAL</code>	소켓 설명자 <code>fd</code> 에 문제가 있습니다. 초기화되지 않았을지도 모릅니다.

반환값

이벤트가 발생한 `ufds` 배열 요소의 수를 반환합니다. 타임아웃이 발생했다면 0일 수 있습니다. 오류가 발생하면 -1을 반환하고 `errno`를 적절히 설정합니다.

예제

```
int s1, s2;
int rv;
char buf1[256], buf2[256];
struct pollfd ufds[2];

s1 = socket(PF_INET, SOCK_STREAM, 0);
s2 = socket(PF_INET, SOCK_STREAM, 0);

// 이 시점에서 둘 다 서버에 연결되었다고 가정합니다
//connect(s1, ...)...
//connect(s2, ...)...

// 파일 설명자 배열을 설정합니다.
//
// 이 예제에서는 일반 데이터나 out-of-band 데이터가
// recv()로 수신될 준비가 되었는지 알고 싶습니다...

ufds[0].fd = s1;
ufds[0].events = POLLIN | POLLPRI; // 일반 데이터나 out-of-band 확인

ufds[1].fd = s2;
ufds[1].events = POLLIN; // 일반 데이터만 확인

// 소켓 이벤트를 기다립니다. 타임아웃은 3.5초
rv = poll(ufds, 2, 3500);

if (rv == -1) {
    perror("poll"); // poll()에서 오류 발생
} else if (rv == 0) {
    printf("Timeout occurred! No data after 3.5 seconds.\n");
} else {
    // s1의 이벤트 확인:
    if (ufds[0].revents & POLLIN) {
        recv(s1, buf1, sizeof buf1, 0); // 일반 데이터 수신
    }
    if (ufds[0].revents & POLLPRI) {
        recv(s1, buf1, sizeof buf1, MSG_OOB); // out-of-band 데이터
    }

    // s2의 이벤트 확인:
    if (ufds[1].revents & POLLIN) {
        recv(s2, buf2, sizeof buf2, 0);
    }
}
```

함께 보기

`select()`

9.18 `recv()`, `recvfrom()`

소켓에서 데이터를 받습니다

개요

```
#include <sys/types.h>
#include <sys/socket.h>

ssize_t recv(int s, void *buf, size_t len, int flags);
ssize_t recvfrom(int s, void *buf, size_t len, int flags,
                 struct sockaddr *from, socklen_t *fromlen);
```

설명

소켓을 만들고 연결했다면, `recv()` (TCP `SOCK_STREAM` 소켓용)와 `recvfrom()` (UDP `SOCK_DGRAM` 소켓용)을 사용해서 원격 쪽에서 들어오는 데이터를 읽을 수 있습니다.

두 함수 모두 소켓 설명자 `s`, 버퍼에 대한 포인터 `buf`, 버퍼의 바이트 단위 크기 `len`, 함수 동작을 제어하는 `flags` 집합을 받습니다.

추가로 `recvfrom()` 은 데이터가 어디에서 왔는지 알려줄 `struct sockaddr*` 인 `from` 을 받습니다. `fromlen` 에는 호출 전에 `from` 이 가리키는 주소 구조체의 크기를 넣어야 하며, 호출 뒤에는 실제 송신자 주소 길이가 저장됩니다.

그러면 이 함수에 넘길 수 있는 놀라운 플래그에는 무엇이 있을까요? 일부는 아래와 같습니다. 더 자세한 정보와 여러분 시스템에서 실제로 지원하는 값은 로컬 맨페이지를 확인하세요. 이 값들을 비트 OR하거나, 평범한 기본 `recv()` 를 원한다면 `flags` 를 그냥 `0` 으로 설정하면 됩니다.

매크로	설명
<code>MSG_OOB</code>	Out-of-band 데이터를 받습니다. <code>send()</code> 에서 <code>MSG_OOB</code> 플래그로 여러분에게 보낸 데이터를 얻는 방법입니다. 수신 측에서는 긴급 데이터가 있다는 사실을 알려주는 <code>SIGURG</code> 신호를 받게 됩니다. 그 신호 처리기 안에서 이 <code>MSG_OOB</code> 플래그와 함께 <code>recv()</code> 를 호출할 수 있습니다.
<code>MSG_PEEK</code>	<code>recv()</code> 를 “그냥 보는 척” 호출하고 싶다면 이 플래그를 사용할 수 있습니다. 이 플래그는 “진짜로” <code>recv()</code> 를 호출할 때(즉 <code>MSG_PEEK</code> 플래그 없이 호출할 때) 버퍼에서 기다리고 있을 내용을 알려줍니다. 다음 <code>recv()</code> 호출을 살짝 미리 보는 것과 같습니다.
<code>MSG_WAITALL</code>	<code>len</code> 매개변수로 지정한 데이터가 모두 수신될 때까지 <code>recv()</code> 가 반환하지 않게 합니다. 하지만 신호가 호출을 인터럽트하거나, 오류가 발생하거나, 원격 쪽이 연결을 닫는 등 극단적인 상황에서는 여러분의 바람을 무시할 것입니다. 너무 화내지는 마세요.

`recv()` 를 호출하면 읽을 데이터가 생길 때까지 블록됩니다. 블록되지 않기를 원한다면 소켓을 논블로킹으로 설정하거나, `recv()` 또는 `recvfrom()` 을 호출하기 전에 `select()` 나 `poll()` 로 들어오는 데이터가 있는지 확인하세요.

반환값

실제로 받은 바이트 수를 반환합니다. 이 값은 `len` 매개변수로 요청한 것보다 작을 수 있습니다. 오류가 발생하면 `-1` 을 반환하고 `errno` 를 적절히 설정합니다.

원격 쪽이 연결을 닫았다면 `recv()` 는 `0` 을 반환합니다. 이것이 원격 쪽이 연결을 닫았는지 판단하는 일반적인 방법입니다. 평범함도 좋은 겁니다, 반항아 여러분!

예제

```
// 스트림 소켓과 recv()

struct addrinfo hints, *res;
int sockfd;
char buf[512];
int byte_count;

// 호스트 정보를 얻고, 소켓을 만들고, 연결합니다
memset(&hints, 0, sizeof hints);
hints.ai_family = AF_UNSPEC; // IPv4와 IPv6 어느 쪽이든 사용
hints.ai_socktype = SOCK_STREAM;
getaddrinfo("www.example.com", "3490", &hints, &res);
sockfd = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
connect(sockfd, res->ai_addr, res->ai_addrlen);

// 좋습니다! 이제 연결되었으니 데이터를 받을 수 있습니다!
byte_count = recv(sockfd, buf, sizeof buf, 0);
printf("recv()'d %d bytes of data in buf\n", byte_count);
```

```
// 데이터그램 소켓과 recvfrom()

struct addrinfo hints, *res;
int sockfd;
int byte_count;
socklen_t fromlen;
struct sockaddr_storage addr;
char buf[512];
char ipstr[INET6_ADDRSTRLEN];

// 호스트 정보를 얻고, 소켓을 만들고, 4950번 포트에 바인드합니다
memset(&hints, 0, sizeof hints);
hints.ai_family = AF_UNSPEC; // IPv4와 IPv6 어느 쪽이든 사용
hints.ai_socktype = SOCK_DGRAM;
hints.ai_flags = AI_PASSIVE;
getaddrinfo(NULL, "4950", &hints, &res);
sockfd = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
bind(sockfd, res->ai_addr, res->ai_addrlen);
```

```
// accept()는 필요 없습니다. 그냥 recvfrom()입니다:

fromlen = sizeof addr;
byte_count = recvfrom(sockfd, buf, sizeof buf, 0, (struct sockaddr *)&addr,
    ↪ &fromlen);

printf("recvfrom() 'd %d bytes of data in buf\n", byte_count);
printf("from IP address %s\n",
    inet_ntop(addr.ss_family,
        addr.ss_family == AF_INET?
            (const void *)&((struct sockaddr_in *)&addr)->sin_addr:
            (const void *)&((struct sockaddr_in6 *)&addr)->sin6_addr,
        ipstr, sizeof ipstr));
```

함께 보기

`send()`, `sendto()`, `select()`, `poll()`, 블로킹

9.19 select()

소켓 설명자가 읽기/쓰기를 할 준비가 되었는지 확인합니다

개요

```
#include <sys/select.h>

int select(int n, fd_set *readfds, fd_set *writefds, fd_set *exceptfds,
    struct timeval *timeout);

FD_SET(int fd, fd_set *set);
FD_CLR(int fd, fd_set *set);
FD_ISSET(int fd, fd_set *set);
FD_ZERO(fd_set *set);
```

설명

`select()` 함수는 여러 소켓을 동시에 확인할 방법을 제공합니다. 각 소켓에 `recv()` 할 데이터가 기다리고 있는지, 대기하지 않고 데이터를 `send()` 할 수 있는지, 또는 어떤 예외가 발생했는지 확인할 수 있습니다.

위의 `FD_SET()` 같은 매크로를 사용해서 소켓 설명자 집합을 채웁니다. 집합을 만들고 나면 아래 매개변수 중 하나로 함수에 넘깁니다. 집합 안의 소켓 중 어느 것이 데이터를 `recv()` 할 준비가 되었는지 알고 싶다면 `readfds`, 어느 소켓에 데이터를 `send()` 할 준비가 되었는지 알고 싶다면 `writefds`, 어느 소켓에서 예외(오류)가 발생했는지 알아야 한다면 `exceptfds` 입니다. 이런 이벤트에 관심이 없다면 이 매개변수 중 일부 또는 전부가 `NULL` 일 수 있습니다. `select()` 가 반환된 뒤에는 집합의 값이 바뀌어 어떤 소켓이 읽거나 쓰기 준비가 되었는지, 어떤 소켓에 예외가 있는지를 보여줍니다.

첫 번째 매개변수 `n` 은 가장 큰 번호의 소켓 설명자(그냥 `int` 라는 점을 기억하세요)에 1을 더한 값입니다.

마지막에 있는 `struct timeval` 인 `timeout` 은 `select()` 가 이 집합들을 얼마나 오래 확인할지 알려줍니다. 타임아웃이 지나거나 이벤트가 발생하면, 둘 중 먼저 일어난 시점에 반환합니다. `struct timeval` 에는 두 필드가 있습니다. `tv_sec` 은 초 단위 값이고, 여기에 마이크로초 단위 값인 `tv_usec` 이 더해집니다. (1초는 1,000,000마이크로초입니다.)

도우미 매크로들은 아래 일을 합니다:

매크로	설명
<code>FD_SET(int fd, fd_set *set);</code>	<code>fd</code> 를 <code>set</code> 에 추가합니다.
<code>FD_CLR(int fd, fd_set *set);</code>	<code>fd</code> 를 <code>set</code> 에서 제거합니다.
<code>FD_ISSET(int fd, fd_set *set);</code>	<code>fd</code> 가 <code>set</code> 안에 있으면 참을 반환합니다.
<code>FD_ZERO(fd_set *set);</code>	<code>set</code> 의 모든 항목을 비웁니다.

Linux 사용자를 위한 노트: Linux의 `select()` 는 "읽을 준비가 됐다"고 반환했는데 실제로는 읽을 준비가 되어 있지 않아서, 뒤따르는 `read()` 호출이 블록될 수 있습니다. 수신 소켓에 `O_NONBLOCK` 플래그를 설정해서 이 경우 `EWOULDBLOCK` 오류가 나게 하고, 그 오류가 발생하면 무시하는 방식으로 이 버그를 우회할 수 있습니다. 소켓을 논블로킹으로 설정하는 방법은 `fcntl()` 매뉴얼 페이지를 참고하세요.

반환값

성공하면 집합 안에서 준비된 설명자의 수를 반환합니다. 타임아웃에 도달했다면 `0`, 오류가 발생했다면 `-1` 을 반환하고 `errno` 를 적절히 설정합니다. 또한 어떤 소켓이 준비되었는지를 보여주도록 집합이 수정됩니다.

예제

```
int s1, s2, n, rv;
fd_set readfds;
struct timeval tv;
char buf1[256], buf2[256];

// 이 시점에서 둘 다 서버에 연결되었다고 가정합니다
//s1 = socket(...);
//s2 = socket(...);
//connect(s1, ...)...
//connect(s2, ...)...

// 미리 집합을 비웁니다
FD_ZERO(&readfds);

// 설명자들을 집합에 추가합니다
FD_SET(s1, &readfds);
FD_SET(s2, &readfds);

// s2를 두 번째로 얻었으므로 이것이 "더 큰" 값입니다.
```

```
// select()의 n 매개변수에는 이 값을 사용합니다
n = s2 + 1;

// 둘 중 한 소켓에 recv()할 데이터가 준비될 때까지 기다립니다
// (타임아웃 10.5초)
tv.tv_sec = 10;
tv.tv_usec = 500000;
rv = select(n, &readfds, NULL, NULL, &tv);

if (rv == -1) {
    perror("select"); // select()에서 오류 발생
} else if (rv == 0) {
    printf("Timeout occurred! No data after 10.5 seconds.\n");
} else {
    // 설명자 하나 또는 둘 모두에 데이터가 있습니다
    if (FD_ISSET(s1, &readfds)) {
        recv(s1, buf1, sizeof buf1, 0);
    }
    if (FD_ISSET(s2, &readfds)) {
        recv(s2, buf2, sizeof buf2, 0);
    }
}
}
```

함께 보기

`poll()`

9.20 `setsockopt()`, `getsockopt()`

소켓의 여러 옵션을 설정합니다

개요

```
#include <sys/types.h>
#include <sys/socket.h>

int getsockopt(int s, int level, int optname, void *optval,
               socklen_t *optlen);
int setsockopt(int s, int level, int optname, const void *optval,
               socklen_t optlen);
```

설명

소켓은 꽤 많이 설정할 수 있는 녀석입니다. 사실 너무 많이 설정할 수 있어서 여기에서 전부 다루지는 않을 것입니다. 어차피 시스템에 따라 달라질 가능성도 큼니다. 하지만 기본은 이야기하겠습니다.

분명하겠지만, 이 함수들은 소켓의 특정 옵션을 얻고 설정합니다. Linux 시스템에서는 모든 소켓 정보가 7번 섹션의 `socket` 맨페이지에 있습니다. (이 정보를 모두 얻으려면 "`man 7 socket`"을 입력하세요.)

매개변수를 보자면 `s` 는 이야기하고 있는 소켓이고, `level` 은 `SOL_SOCKET` 으로 설정해야 합니다. 그런 다음 `optname` 을 관심 있는 이름으로 설정합니다. 다시 말하지만 모든 옵션은 맨페이지를 보세요. 여기에는 그중 재미있는 것 몇 가지가 있습니다:

optname	설명
<code>SO_BINDTODEVICE</code>	<code>bind()</code> 로 IP 주소에 바인드하는 대신 이 소켓을 <code>eth0</code> 같은 심볼릭 장치 이름에 바인드합니다. 유닉스에서 장치 이름을 보려면 <code>ifconfig</code> 명령을 입력하세요.
<code>SO_REUSEADDR</code>	이 포트에 이미 활성 리스닝 소켓이 바인드되어 있지 않다면, 다른 소켓도 이 포트에 <code>bind()</code> 할 수 있게 합니다. 서버가 죽은 뒤 다시 시작하려 할 때 보이는 "Address already in use" 오류 메시지를 피할 수 있게 해 줍니다.
<code>SO_BROADCAST</code>	UDP 데이터그램(<code>SOCK_DGRAM</code>) 소켓이 브로드캐스트 주소로 패킷을 보낼 수 있게 합니다. TCP 스트림 소켓에는 아무 일도, <i>아무 일도!!</i> 하지 않습니다! 하하하!

(역자 주: `SO_BINDTODEVICE` 는 Linux 전용 옵션입니다. 최근 Linux에서는 인터페이스 이름을 확인할 때 `ifconfig` 대신 `ip link` 를 흔히 사용합니다.)

`optval` 매개변수는 보통 해당 값을 나타내는 `int` 에 대한 포인터입니다. 불리언의 경우 0은 거짓이고 0이 아니면 참입니다. 여러분 시스템에서 다르지 않은 한, 이것은 절대적 사실입니다. 넘길 매개변수가 없다면 `optval` 은 `NULL` 일 수 있습니다.

마지막 매개변수 `optlen` 은 `optval` 의 길이로 설정해야 합니다. 아마 `sizeof(int)` 일 것이지만 옵션에 따라 달라집니다. `getsockopt()` 의 경우 이것은 `socklen_t` 에 대한 포인터이며, `optval` 에 저장될 객체의 최대 크기를 지정한다는 점에 주목하세요. (버퍼 오버플로를 막기 위해서입니다.) 그리고 `getsockopt()` 는 실제로 설정된 바이트 수를 반영하도록 `optlen` 값을 수정합니다.

경고: 일부 시스템(특히 Sun과 Windows)에서는 옵션이 `int` 가 아니라 `char` 일 수 있으며, 예를 들어 `int` 값 1 대신 문자 값 '1' 로 설정됩니다. 다시 말하지만 더 자세한 정보는 "`man setsockopt`"와 "`man 7 socket`"으로 여러분의 맨페이지를 확인하세요!

반환값

성공하면 0을 반환합니다. 오류가 발생하면 -1을 반환하고 `errno` 를 적절히 설정합니다.

예제

```
int optval;
socklen_t optlen = sizeof optval;
char *optval2;

// 소켓의 SO_REUSEADDR을 참(1)으로 설정합니다:
optval = 1;
setsockopt(s1, SOL_SOCKET, SO_REUSEADDR, &optval, sizeof optval);

// 소켓을 장치 이름에 바인드합니다(모든 시스템에서 동작하지는 않을 수 있습니다):
optval2 = "eth1"; // 길이가 4바이트이므로 아래에서 4:
setsockopt(s2, SOL_SOCKET, SO_BINDTODEVICE, optval2, 4);

// SO_BROADCAST 플래그가 설정되어 있는지 확인합니다:
```

```
getsockopt(s3, SOL_SOCKET, SO_BROADCAST, &optval, &optlen);
if (optval != 0) {
    printf("SO_BROADCAST enabled on s3!\n");
}
```

함께 보기

fcntl()

9.21 send(), sendto()

소켓으로 데이터를 보냅니다

개요

```
#include <sys/types.h>
#include <sys/socket.h>

ssize_t send(int s, const void *buf, size_t len, int flags);
ssize_t sendto(int s, const void *buf, size_t len,
               int flags, const struct sockaddr *to,
               socklen_t tolen);
```

설명

이 함수들은 소켓으로 데이터를 보냅니다. 일반적으로 `send()` 는 TCP `SOCK_STREAM` 연결 소켓에 사용하고, `sendto()` 는 UDP `SOCK_DGRAM` 비연결 데이터그램 소켓에 사용합니다. 비연결 소켓에서는 패킷을 보낼 때마다 목적지를 지정해야 하므로, `sendto()` 의 마지막 매개변수들이 패킷이 어디로 가는지를 정의합니다.

`send()` 와 `sendto()` 에서 `s` 매개변수는 소켓이고, `buf` 는 보내려는 데이터에 대한 포인터이며, `len` 은 보내려는 바이트 수입니다. `flags` 를 사용하면 데이터를 어떻게 보낼지에 대한 추가 정보를 지정할 수 있습니다. “일반” 데이터를 원한다면 `flags` 를 0으로 설정하세요. 흔히 쓰는 플래그 몇 가지는 아래와 같습니다. 더 자세한 내용은 여러분의 로컬 `send()` 맨페이지를 확인하세요:

매크로	설명
<code>MSG_OOB</code>	“out of band” 데이터로 보냅니다. TCP는 이것을 지원하며, 이 데이터가 일반 데이터보다 더 높은 우선순위를 가진다고 수신 시스템에 알리는 방법입니다. 수신자는 <code>SIGURG</code> 신호를 받고, 큐에 있는 일반 데이터를 모두 받기 전에 이 데이터를 받을 수 있습니다. 이 데이터를 라우터를 통해 보내지 않고 로컬에만 둡니다.
<code>MSG_DONTROUTE</code>	나가는 트래픽이 막혀 <code>send()</code> 가 대기하게 될 상황이면 <code>EAGAIN</code> 을 반환하게 합니다. “이번 <code>send()</code> 에 대해서만 논블로킹을 켜다”와 비슷합니다. 더 자세한 내용은 블로킹 절을 보세요.
<code>MSG_DONTWAIT</code>	더 이상 <code>recv()</code> 하지 않는 원격 호스트에 <code>send()</code> 하면 보통 <code>SIGPIPE</code> 신호를 받습니다. 이 플래그를 추가하면 그 신호가 발생하지 않습니다.
<code>MSG_NOSIGNAL</code>	

반환값

실제로 보낸 바이트 수를 반환하거나, 오류가 발생하면 `-1` 을 반환하고 `errno` 를 적절히 설정합니다. 실제로 보낸 바이트 수는 여러분이 보내라고 요청한 수보다 작을 수 있다는 점에 주목하세요! 이것을 처리하는 도우미 함수는 부분 `send()` 처리 절을 보세요.

또한 어느 한쪽에서 소켓을 닫았다면, `send()` 를 호출한 프로세스는 `SIGPIPE` 신호를 받습니다. (`MSG_NOSIGNAL` 플래그와 함께 `send()` 를 호출한 경우는 제외합니다.)

예제

```
int spatula_count = 3490;
char *secret_message = "The Cheese is in The Toaster";

int stream_socket, dgram_socket;
struct sockaddr_in dest;
int temp;

// 먼저 TCP 스트림 소켓:

// 소켓이 만들어지고 연결되었다고 가정합니다
//stream_socket = socket(...)
//connect(stream_socket, ...

// 네트워크 바이트 순서로 변환
temp = htonl(spatula_count);
// 데이터를 일반 방식으로 전송:
send(stream_socket, &temp, sizeof temp, 0);

// 비밀 메시지를 out-of-band로 전송:
send(stream_socket, secret_message, strlen(secret_message)+1,
      MSG_OOB);

// 이제 UDP 데이터그램 소켓:
//getaddrinfo(...)
//dest = ... // "dest"가 목적지 주소를 담고 있다고 가정합니다
//dgram_socket = socket(...)

// 비밀 메시지를 일반 방식으로 전송:
sendto(dgram_socket, secret_message, strlen(secret_message)+1, 0,
       (struct sockaddr*)&dest, sizeof dest);
```

함께 보기

`recv()`, `recvfrom()`

9.22 shutdown()

소켓에서 이후 송수신을 중단합니다

개요

```
#include <sys/socket.h>

int shutdown(int s, int how);
```

설명

됐습니다! 이제 지긋지긋합니다! 이 소켓에서는 더 이상 `send()` 를 허용하지 않지만, 그래도 `recv()` 로 데이터는 받고 싶습니다! 또는 그 반대일 수도 있습니다! 어떻게 해야 할까요?

소켓 설명자를 `close()` 하면, 그 소켓의 읽기와 쓰기 양쪽이 모두 닫히고 소켓 설명자가 해제됩니다. 한쪽만 닫고 싶다면 `shutdown()` 호출을 사용할 수 있습니다.

매개변수를 보자면 `s` 는 당연히 이 동작을 수행할 소켓이고, 어떤 동작을 할지는 `how` 매개변수로 지정할 수 있습니다. `how` 는 이후 `recv()` 를 막는 `SHUT_RD`, 이후 `send()` 를 금지하는 `SHUT_WR`, 또는 둘 다 하는 `SHUT_RDWR` 일 수 있습니다.

`shutdown()` 은 소켓 설명자를 해제하지 않는다는 점에 주목하세요. 완전히 종료한 경우에도 결국 소켓을 `close()` 해야 합니다.

이것은 드물게 쓰는 시스템 호출입니다.

반환값

성공하면 0을 반환합니다. 오류가 발생하면 `-1` 을 반환하고 `errno` 를 적절히 설정합니다.

예제

```
int s = socket(PF_INET, SOCK_STREAM, 0);

// ...여기에서 send() 몇 번과 여러 일을 합니다...

// 이제 끝났으니 더 이상 send()를 허용하지 않습니다:
shutdown(s, SHUT_WR);
```

함께 보기

`close()`

9.23 socket()

소켓 설명자를 할당합니다

개요

```
#include <sys/types.h>
#include <sys/socket.h>

int socket(int domain, int type, int protocol);
```

설명

소켓다운 일을 할 때 사용할 수 있는 새 소켓 설명자를 반환합니다. 이것은 보통 소켓 프로그램을 작성하는 거대한 과정에서 첫 번째 호출이며, 그 결과를 이후 `listen()`, `bind()`, `accept()` 또는 여러 다른 함수 호출에 사용할 수 있습니다.

일반적인 사용에서는 아래 예제처럼 `getaddrinfo()` 호출에서 이 매개변수들의 값을 얻습니다. 하지만 정말 원한다면 직접 채울 수도 있습니다.

매개변수 설명

domain domain 은 여러분이 관심 있는 소켓의 종류를 설명합니다. 믿어 주세요, 이것에는 정말 다양한 값이 올 수 있습니다. 하지만 이것은 소켓 안내서이므로 IPv4에는 `PF_INET`, IPv6에는 `PF_INET6` 이 될 것입니다.

type type 매개변수에도 여러 값이 올 수 있지만, 아마 신뢰성 있는 TCP 소켓(`send()`, `recv()`)을 위한 `SOCK_STREAM` 이나, 신뢰성은 없지만 빠른 UDP 소켓(`sendto()`, `recvfrom()`)을 위한 `SOCK_DGRAM` 중 하나로 설정할 것입니다. (또 다른 흥미로운 소켓 타입으로는 패킷을 직접 구성할 때 쓸 수 있는 `SOCK_RAW` 가 있습니다. 꽤 멋집니다.)

protocol protocol 매개변수는 특정 소켓 타입에 어떤 프로토콜을 사용할지 알려줍니다. 이미 말했듯이 예를 들어 `SOCK_STREAM` 은 TCP를 사용합니다. 다행히 `SOCK_STREAM` 이나 `SOCK_DGRAM` 을 쓸 때는 프로토콜을 그냥 0으로 설정하면 적절한 프로토콜을 자동으로 사용합니다. 그렇지 않다면 `getprotobyne()` 으로 적절한 프로토콜 번호를 찾을 수 있습니다.

반환값

이후 호출에서 사용할 새 소켓 설명자를 반환합니다. 오류가 발생하면 `-1` 을 반환하고 `errno` 를 적절히 설정합니다.

예제

```
struct addrinfo hints, *res;
int sockfd;

// 먼저 getaddrinfo()로 주소 구조체 목록을 가져옵니다:

memset(&hints, 0, sizeof hints);
hints.ai_family = AF_UNSPEC; // AF_INET, AF_INET6, 또는 AF_UNSPEC
hints.ai_socktype = SOCK_STREAM; // SOCK_STREAM 또는 SOCK_DGRAM

getaddrinfo("www.example.com", "3490", &hints, &res);

// getaddrinfo()에서 얻은 정보를 사용해 소켓을 만듭니다:
sockfd = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
```

함께 보기

`accept()`, `bind()`, `getaddrinfo()`, `listen()`

9.24 struct sockaddr 와 친구들

인터넷 주소를 다루는 구조체들

개요

```
#include <netinet/in.h>

// 소켓 주소 구조체에 대한 모든 포인터는 여러 함수와 시스템 호출에서
// 사용되기 전에 이 타입에 대한 포인터로 형변환되는 경우가 많습니다:

struct sockaddr {
    unsigned short    sa_family;    // 주소 계열, AF_xxx
    char              sa_data[14];  // 14바이트 프로토콜 주소
};

// IPv4 AF_INET 소켓:

struct sockaddr_in {
    short             sin_family;    // 예: AF_INET
    unsigned short    sin_port;     // 예: htons(3490)
    struct in_addr    sin_addr;     // 아래 struct in_addr를 보세요
    char              sin_zero[8];  // 원한다면 0으로 만드세요
};

struct in_addr {
    unsigned long     s_addr;        // inet_pton()으로 채웁니다
};

// IPv6 AF_INET6 소켓:

struct sockaddr_in6 {
    u_int16_t         sin6_family;  // 주소 계열, AF_INET6
    u_int16_t         sin6_port;    // 포트 번호, 네트워크 바이트 순서
    u_int32_t         sin6_flowinfo; // IPv6 흐름 정보
    struct in6_addr    sin6_addr;   // IPv6 주소
    u_int32_t         sin6_scope_id; // 스코프 ID
};

struct in6_addr {
    unsigned char      s6_addr[16]; // inet_pton()으로 채웁니다
};

// struct sockaddr_in 또는 struct sockaddr_in6 데이터를 담기에 충분히 큰
```

```
// 일반 소켓 주소 보관 구조체:

struct sockaddr_storage {
    sa_family_t  ss_family;    // 주소 계열

    // 이것들은 모두 패딩이고 구현에 따라 다릅니다. 무시하세요:
    char         __ss_pad1[_SS_PAD1SIZE];
    int64_t      __ss_align;
    char         __ss_pad2[_SS_PAD2SIZE];
};
```

설명

이것들은 인터넷 주소를 다루는 모든 시스템 호출과 함수의 기본 구조체입니다. 보통은 `getaddrinfo()` 를 사용해 이 구조체들을 채우고, 필요할 때 읽게 될 것입니다.

메모리에서 `struct sockaddr_in` 과 `struct sockaddr_in6` 은 `struct sockaddr` 와 같은 시작 구조를 공유합니다. 그래서 우주가 끝날 가능성만 빼면, 한 타입의 포인터를 다른 타입으로 자유롭게 형변환해도 해가 없습니다.

우주가 끝난다는 건 농담입니다... `struct sockaddr_in*` 을 `struct sockaddr*` 로 형변환했을 때 정말 우주가 끝난다면, 그건 순전히 우연일 뿐이니 걱정하지 않아도 된다고 약속드립니다.

그러니 이 점을 염두에 두고, 어떤 함수가 `struct sockaddr*` 를 받는다고 할 때마다 여러분의 `struct sockaddr_in*`, `struct sockaddr_in6*`, 또는 `struct sockaddr_storage*` 를 그 타입으로 쉽고 안전하게 형변환할 수 있음을 기억하세요.

`struct sockaddr_in` 은 IPv4 주소(예: "192.0.2.10")에 사용하는 구조체입니다. 여기에는 주소 계열(`AF_INET`), `sin_port` 안의 포트, `sin_addr` 안의 IPv4 주소가 들어갑니다.

`struct sockaddr_in` 에는 `sin_zero` 필드도 있는데, 어떤 사람들은 이것을 반드시 0으로 설정해야 한다고 주장합니다. 다른 사람들은 이것에 대해 아무 주장도 하지 않고(Linux 문서에서는 아예 언급하지도 않습니다), 실제로 0으로 설정할 필요가 있어 보이지도 않습니다. 그러니 마음이 내키면 `memset()` 으로 0으로 설정하세요.

이제, 저 `struct in_addr` 는 시스템에 따라 이상한 녀석입니다. 때로는 온갖 `#define` 과 여러 잡동사니가 붙은 정신없는 `union` 입니다. 하지만 여러분이 해야 할 일은 이 구조체의 `s_addr` 필드만 사용하는 것입니다. 많은 시스템이 그것 하나만 구현하기 때문입니다.

`struct sockaddr_in6` 과 `struct in6_addr` 도 매우 비슷하지만, IPv6에 사용된다는 점이 다릅니다.

`struct sockaddr_storage` 는 IP 버전에 무관한 코드를 작성하려고 할 때, 그리고 새 주소가 IPv4일지 IPv6일지 모를 때 `accept()` 나 `recvfrom()` 에 넘길 수 있는 구조체입니다. `struct sockaddr_storage` 구조체는 원래의 작은 `struct sockaddr` 와 달리 두 타입을 모두 담을 만큼 큼니다.

예제

```
// IPv4:

struct sockaddr_in ip4addr;
int s;
```

```
ip4addr.sin_family = AF_INET;
ip4addr.sin_port = htons(3490);
inet_pton(AF_INET, "10.0.0.1", &ip4addr.sin_addr);

s = socket(PF_INET, SOCK_STREAM, 0);
bind(s, (struct sockaddr*)&ip4addr, sizeof ip4addr);
```

```
// IPv6:

struct sockaddr_in6 ip6addr;
int s;

ip6addr.sin6_family = AF_INET6;
ip6addr.sin6_port = htons(4950);
inet_pton(AF_INET6, "2001:db8:8714:3a90::12", &ip6addr.sin6_addr);

s = socket(PF_INET6, SOCK_STREAM, 0);
bind(s, (struct sockaddr*)&ip6addr, sizeof ip6addr);
```

함께 보기

`accept()`, `bind()`, `connect()`, `inet_aton()`, `inet_ntoa()`

Chapter 10

더 많은 참고문헌

여기까지 왔고, 이제 더 많은 것을 원하시나요? 이 모든 것에 대해 더 배우려면 어디로 가야 할까요?

10.1 책들

옛날식으로, 정말 손에 들 수 있는 종이책을 원한다면 아래의 훌륭한 책들을 살펴보세요. 이 링크들은 유명 서점의 제휴 링크로 연결되며, 저에게 소소한 수수료를 안겨 줍니다. 그냥 너그러운 마음이 든다면 PayPal로 beej@beej.us 에 기부해 주셔도 됩니다. :-)

Unix Network Programming, volumes 1-2 W. Richard Stevens 지음. Addison-Wesley Professional와 Prentice Hall 출판. 1~2권 ISBN: 978-0131411555¹, 978-0130810816².

Internetworking with TCP/IP, volume I Douglas E. Comer 지음. Pearson 출판. ISBN 978-0136085300³.

TCP/IP Illustrated, volumes 1-3 W. Richard Stevens와 Gary R. Wright 지음. Addison Wesley 출판. 1~3권 ISBN(그리고 3권 통합본 ISBN): 978-0201633467⁴, 978-0201633542⁵, 978-0201634952⁶, (978-0201776317⁷).

TCP/IP Network Administration Craig Hunt 지음. O'Reilly & Associates, Inc. 출판. ISBN 978-0596002978⁸.

Advanced Programming in the UNIX Environment W. Richard Stevens 지음. Addison Wesley 출판. ISBN 978-0321637734⁹.

10.2 웹 참고문헌

인터넷에는 아래와 같은 정보가 있습니다.

BSD Sockets: A Quick And Dirty Primer¹⁰ (유닉스 시스템 프로그래밍 정보도 있습니다!)

¹<https://beej.us/guide/url/unixnet1>

²<https://beej.us/guide/url/unixnet2>

³<https://beej.us/guide/url/intertcp1>

⁴<https://beej.us/guide/url/tcp1>

⁵<https://beej.us/guide/url/tcp2>

⁶<https://beej.us/guide/url/tcp3>

⁷<https://beej.us/guide/url/tcp123>

⁸<https://beej.us/guide/url/tcpna>

⁹<https://beej.us/guide/url/advunix>

¹⁰<https://cis.temple.edu/~giorgio/old/cis307s96/readings/docs/sockets.html>

The Unix Socket FAQ¹¹

TCP/IP FAQ¹²

The Winsock FAQ¹³

그리고 관련된 위키백과 페이지도 있습니다.

Berkeley Sockets¹⁴

Internet Protocol (IP)¹⁵

Transmission Control Protocol (TCP)¹⁶

User Datagram Protocol (UDP)¹⁷

Client-Server¹⁸

Serialization¹⁹ (데이터 패키징과 언패킹)

10.3 RFCs

RFC²⁰—진짜 알짜배기입니다! 이것들은 인터넷에서 쓰이는 할당 번호와 프로그래밍 API, 프로토콜 등을 설명하는 문서입니다. 여러분의 즐거움을 위해 그중 일부에 대한 링크를 포함했습니다. 팝콘 한 통을 챙기고, 머리를 굴릴 준비를 해 봅시다.

(역자 주: 아래 목록에는 현재 표준에서 교체되었거나 폐기된 RFC도 있으므로, 실제 구현의 기준으로 삼을 때는 RFC Editor에서 문서의 현재 상태를 확인하세요.)

RFC 1²¹—최초의 RFC입니다. 이 문서는 “인터넷”이 막 태어나던 시점에 어떤 모습이었는지, 그리고 인터넷이 밑바닥부터 어떻게 설계되고 있었는지 엿볼 수 있게 해 줍니다. (물론 이 RFC는 완전히 폐기되었습니다!)

RFC 768²²—사용자 데이터그램 프로토콜(UDP)

RFC 791²³—인터넷 프로토콜(IP)

RFC 793²⁴—전송 제어 프로토콜(TCP)

RFC 854²⁵—Telnet 프로토콜

RFC 959²⁶—파일 전송 프로토콜(FTP)

RFC 1350²⁷—간이 파일 전송 프로토콜(TFTP)

RFC 1459²⁸—인터넷 릴레이 채팅 프로토콜(IRC)

¹¹<https://developerweb.net/?f=70>

¹²<http://www.faqs.org/faqs/internet/tcp-ip/tcp-ip-faq/part1/>

¹³<https://tangentsoft.net/wskfaq/>

¹⁴https://en.wikipedia.org/wiki/Berkeley_sockets

¹⁵https://en.wikipedia.org/wiki/Internet_Protocol

¹⁶https://en.wikipedia.org/wiki/Transmission_Control_Protocol

¹⁷https://en.wikipedia.org/wiki/User_Datagram_Protocol

¹⁸<https://en.wikipedia.org/wiki/Client-server>

¹⁹<https://en.wikipedia.org/wiki/Serialization>

²⁰<https://www.rfc-editor.org/>

²¹<https://tools.ietf.org/html/rfc1>

²²<https://tools.ietf.org/html/rfc768>

²³<https://tools.ietf.org/html/rfc791>

²⁴<https://tools.ietf.org/html/rfc793>

²⁵<https://tools.ietf.org/html/rfc854>

²⁶<https://tools.ietf.org/html/rfc959>

²⁷<https://tools.ietf.org/html/rfc1350>

²⁸<https://tools.ietf.org/html/rfc1459>

RFC 1918²⁹ —사설 인터넷을 위한 주소 할당
RFC 2131³⁰ —동적 호스트 구성 프로토콜(DHCP)
RFC 9110³¹ —하이퍼텍스트 전송 프로토콜(HTTP)
RFC 2821³² —단순 메일 전송 프로토콜(SMTP)
RFC 3330³³ —특수 용도 IPv4 주소
RFC 3493³⁴ —IPv6을 위한 기본 소켓 인터페이스 확장
RFC 3542³⁵ —IPv6을 위한 고급 소켓 응용프로그램 인터페이스(API)
RFC 3849³⁶ —문서화를 위해 예약된 IPv6 주소 접두사
RFC 3920³⁷ —확장 가능 메시징 및 프레즌스 프로토콜(XMPP)
RFC 3977³⁸ —네트워크 뉴스 전송 프로토콜(NNTP)
RFC 4193³⁹ —고유 로컬 IPv6 유니캐스트 주소
RFC 4506⁴⁰ —외부 데이터 표현 표준(XDR)
IETF에는 RFC를 찾고 살펴보기 위한 좋은 도구⁴¹가 있습니다.

²⁹<https://tools.ietf.org/html/rfc1918>

³⁰<https://tools.ietf.org/html/rfc2131>

³¹<https://tools.ietf.org/html/rfc9110>

³²<https://tools.ietf.org/html/rfc2821>

³³<https://tools.ietf.org/html/rfc3330>

³⁴<https://tools.ietf.org/html/rfc3493>

³⁵<https://tools.ietf.org/html/rfc3542>

³⁶<https://tools.ietf.org/html/rfc3849>

³⁷<https://tools.ietf.org/html/rfc3920>

³⁸<https://tools.ietf.org/html/rfc3977>

³⁹<https://tools.ietf.org/html/rfc4193>

⁴⁰<https://tools.ietf.org/html/rfc4506>

⁴¹<https://tools.ietf.org/rfc/>

Index

10.x.x.x, 21
192.168.x.x, 21
255.255.255.255, 84, 115

accept() function, 33, 95
Address already in use, 31, 89
AF_INET macro, 18, 29, 92
AF_INET6 macro, 18

Bapper, 86
bind() function, 30, 31, 89, 97
 implicit, 33
Blocking, 35, 51–52
Broadcast, 84
BSD, 2
Byte ordering, 15, 18, 69, 113

Client
 datagram, 49–50
 stream, 45–47
Client/Server, 41–50
close() function, 37, 100
closesocket() function, 4, 38, 100
Compilers
 GCC, 1
Compression, 92
connect(), 30
 on datagram sockets, 99
connect() function, 9, 32, 99
 on datagram sockets, 37, 50
Connection refused, 47
CreateProcess() function, 4, 93
CreateThread() function, 4
CSocket class, 4
Cygwin, 3

Data encapsulation
 header, 11
Data encapsulation, 11, 68
 footer, 11
Datagram socket, 10
Datagram sockets, 9
DHCP, 137

EAGAIN macro, 51, 128
Emailing Beej, 4

Encryption, 92
EPIPE macro, 100
errno variable, 110, 119
Ethernet, 11
EWOULDBLOCK macro, 51
Excalibur, 84

F_SETFL macro, 112
fcntl() function, 51, 96, 111
FD_CLR() macro, 60, 125
FD_ISSET() macro, 60, 125
FD_SET() macro, 60, 125
FD_ZERO() macro, 60, 125
File descriptor, 9
Firewall, 21, 86, 94
 poking holes in, 94
fork() function, 4, 41, 93
freeaddrinfo() function, 101
FTP, 136

gai_strerror() function, 101
getaddrinfo() function, 17, 23, 25, 38, 101
gethostbyaddr() function, 38, 105
gethostbyname() function, 105, 105
gethostname() function, 38, 104
getnameinfo() function, 24, 38, 108
getpeername() function, 38, 109
getprotobyname() function, 131
getsockopt() function, 126
gettimeofday() function, 62
Goat, 89
goto statement, 90

Header files, 89
herror() function, 106
hstrerror() function, 106
htonl() function, 16, 112
htons() function, 16, 18, 69, 112
HTTP protocol, 10, 137

ICMP, 90
IEEE-754, 70
illumos, 2
INADDR_BROADCAST macro, 84
inet_addr() function, 20, 114
inet_aton() function, 20, 114

- inet_ntoa() function, 20, 114
- inet_ntop() function, 20, 38, 115
- inet_pton() function, 20, 115
- ioctl() function, 94
- IP, 11, 136
- IP address, 13, 19, 30, 36, 38
- ip route command, 89
- IPv4, 13
- IPv6, 13, 19, 21, 23
- IRC, 68, 136
- ISO/OSI, 11

- Layered network model, 11
- Linux, 2
- listen() function, 30, 33, 117
 - backlog, 33
 - with select(), 62
- localhost, 89
- Loopback device, 89

- man pages, 95
- MSG_DONTROUTE macro, 128
- MSG_DONTWAIT macro, 128
- MSG_NOSIGNAL macro, 128
- MSG_OOB macro, 122, 128
- MSG_PEEK macro, 122
- MSG_WAITALL macro, 122
- MTU, 93

- NAT, 21
- netstat command, 89
- NNTP, 137
- Non-blocking sockets, 51, 128
- ntohl() function, 16, 112
- ntohs() function, 16, 112

- O_ASYNC macro, 112
- O_NONBLOCK macro, 67, 96, 112, 125
- OpenSSL, 92
- Out-of-band data, 122, 128

- Packet sniffer, 94
- Pat, 86
- perror() function, 111, 118
- PF_INET macro, 92, 131
- ping command, 90
- poll(), 52–59
- poll() function, 52, 67, 120
- Port, 30, 31, 36
- Private network, 21
- Promiscuous mode, 94

- Raw sockets, 9, 90
- read() function, 9
- recv() function, 9, 36, 122
 - timeout, 91
- recvfrom() function, 36, 122
- recvtimeout() function, 92
- References
 - books, 135
 - FRFCs, 136–137
 - web-based, 135–136
- RFCs, 136–137
- route command, 89

- SA_RESTART macro, 90
- Security, 93
- select() function, 4, 89, 91, 124
 - with listen(), 62
- select() 함수, 59–67
- send() function, 9, 12, 35, 128
- sendall() function, 82
- sendall() 함수, 67–68
- sendto() function, 12, 128
- Serialization, 68–82
- Server
 - datagram, 47–49
 - stream, 41–44
- setsockopt() function, 31, 84, 89, 94, 126
- SHUT_RD macro, 130
- SHUT_RDWR macro, 130
- SHUT_WR macro, 130
- shutdown() function, 37, 129
- sigaction() function, 44, 90
- SIGIO signal, 112
- SIGPIPE macro, 100, 128
- SIGURG macro, 122, 128
- SMTP, 137
- SO_BINDTODEVICE macro, 127
- SO_BROADCAST macro, 84, 127
- SO_RCVTIMEO macro, 94
- SO_REUSEADDR macro, 31, 89, 127
- SO_SNDTIMEO macro, 94
- SOCK_DGRAM macro, 9, 11, 36, 122, 131
- SOCK_RAW macro, 90, 131
- SOCK_STREAM macro, 9, 122, 131
- Socket descriptor, 9, 17
- socket() function, 9, 29, 130
- SOL_SOCKET macro, 127
- Solaris, 2, 127
- SSL, 92
- Stream sockets, 9
- strerror() function, 111, 118
- struct addrinfo type, 17
- struct hostent type, 106
- struct in6_addr type, 132
- struct in_addr type, 132
- struct pollfd type, 52, 120
- struct sockaddr type, 17, 18, 37, 122, 132

- struct sockaddr_in type, 132
- struct sockaddr_in6 type, 132
- struct sockaddr_storage type, 132
- struct timeval type, 125
- struct timeval 형, 60–62
- SunOS, 2, 127

- TCP, 10, 136
- telnet, 10, 136
- TFTP, 11, 136
- Timeout
 - setting, 94
- Translating the Guide, 5
- TRON, 32

- UDP, 10, 11, 84, 136

- Vint Cerf, 13

- Windows, 2, 38, 89, 100, 127
- Windows Subsystem For Linux, 2
- Winsock, 3, 38
- write() function, 9
- WSACleanup() function, 3
- WSAStartup() function, 3
- WSL, 2

- XDR, 82, 137
- XMPP, 137